

+

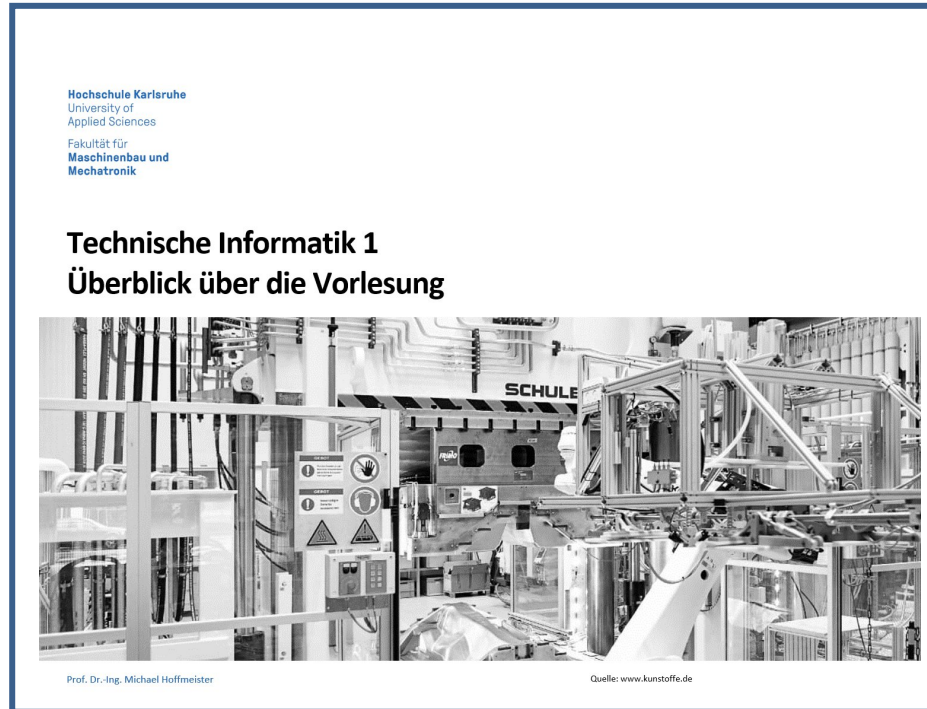
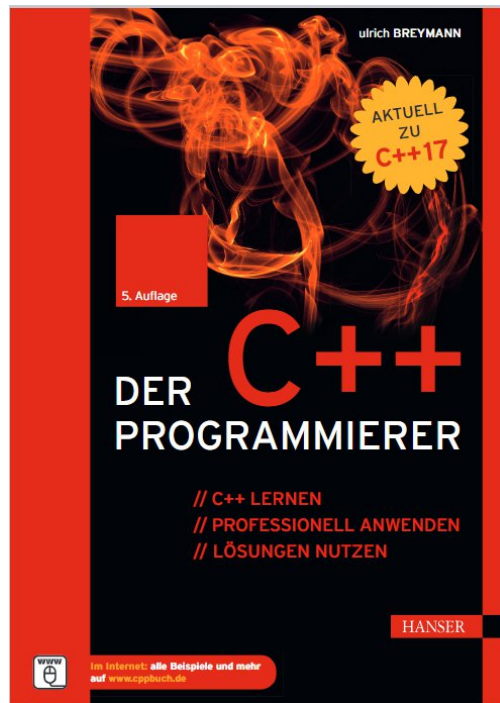
Informatik 2 – Dynamische Speicherverwaltung & Datenstrukturen

K

A



©image: various



Prof. Dr.-Ing. Michael Hoffmeister

Vorlesung Machine Learning

Trees

Prof. Dr.-Ing. Steffen Schober; Dozent: Dr. Alexander Hanuschkin
WS 22/23

Hochschule Esslingen

Grundlagen der Künstlichen Intelligenz (6/6) Monte Carlo Tree Search



Dr. Christian Meilicke, Research Group Data and Web Science



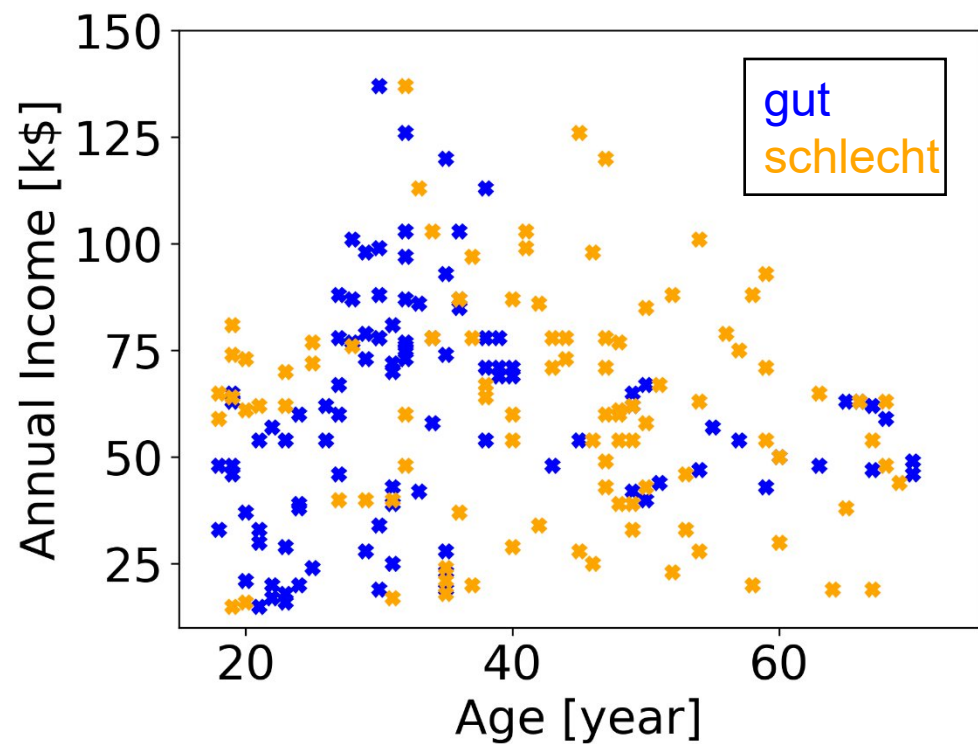
Algorithmen

- Entscheidungsbäume (CART)
- Monte Carlo Tree Search (MCTS)
- Dijkstra's algorithm

C++

- Dynamische Speicherverwaltung
(new, delete)
- Datenstrukturen
(verkettete Listen, Stapel, etc.)

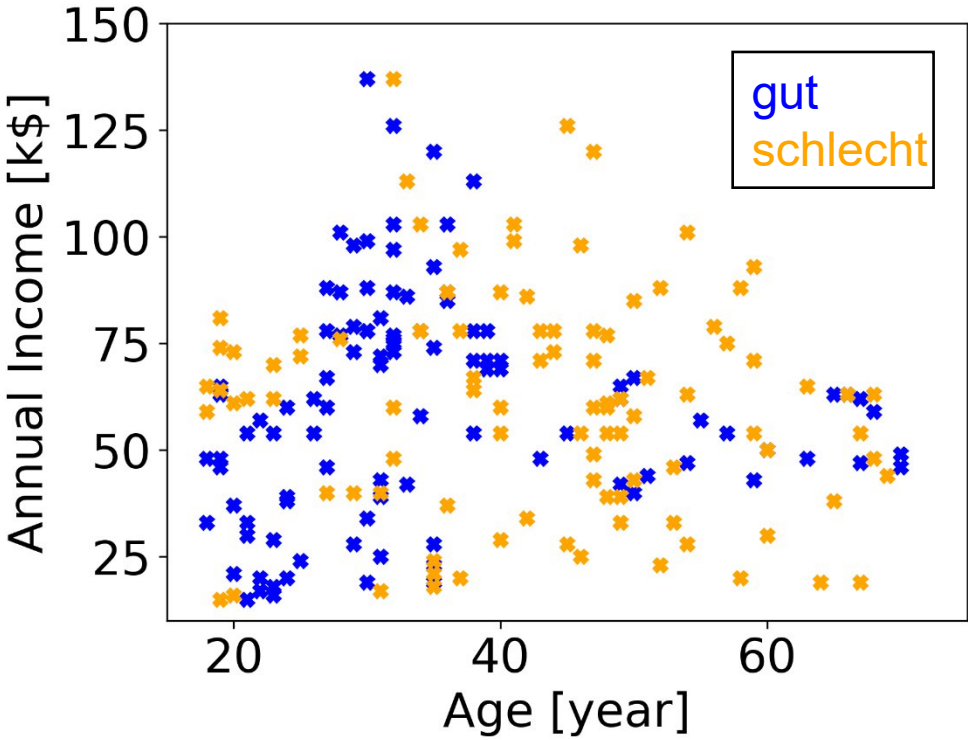
Entscheidungsbäume



CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
1	Male	19	15	schlecht
2	Male	21	15	gut
3	Female	20	16	schlecht
4	Female	23	16	gut
5	Female	31	17	schlecht

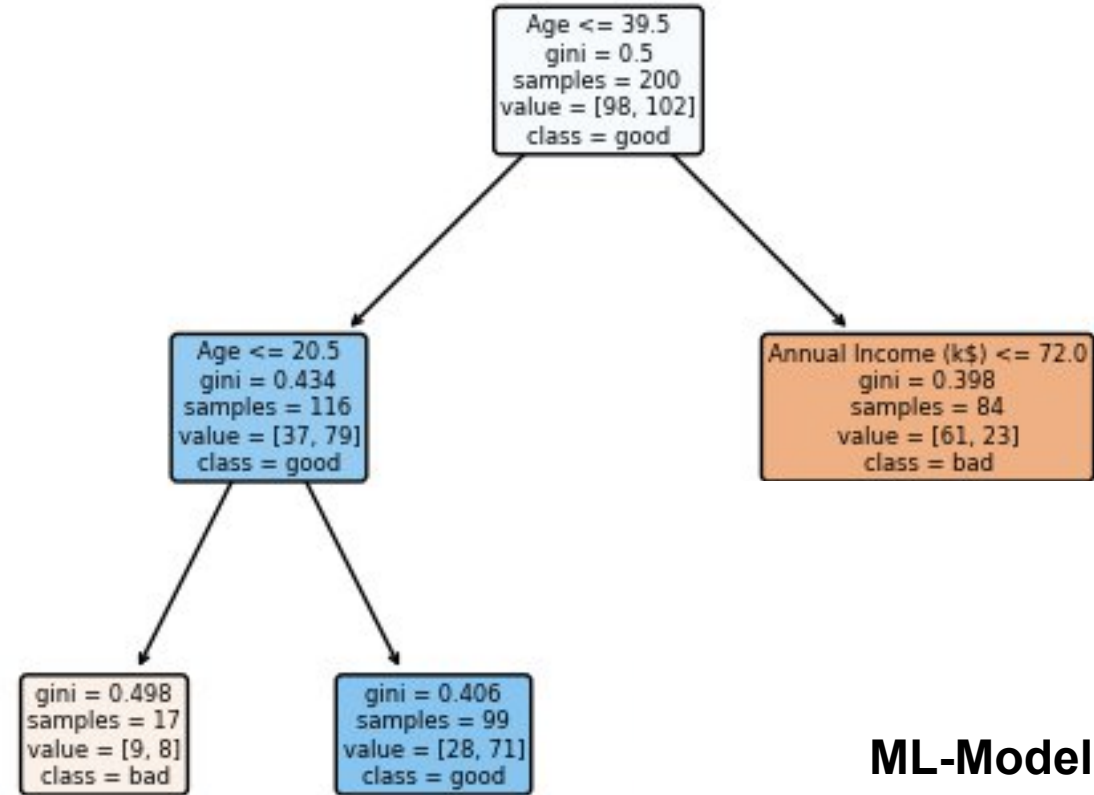
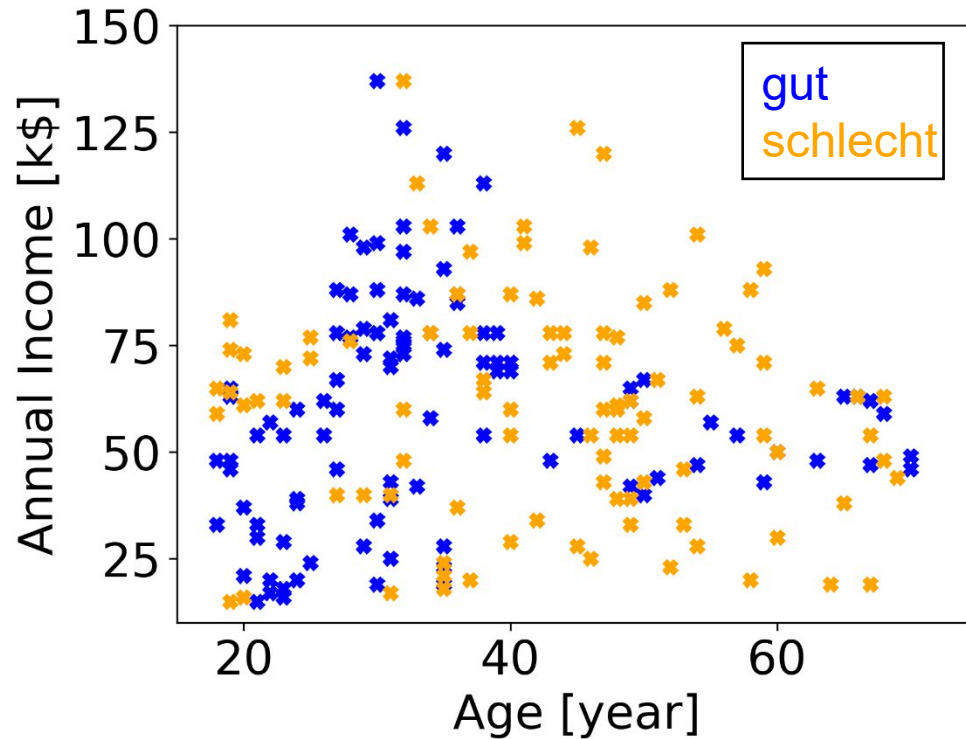


Wie können wir vorhersagen, welcher neue Kunde viel Geld ausgeben wird?



CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
1	Male	19	15	schlecht
2	Male	21	15	gut
3	Female	20	16	schlecht
4	Female	23	16	gut
5	Female	31	17	schlecht

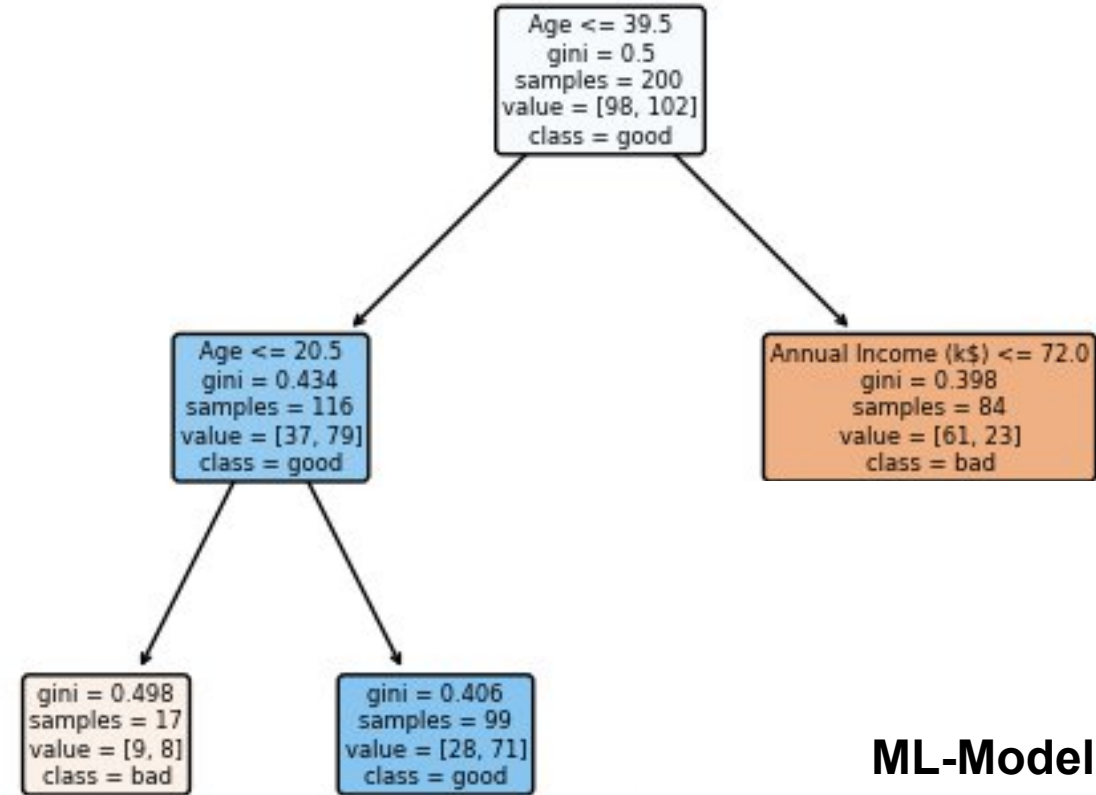
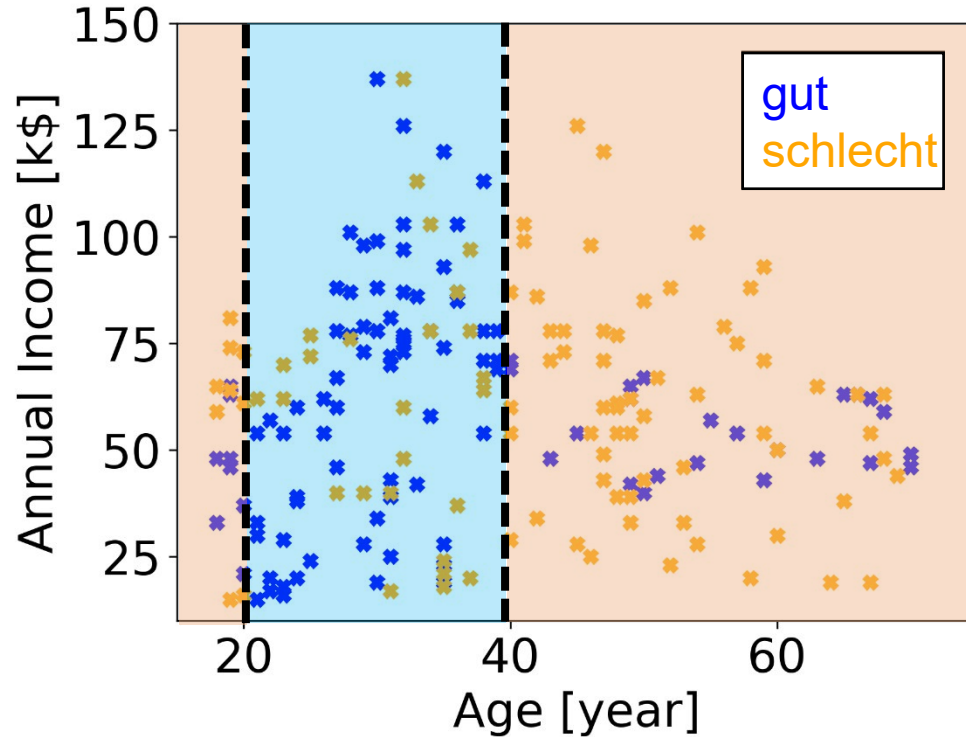
Entscheidungsbäume



ML-Modell

```
from sklearn import tree
clf = tree.DecisionTreeClassifier(max_depth=2)
clf = clf.fit(X_OneHot, y_label)
```

Entscheidungsbäume



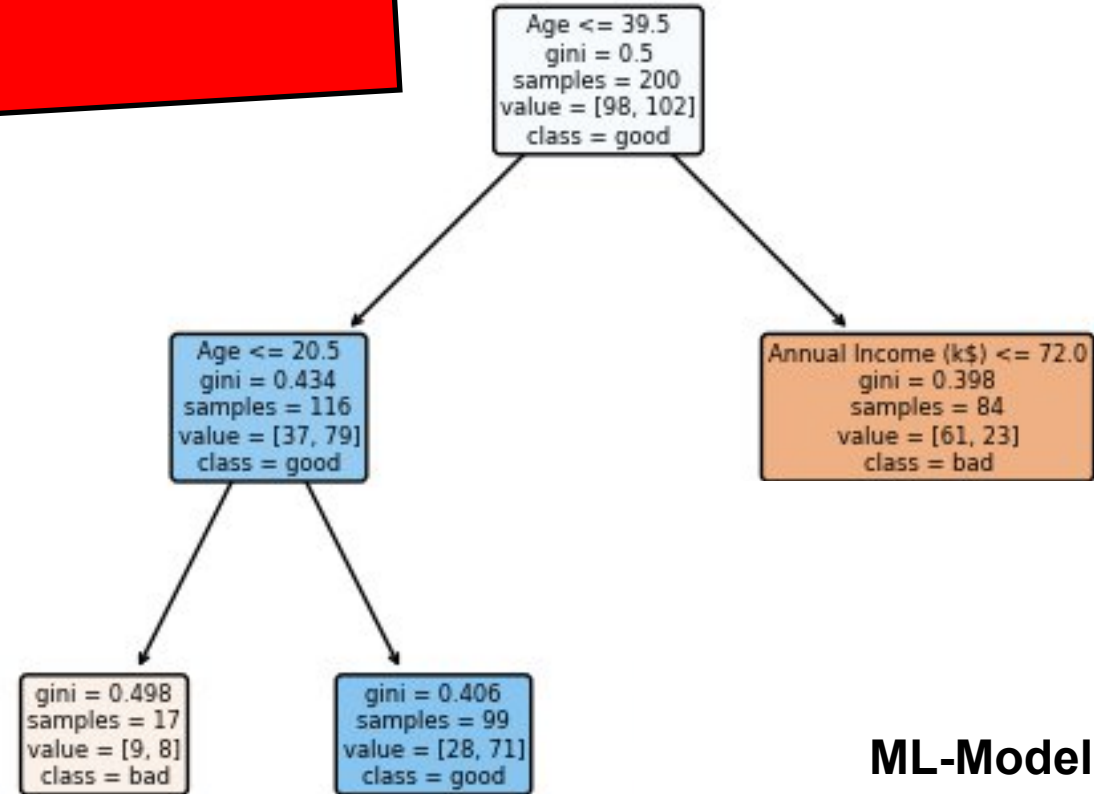
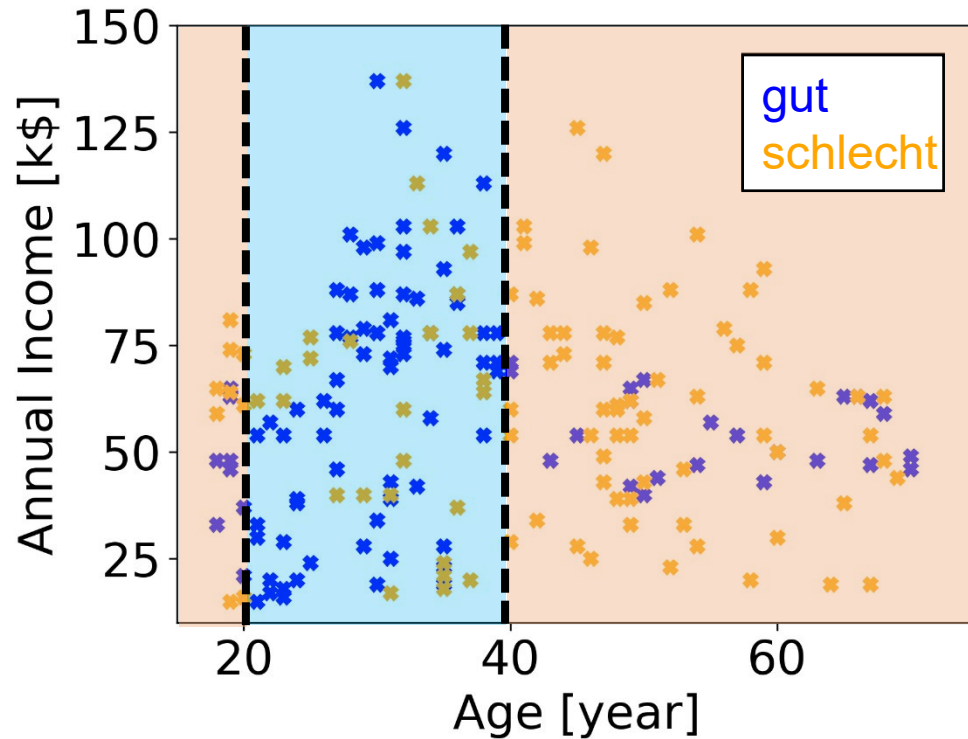
ML-Modell

```
from sklearn import tree
clf = tree.DecisionTreeClassifier(max_depth=2)
clf = clf.fit(X_OneHot, y_label)
```

Entscheidungsbäume

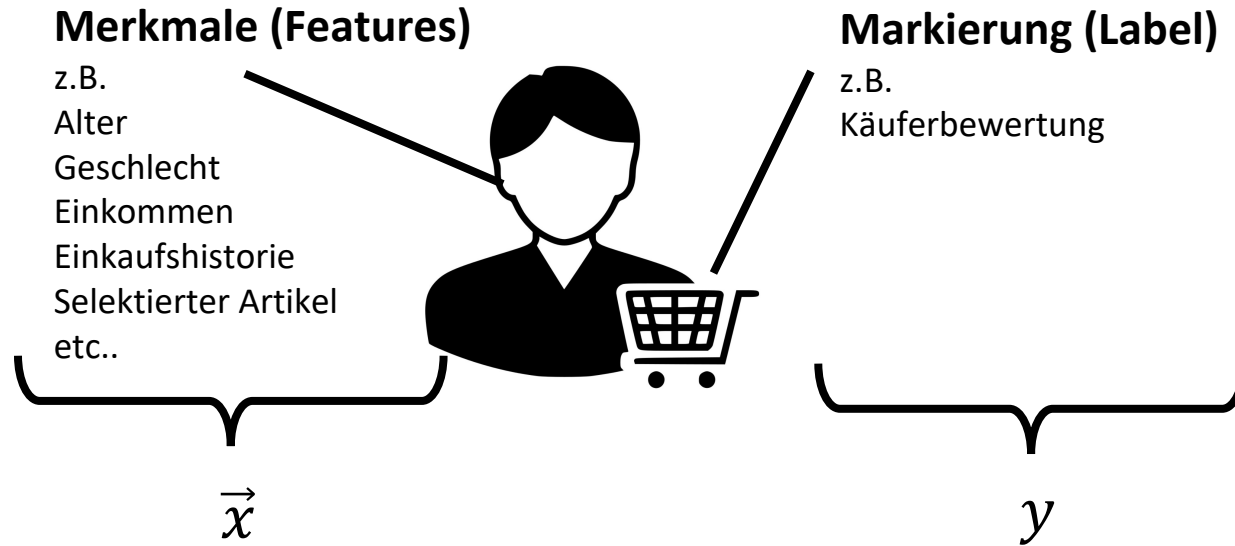


Aber wie wird so ein ML-Modell trainiert?



```
from sklearn import tree
clf = tree.DecisionTreeClassifier(max_depth=2)
clf = clf.fit(X_OneHot, y_label)
```

Supervised learning

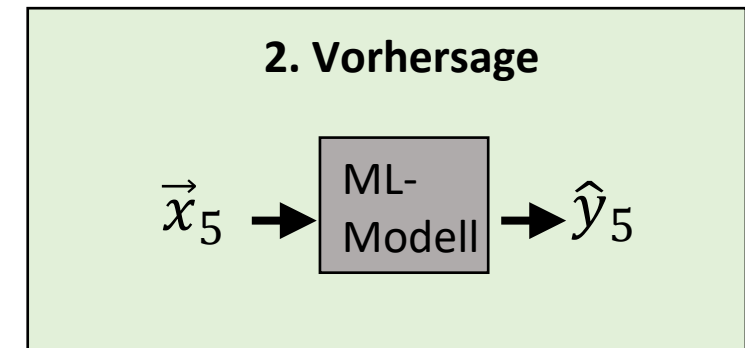
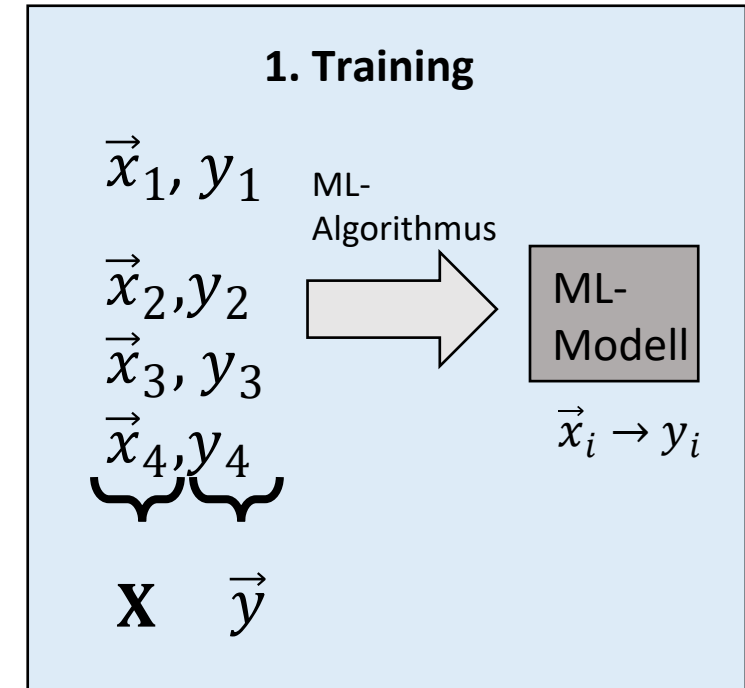


1. Training

Mit Lernbeispielen (Samples) wird ein ML-Modell antrainiert. Beispiele bestehen aus Merkmal-Markierungspaaren $(\vec{x}_i, y_i)$. Das ML-Modell lernt aus $\vec{x}_i$ die Markierungen y_i vorherzusagen.

2. Vorhersage

Das ML-Modell kann aus Merkmalswerten $\vec{x}_i$ eine(n) Klasse/Wert $\hat{y}_i$ vorhersagen.



Supervised learning



Merkmale (Features)

z.B.
Alter
Geschlecht
Einkommen
Einkaufshistorie
Selektierter Artikel
etc..

Markierung (Label)

z.B.
Käuferbewertung



$\vec{x}$

y

CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
1	Male	19	15	schlecht
2	Male	21	15	gut
3	Female	20	16	schlecht
4	Female	23	16	gut
5	Female	31	17	schlecht

1. Training

$\vec{x}_1, y_1$

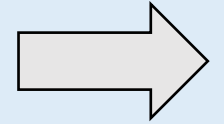
$\vec{x}_2, y_2$

$\vec{x}_3, y_3$

$\vec{x}_4, y_4$

$\mathbf{X} \quad \vec{y}$

ML-
Algorithmus



ML-
Modell

$\vec{x}_i \rightarrow y_i$

2. Vorhersage

$\vec{x}_5$



ML-
Modell

$\hat{y}_5$



Supervised learning



Merkmale (Features)

z.B.
Alter
Geschlecht
Einkommen
Einkaufshistorie
Selektierter Artikel
etc..

Markierung (Label)

z.B.
Käuferbewertung

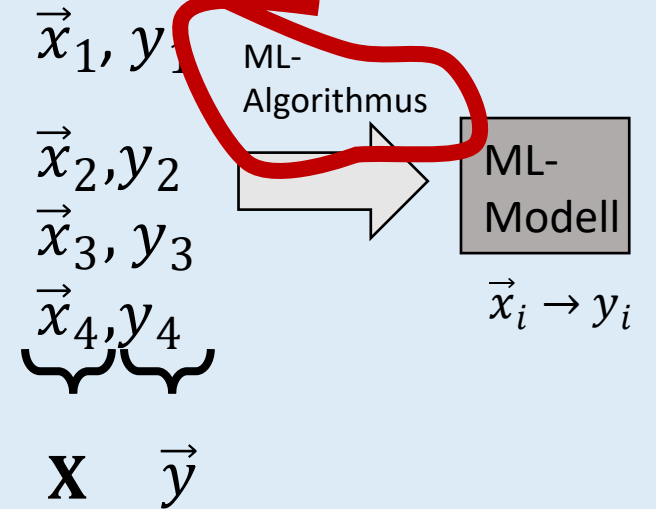


$\vec{x}$

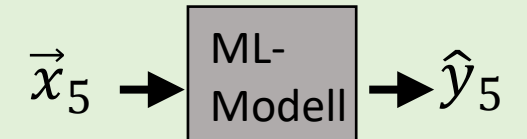
y

CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
1	Male	19	15	schlecht
2	Male	21	15	gut
3	Female	20	16	schlecht
4	Female	23	16	gut
5	Female	31	17	schlecht

1. Training



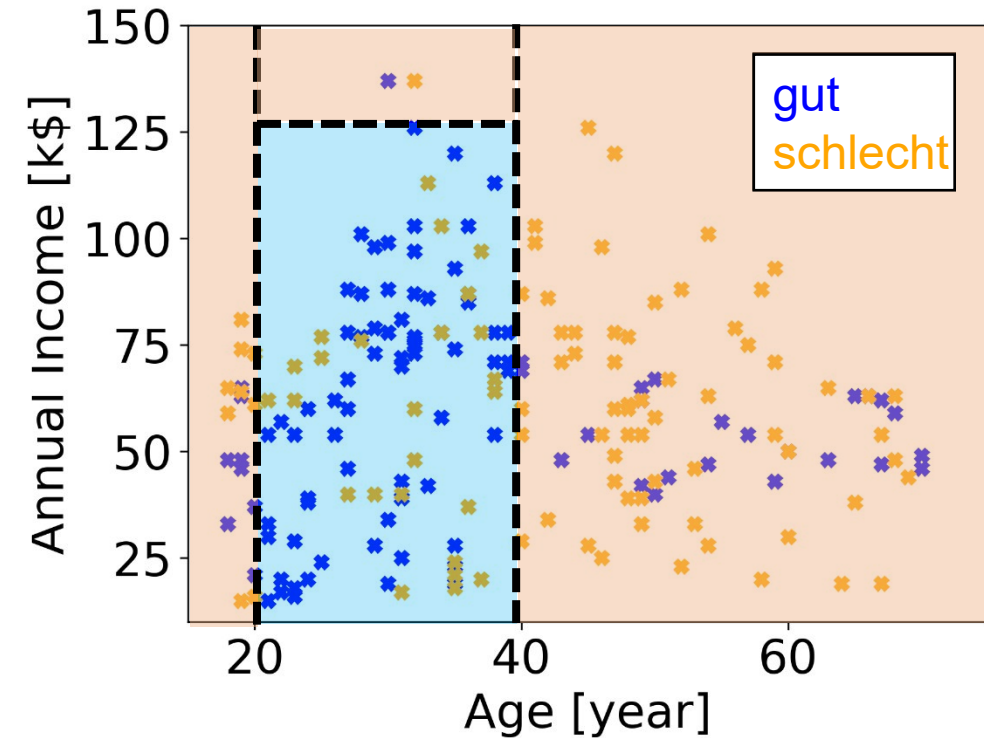
2. Vorhersage



CART Algorithmus

Idea: Tree-based models partition the feature space into a set of **rectangles** and to each one we fit a **simple model** which we use to predict the target value, for example:

- Regression: constant value
- Classification: a single class values, or probability

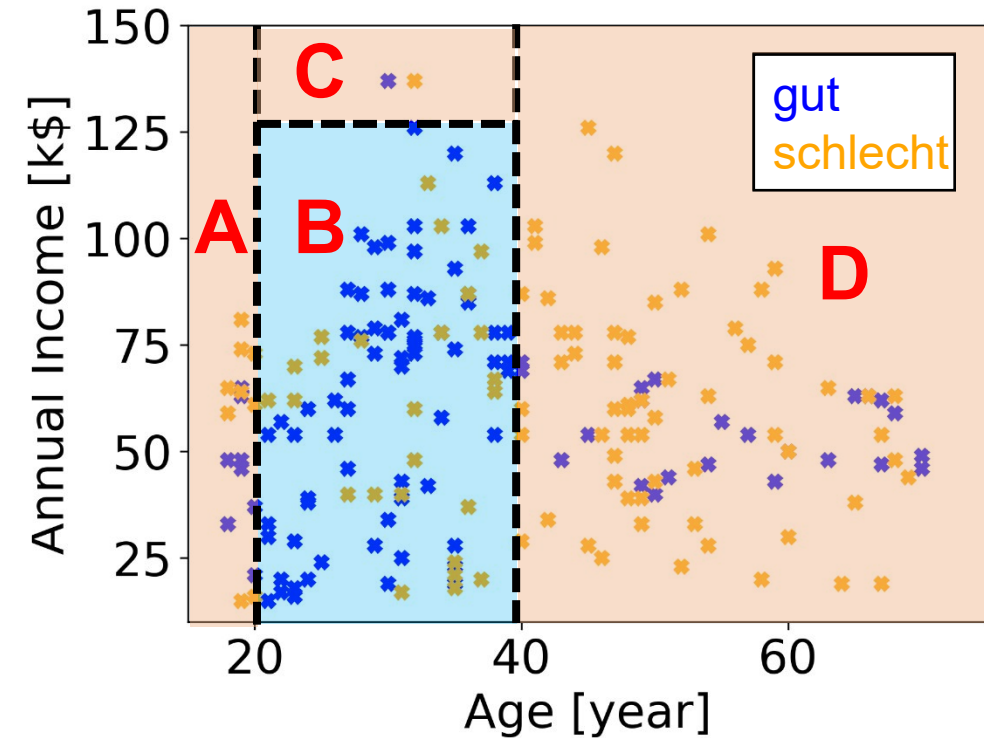


CART Algorithmus



Idea: Tree-based models partition the feature space into a set of **rectangles** and to each one we fit a **simple model** which we use to predict the target value, for example:

- Regression: constant value
- Classification: a single class values, or probability

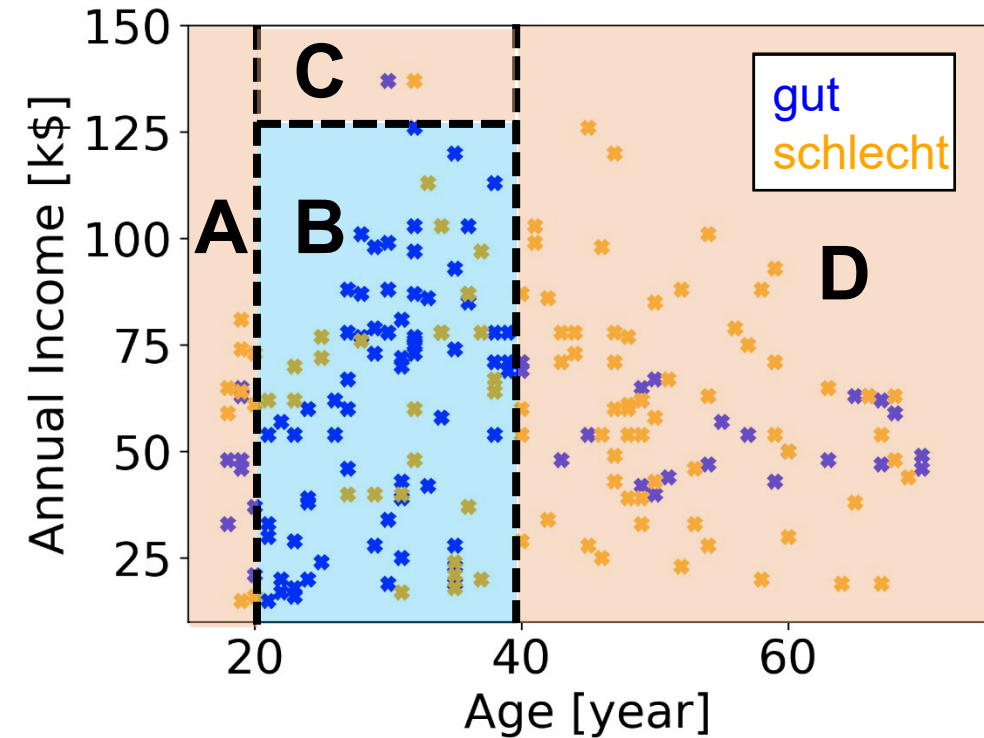


CART Algorithmus

Idea: Tree-based models partition the feature space into a set of **rectangles** and to each one we fit a **simple model** which we use to predict the target value, for example:

- Regression: constant value
- Classification: a single class values, or probability

Such partitioning can be identified with a **binary tree**



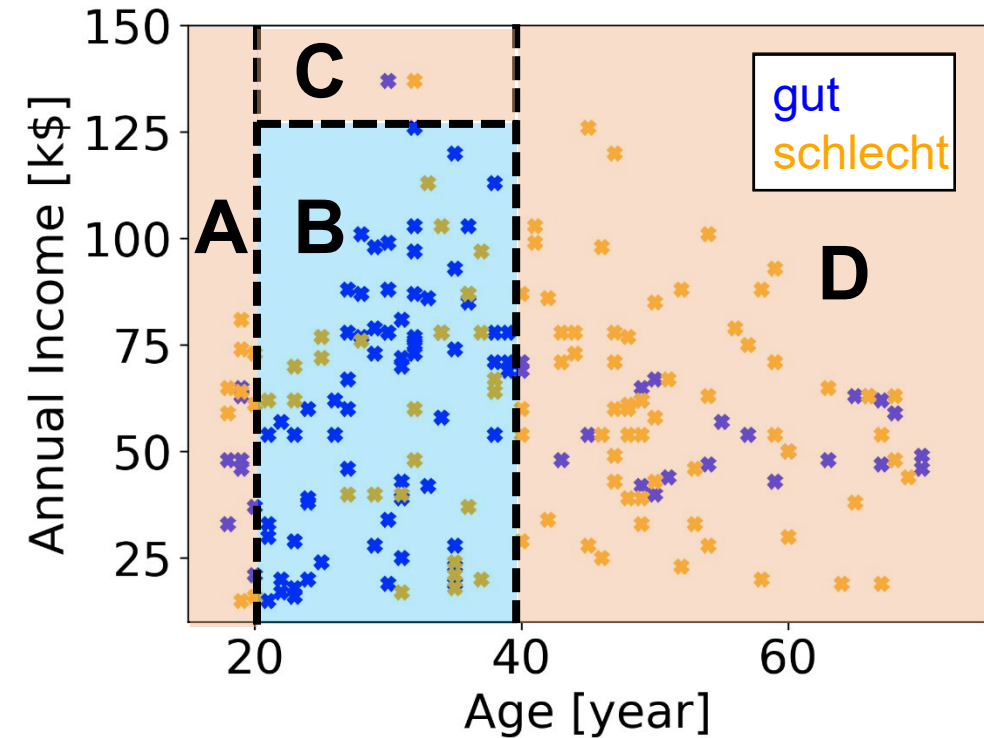
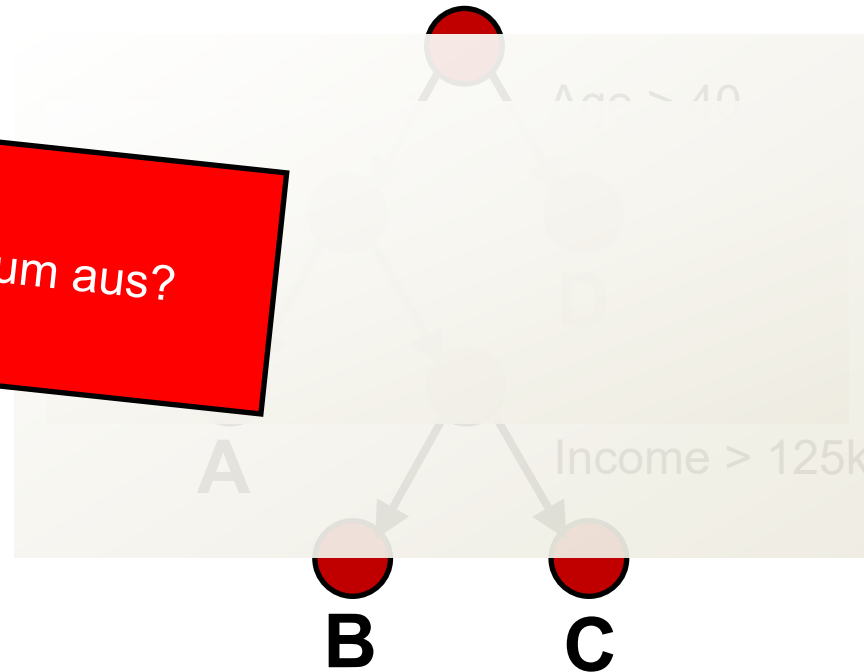
CART Algorithmus

Idea: Tree-based models partition the feature space into a set of **rectangles** and to each one we fit a **simple model** which we use to predict the target value, for example:

- Regression: constant value
- Classification: a single class values, or probability

Such partitioning can be identified with a **binary tree**

Wie sieht der Baum aus?

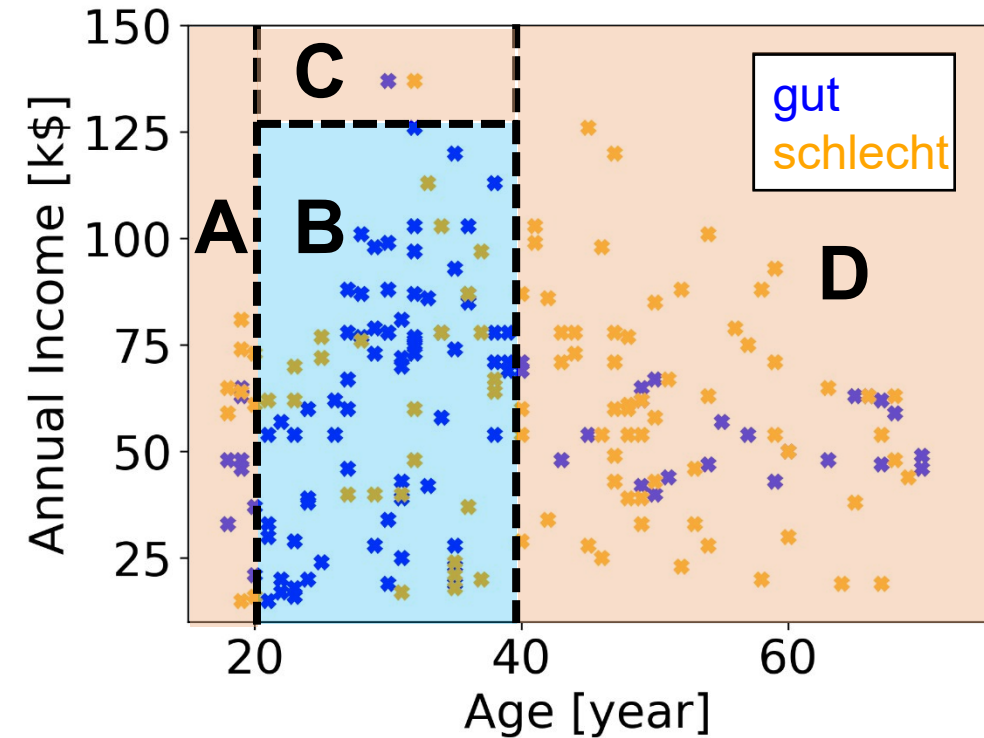
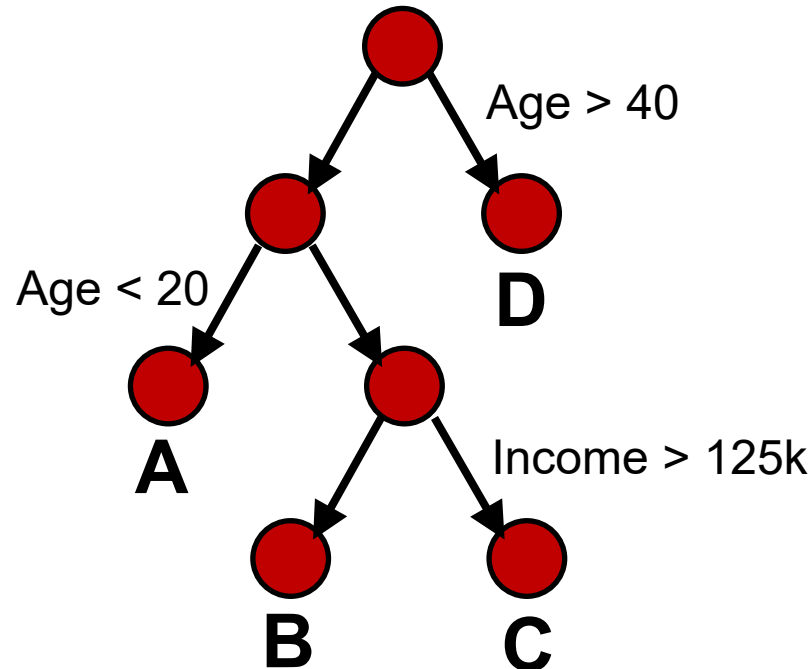


CART Algorithmus

Idea: Tree-based models partition the feature space into a set of **rectangles** and to each one we fit a **simple model** which we use to predict the target value, for example:

- Regression: constant value
- Classification: a single class values, or probability

Such partitioning can be identified with a **binary tree**

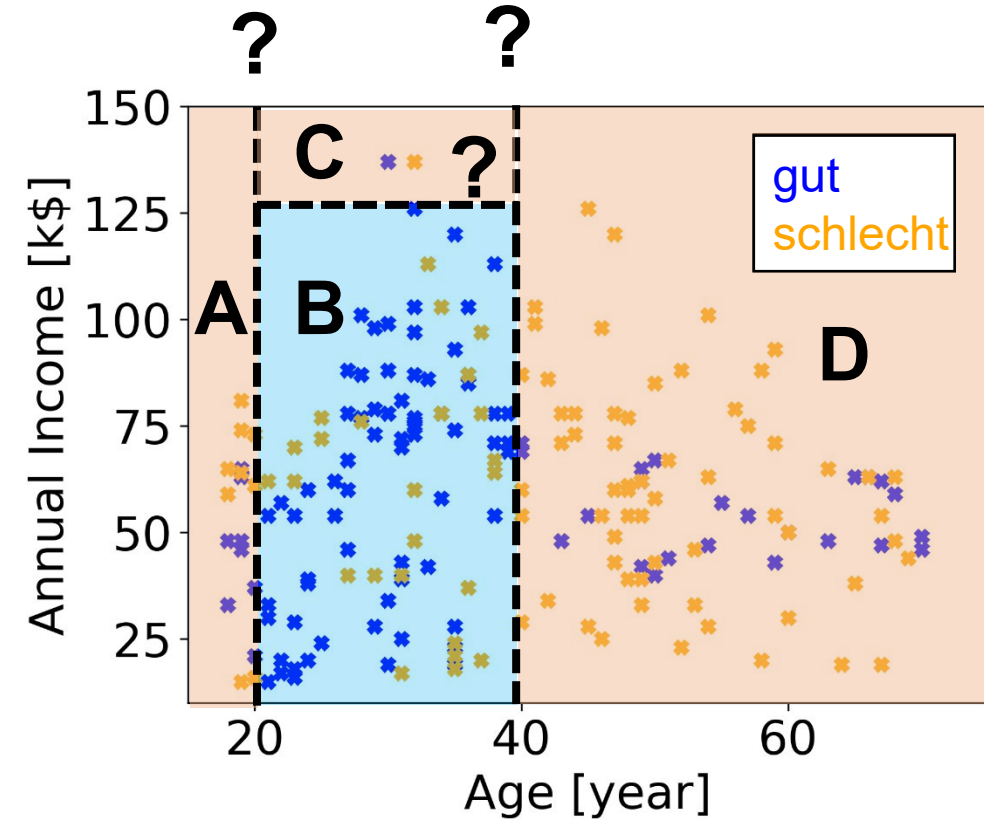
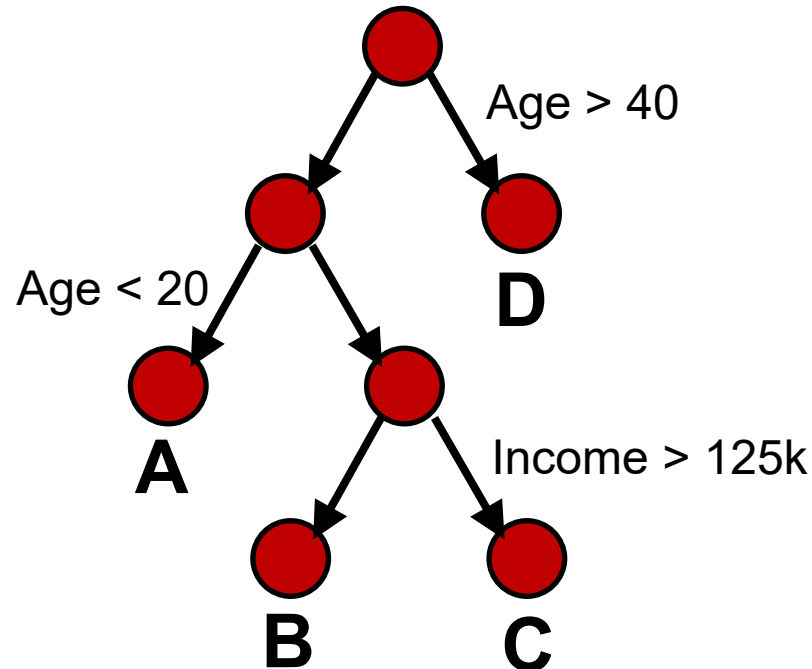


CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss



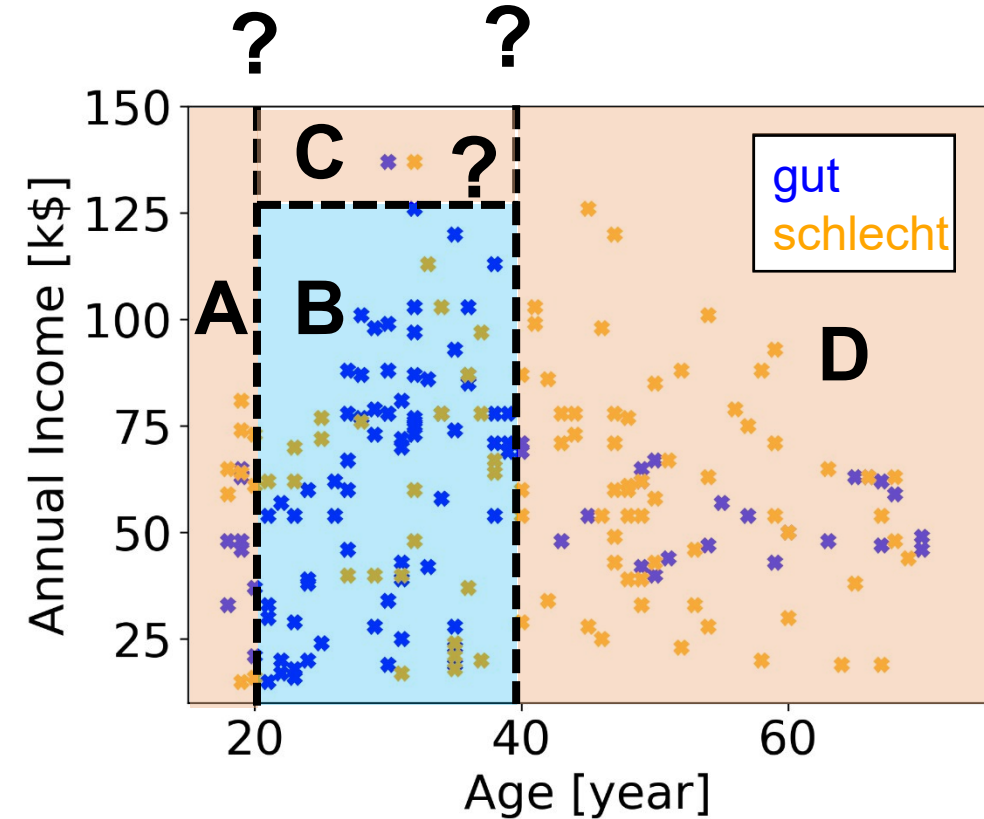
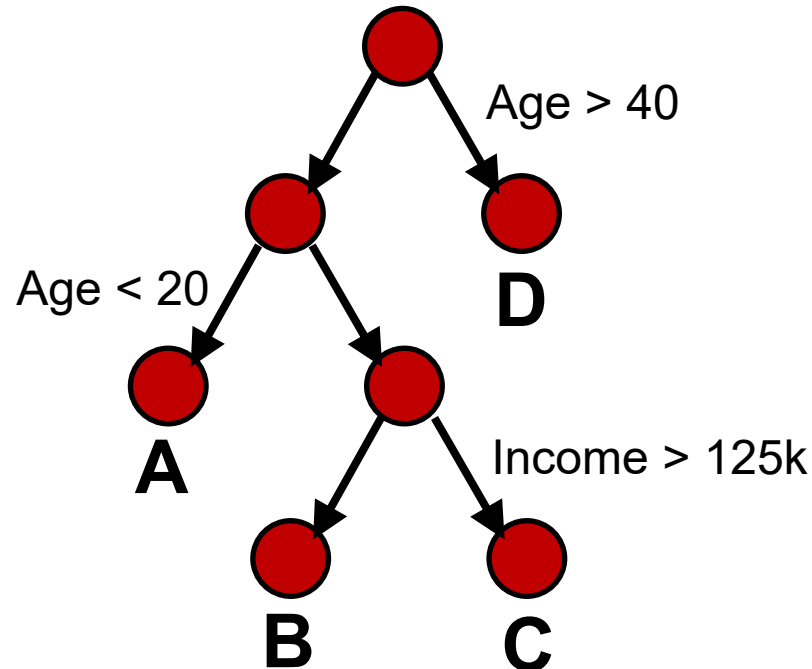
CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss

Maß für
die Güte



CART Algorithmus



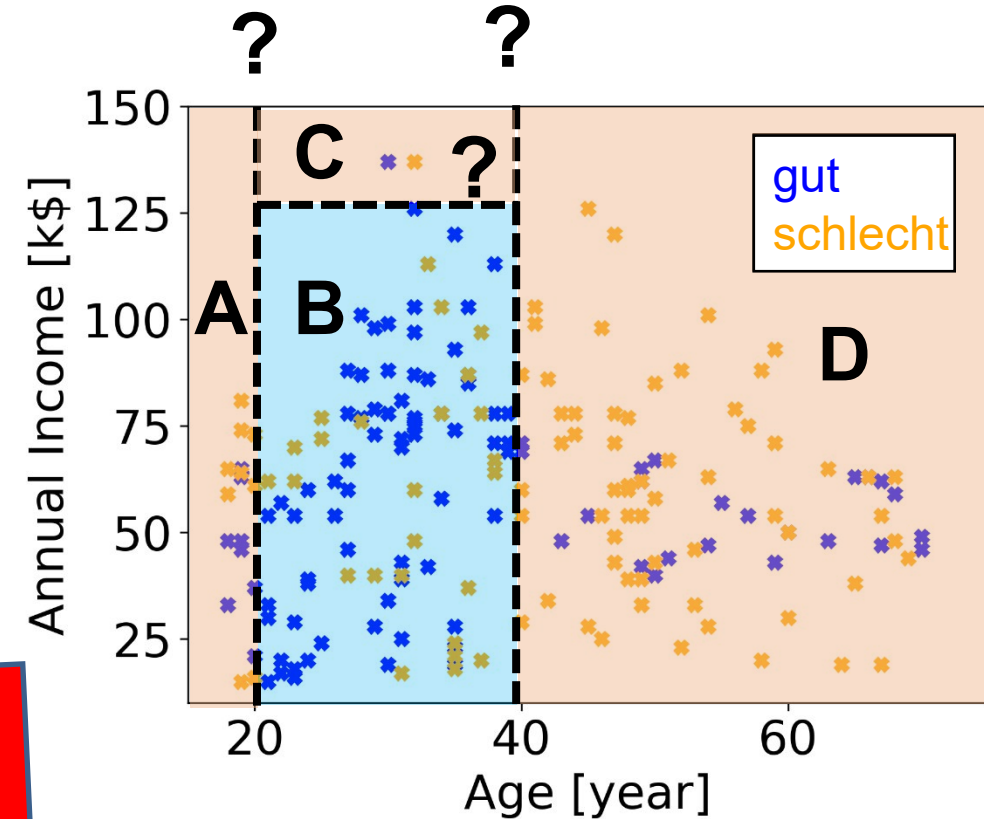
To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss

Maß für
die Güte



Wie kann man die Güte der Klassifikation
einer Region bestimmen?



CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss

Es gibt verschiedene Loss Funktionen (Entropy, Missklassifikation, etc.). Im folgenden wird der Gini Index exemplarisch verwendet:

$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1 - \hat{p}_m(k))$$

CART Algorithmus



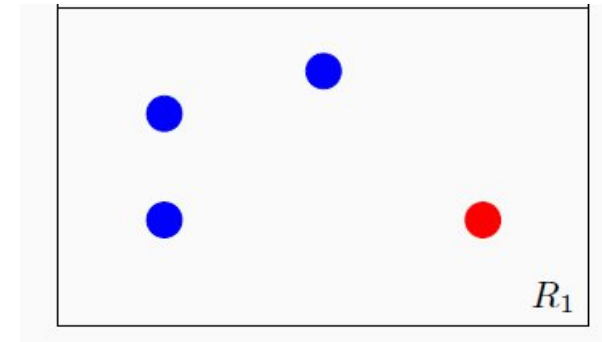
To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss

Es gibt verschiedene Loss Funktionen (Entropy, Missklassifikation, etc.). Im folgenden wird der Gini Index exemplarisch verwendet:

$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1 - \hat{p}_m(k))$$

Alle Datenpunkte
der Region R_m



CART Algorithmus



To learn a tree:

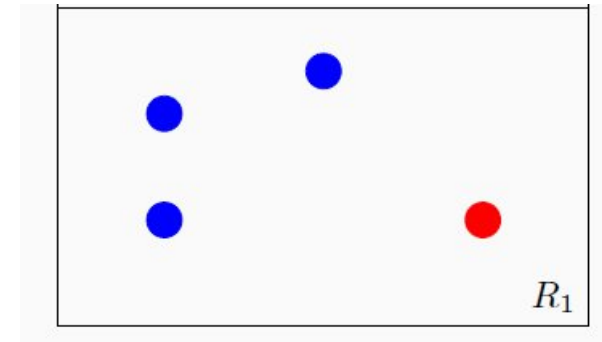
1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss

Es gibt verschiedene Loss Funktionen (Entropy, Missklassifikation, etc.). Im folgenden wird der Gini Index exemplarisch verwendet:

$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1 - \hat{p}_m(k))$$

Alle Datenpunkte
der Region R_m

Summe über alle
Klassen k



$k = 1$; blau
 $k = 2$; rot

CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss

Es gibt verschiedene Loss Funktionen (Entropy, Missklassifikation, etc.). Im folgenden wird der Gini Index exemplarisch verwendet:

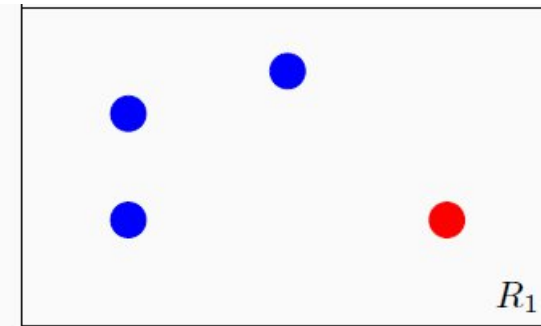
$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1 - \hat{p}_m(k))$$

Alle Datenpunkte der Region R_m → $L_{Gini}(D_m)$

Summe über alle Klassen k → $\sum_{k=1}^K$

Proportion der k Klasse → $\hat{p}_m(k)$

$$\hat{p}_1(Blue) = ? \quad \text{and} \quad \hat{p}_1(Red) = ?$$



$k = 1$; blau
 $k = 2$; rot

CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss

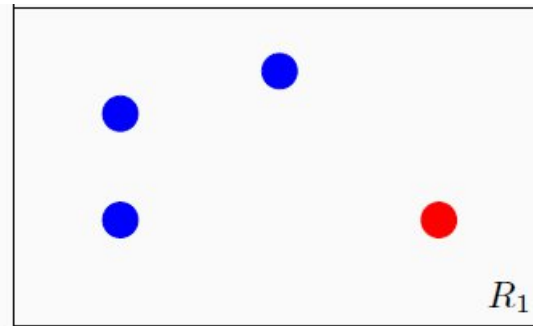
Es gibt verschiedene Loss Funktionen (Entropy, Missklassifikation, etc.). Im folgenden wird der Gini Index exemplarisch verwendet:

$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1 - \hat{p}_m(k))$$

Alle Datenpunkte
der Region R_m

Summe über alle
Klassen k

Proportion der k
Klasse



$k = 1$; blau
 $k = 2$; rot

$$\hat{p}_1(Blue) = \frac{3}{4} \quad \text{and} \quad \hat{p}_1(Red) = \frac{1}{4}$$

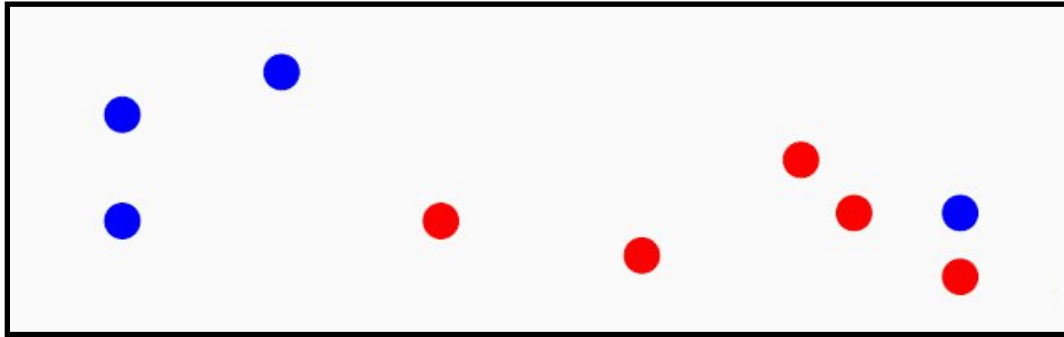
$$L_{Gini}(D_1) = \left(\frac{3}{4} \times \frac{1}{4}\right) + \left(\frac{1}{4} \times \frac{3}{4}\right) = \frac{6}{16} = 0.375$$

CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss



$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1-\hat{p}_m(k))$$

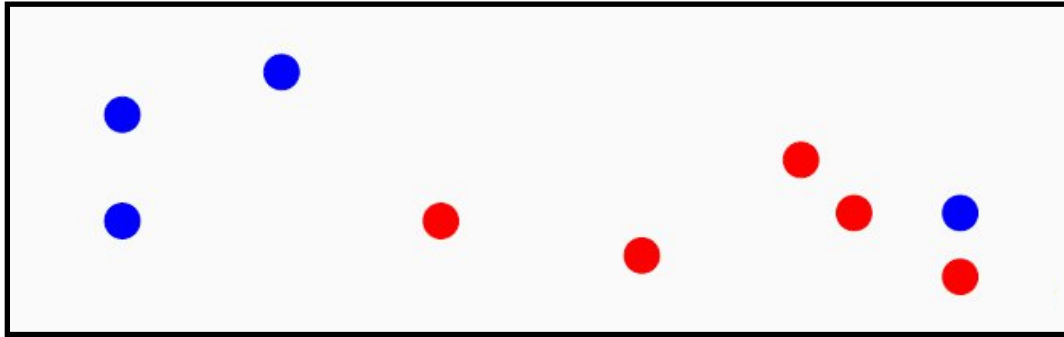
$$L_{Gini}(D_1) = \left(\frac{4}{9} \times \frac{5}{9}\right) + \left(\frac{5}{9} \times \frac{4}{9}\right) = \frac{40}{81} \sim 0.5$$

CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss (or maximize a gain)



$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1-\hat{p}_m(k))$$

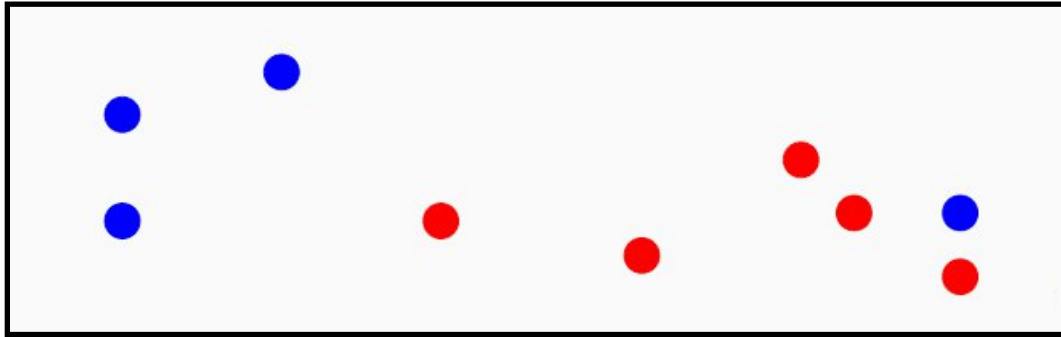
$$L_{Gini}(D_1) = \left(\frac{4}{9} \times \frac{5}{9}\right) + \left(\frac{5}{9} \times \frac{4}{9}\right) = \frac{40}{81} \sim 0.5$$

CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss (or maximize a gain)



Wie müsste eine Grenze gezogen werden, um Regionen mit möglichst reine Zustände (maximalen „Gain“) zu bekommen?

$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1 - \hat{p}_m(k))$$

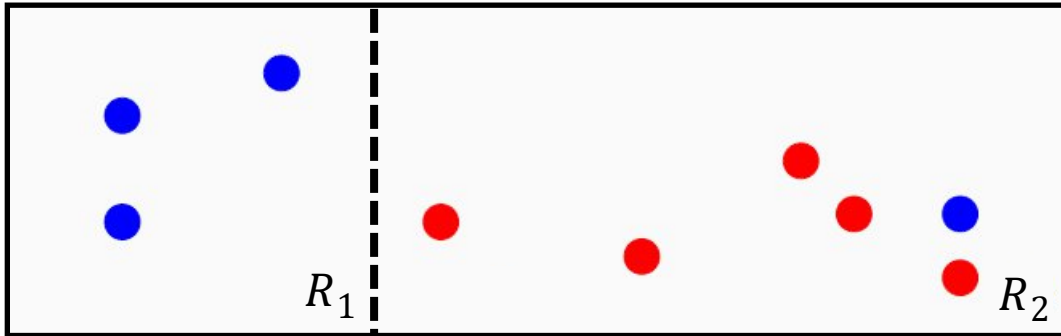
$$L_{Gini}(D_m) = \left(\frac{4}{9} \times \frac{5}{9}\right) + \left(\frac{5}{9} \times \frac{4}{9}\right) = \frac{40}{81} \sim 0.5$$

CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss (or maximize a gain)



$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1-\hat{p}_m(k))$$

$$L_{Gini}(D_m) = \left(\frac{4}{9} \times \frac{5}{9}\right) + \left(\frac{5}{9} \times \frac{4}{9}\right) = \frac{40}{81} \sim 0.5$$

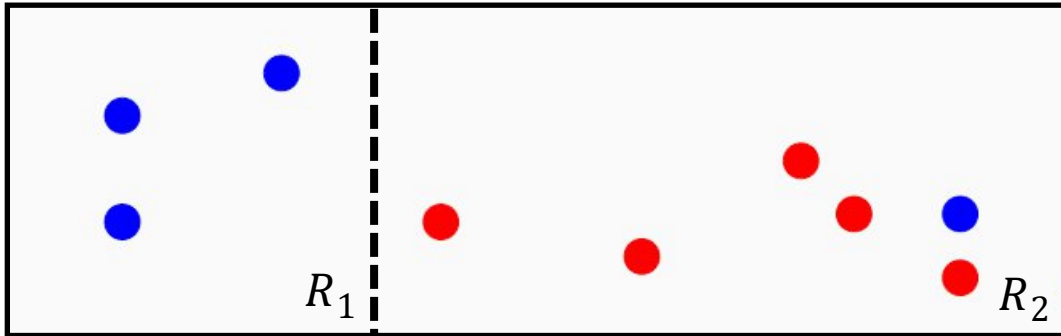
$$L_{Gini}(D_1) = 0$$
$$L_{Gini}(D_2) = \left(\frac{1}{6} \times \frac{5}{6}\right) + \left(\frac{5}{6} \times \frac{1}{6}\right) = \frac{10}{36} \sim 0.27$$

CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss (or maximize a gain)



$$L_{Gini}(D_m) = \sum_{k=1}^K \hat{p}_m(k)(1-\hat{p}_m(k))$$

$$L_{Gini}(D_m) = \left(\frac{4}{9} \times \frac{5}{9}\right) + \left(\frac{5}{9} \times \frac{4}{9}\right) = \frac{40}{81} \sim 0.5$$

$$L_{Gini}(D_1) = 0$$
$$L_{Gini}(D_2) = \left(\frac{1}{6} \times \frac{5}{6}\right) + \left(\frac{5}{6} \times \frac{1}{6}\right) = \frac{10}{36} \sim 0.27$$

$$G(D_m, \Theta) = \frac{N_1}{N_m} L_{Gini}(D_1) + \frac{N_2}{N_m} L_{Gini}(D_2) = \frac{6}{9} \times \frac{10}{36} \sim 0.19$$

Der beste splitting parameter Θ , ist der für den $G(D_m, \Theta)$ minimal ist, d.h.

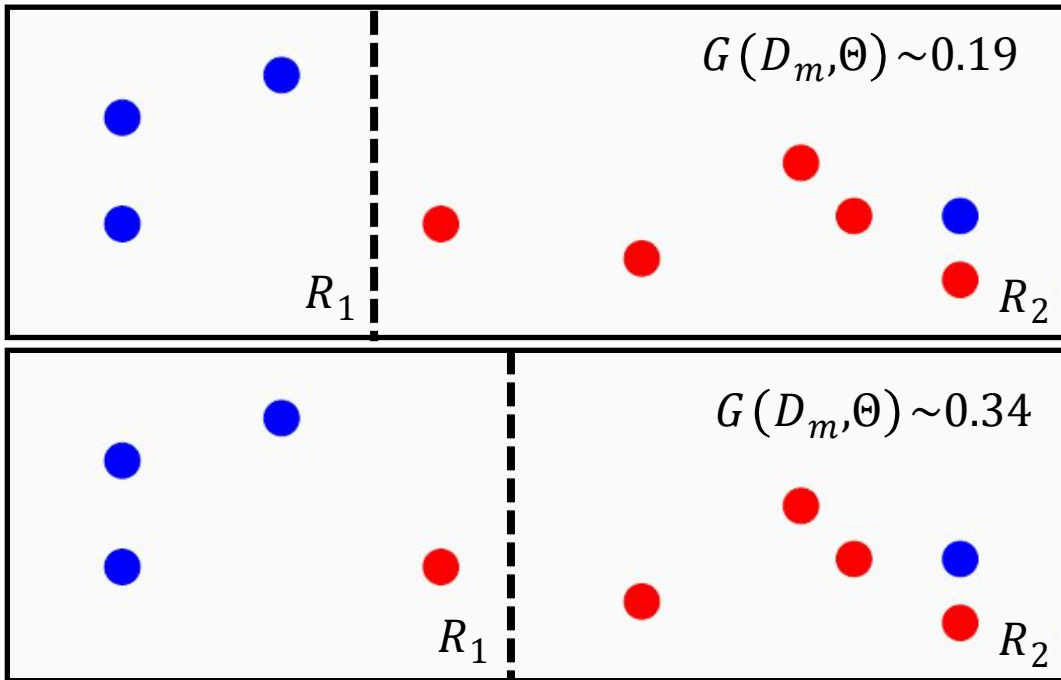
$$\Theta^* = \operatorname{argmin} G(D_m, \Theta)$$

CART Algorithmus



To learn a tree:

1. Determine the structure of the tree, including split variables and thresholds
2. Fit a model on each region, minimizing a given loss (or maximize a gain)

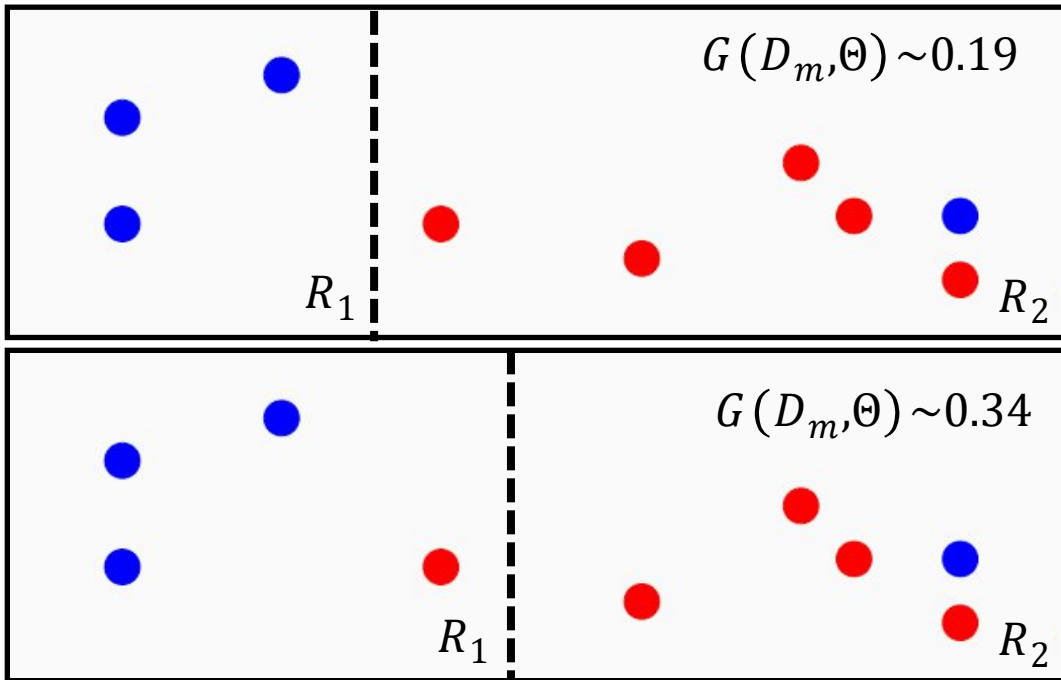


Der beste splitting parameter Θ , ist der für den $G(D_m, \Theta)$ minimal ist, d.h.

$$\Theta^* = \operatorname{argmin} G(D_m, \Theta)$$

Algorithmus

1. Starte mit dem ganzen Raum
2. Suche das Feature x und splitting-Parameter Θ^* mit dem minimalsten $G(D_m, \Theta)$.
3. Führe die Suche rekursive (in den Regionen R_m) aus, bis ein Abbruchkriterium erreicht ist



Der beste splitting parameter Θ , ist der für den $G(D_m, \Theta)$ minimal ist, d.h.

$$\Theta^* = \operatorname{argmin} G(D_m, \Theta)$$

Monte Carlo Tree Search (MCTS)



- Suchalgorithmus um gute Entscheidungen zu treffen
- Strategiespiele mit “wenigen” Zuständen und diskreten Aktionen
- im Besonderen: Brettspiele mit einem sehr hohen Branching Faktor (z.B. Dame, Schach, Go, etc.)

Monte Carlo Tree Search (MCTS)



- Suchalgorithmus um gute Entscheidungen zu treffen
- Strategiespiele mit “wenigen” Zuständen und diskreten Aktionen
- im Besonderen: Brettspiele mit einem sehr hohen Branching Faktor (z.B. Dame, Schach, Go, etc.)
- kann auf unvollständig beobachtbare Spiele (Karten) oder nicht deterministische Spiele (Würfel) angewendet werden (sehr erfolgreich bei Back-Gammon)

Monte Carlo Tree Search (MCTS)

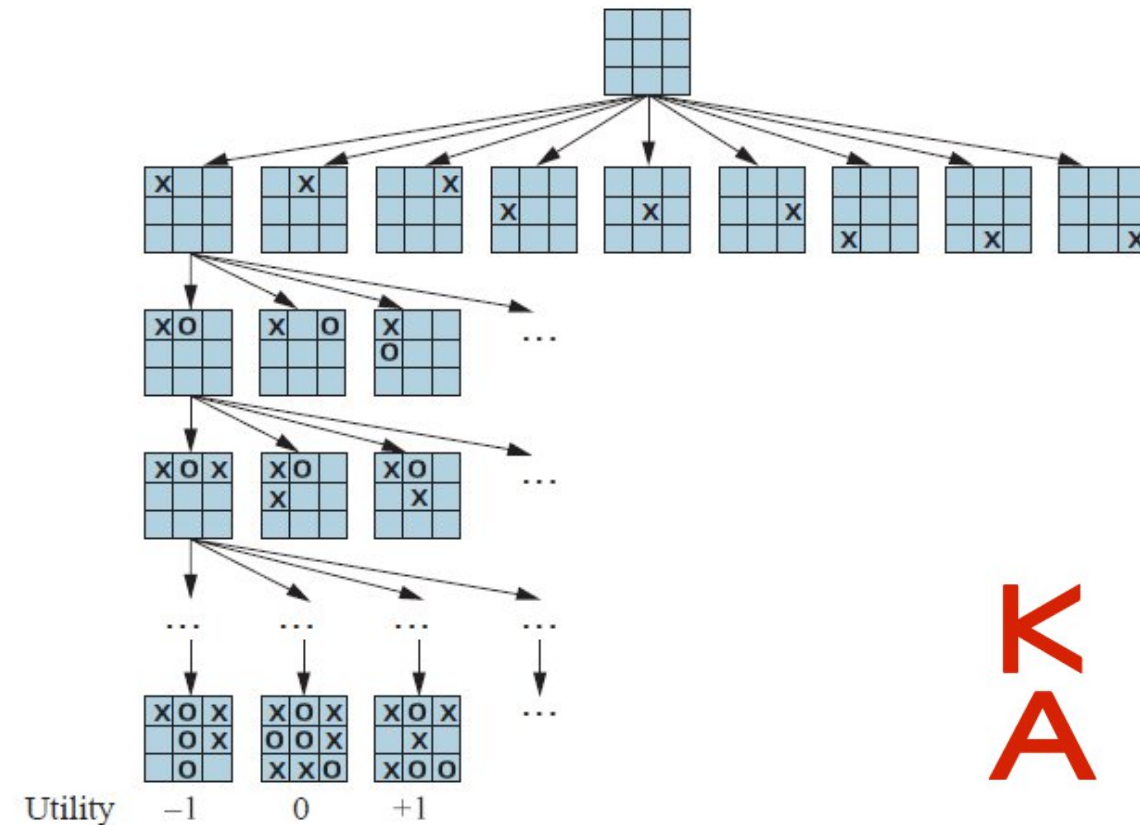


- Suchalgorithmus um gute Entscheidungen zu treffen
- Strategiespiele mit “wenigen” Zuständen und diskreten Aktionen
- im Besonderen: Brettspiele mit einem sehr hohen Branching Faktor (z.B. Dame, Schach, Go, etc.)
- kann auf unvollständig beobachtbare Spiele (Karten) oder nicht deterministische Spiele (Würfel) angewendet werden (sehr erfolgreich bei Back-Gammon)
- **Grundidee:** Simuliere eine große Anzahl an vollständig durchgespielten Partien und baue in Abhängigkeit von den Ergebnissen den Suchbaum auf

Monte Carlo Tree Search (MCTS)



- Suchalgorithmus um gute Entscheidungen zu treffen
- Strategiespiele mit “wenigen” Zuständen und diskreten Aktionen
- im Besonderen: Brettspiele mit einem sehr hohen Branching Faktor (z.B. Dame, Schach, Go, etc.)
- kann auf unvollständig beobachtbare Spiele (Karten) oder nicht deterministische Spiele (Würfel) angewendet werden (sehr erfolgreich bei Back-Gammon)
- **Grundidee:** Simuliere eine große Anzahl an vollständig durchgespielten Partien und baue in Abhängigkeit von den Ergebnissen den **Suchbaum** auf



Monte Carlo Tree Search (MCTS)



- Suchalgorithmus um gute Entscheidungen zu treffen
- Strategiespiele mit “wenigen” Zuständen und diskreten Aktionen
- im Besonderen: Brettspiele mit einem sehr hohen Branching Faktor (z.B. Dame, Schach, Go, etc.)
- kann auf unvollständig beobachtbare Spiele (Karten) angewendet werden (sehr erfolgreich bei Back-Gammon)
- Grundidee: Simuliere eine große Anzahl an vollständigen Spielen, um die Abhängigkeit von den Ergebnissen den Suchbaum aufzubauen (Benötigt kein Expertenwissen über das Spiel (= kein Domänenwissen))
- **erfolgreich mit tiefen Netzwerken kombinierbar**
AlphaGo (2016)
AlphaGo Zero (2017)
Alpha Zero (2017);

Google AI defeats human Go champion

25 May 2017



REUTERS

Chinese Go player Ke Jie has lost two games to AlphaGo

Monte Carlo Tree Search (MCTS)



- Suchalgorithmus um gute Entscheidungen zu treffen
- Strategiespiele mit “wenigen” Zuständen und diskreten Aktionen
- im Besonderen: Brettspiele mit einem sehr hohen Branching Faktor (z.B. Dame, Schach, Go, etc.)
- kann auf unvollständig beobachtbare Spiele (Karten) angewendet werden (sehr erfolgreich bei Back-Gammon)
- Grundidee: Simuliere eine große Anzahl an vollständigen Spielen
Abhängigkeit von den Ergebnissen den Suchbaum aufwärts
(Benötigt kein Expertenwissen über das Spiel (= keine Heuristiken))
- **erfolgreich mit tiefen Netzwerken kombinierbar**
AlphaGo (2016)
AlphaGo Zero (2017)
Alpha Zero (2017);

Google AI defeats human Go champion

25 May 2017



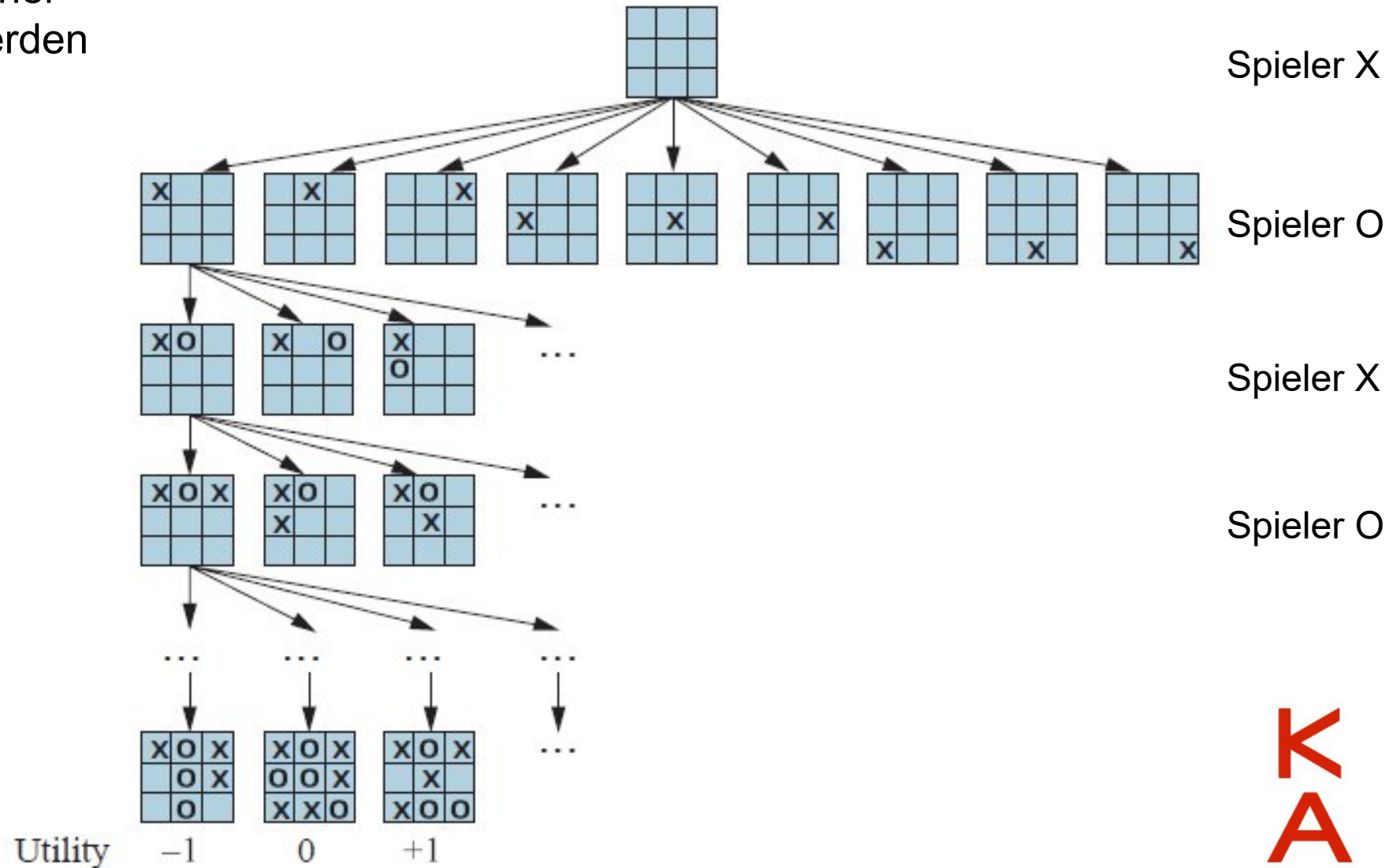
REUTERS

Chinese Go player Ke Jie has lost two games to AlphaGo

Monte Carlo Tree Search (MCTS)



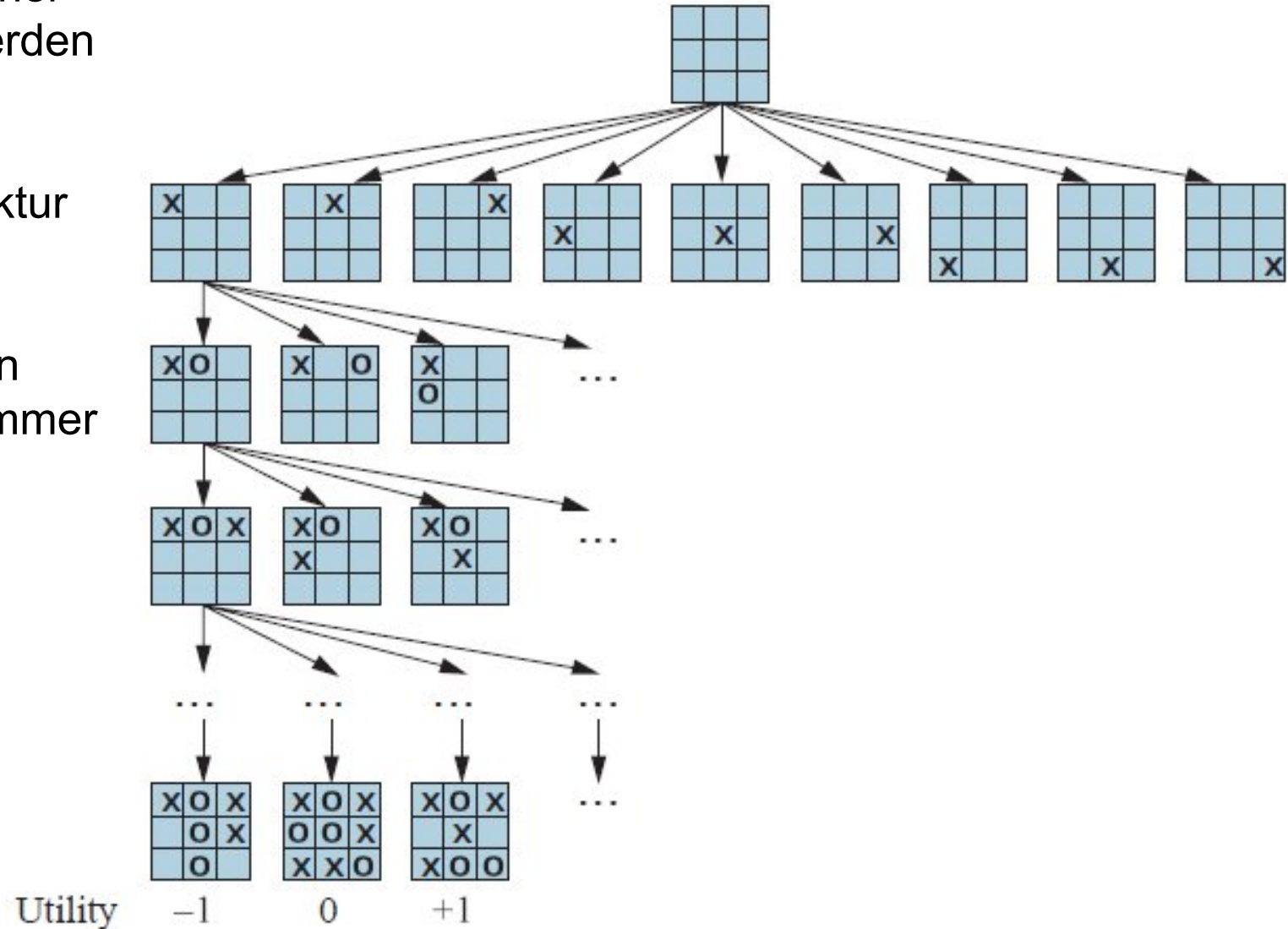
- Mögliche Züge können in einer Baumstruktur dargestellt werden



Monte Carlo Tree Search (MCTS)



- Mögliche Züge können in einer Baumstruktur dargestellt werden
- Eine vollständige Baumstruktur nur bei Spielen mit wenigen Zuständen möglich
- Heuristiken (Abschätzen von erwartetem Gewinn) nicht immer gut möglich (z.B. bei Go)

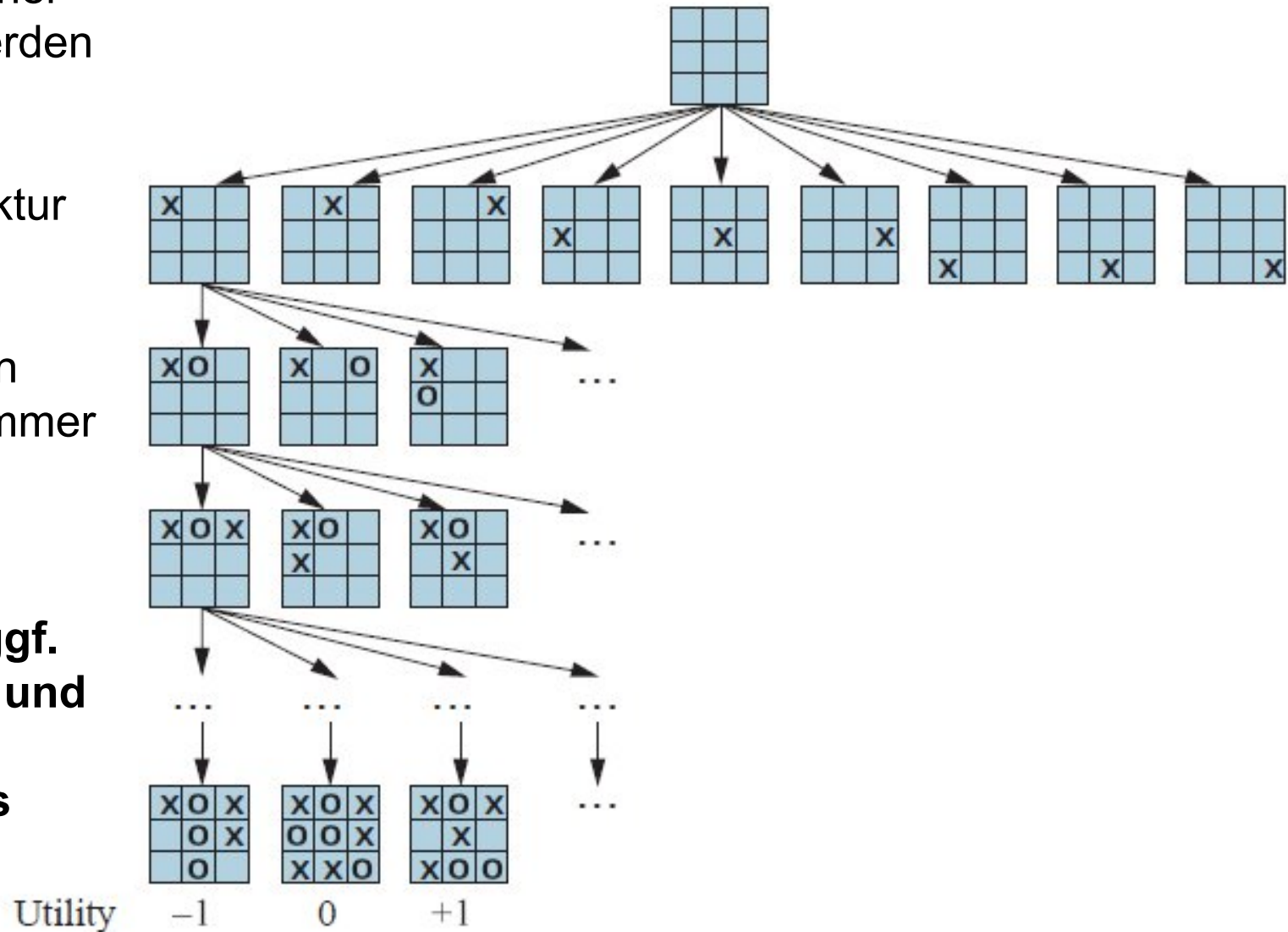


Monte Carlo Tree Search (MCTS)

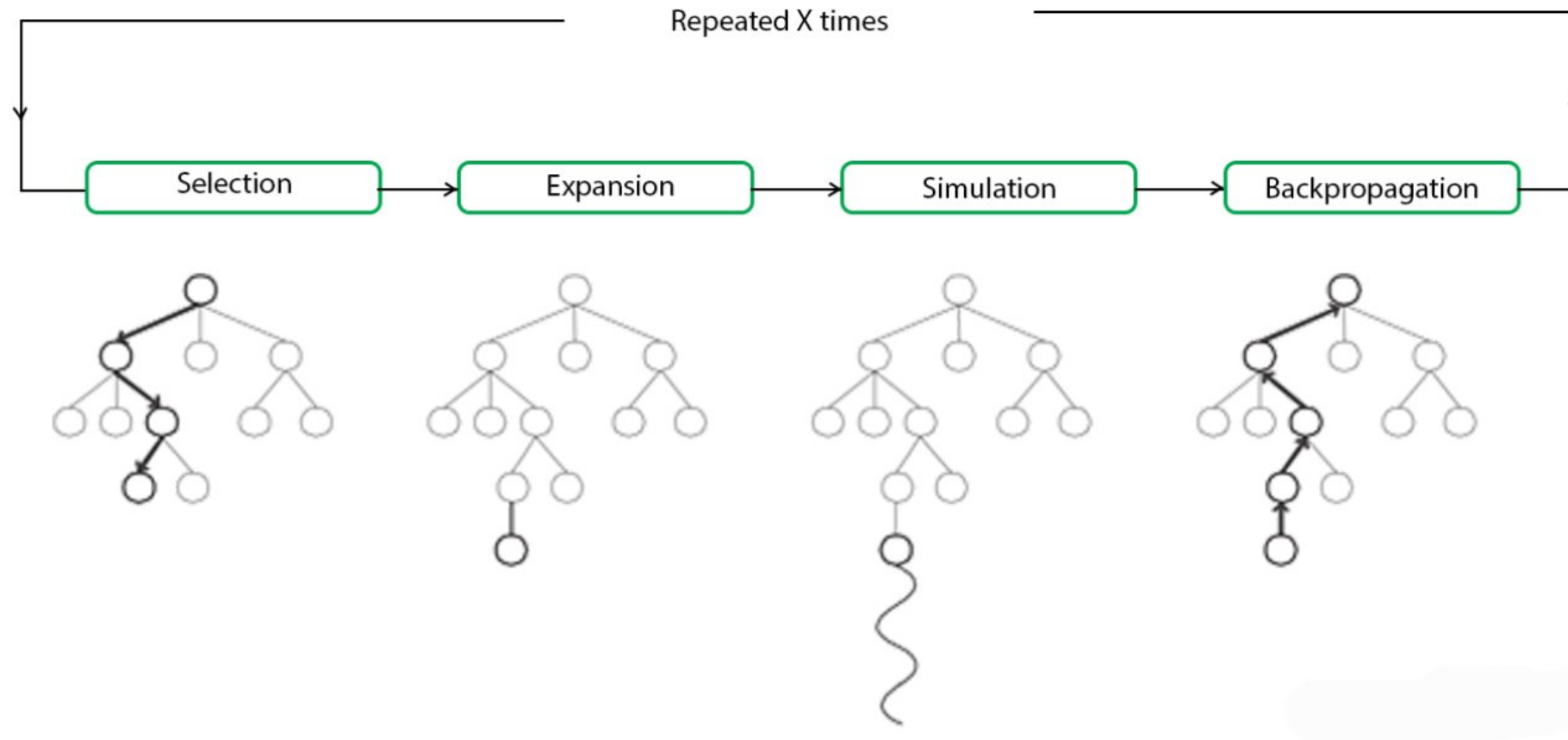


- Mögliche Züge können in einer Baumstruktur dargestellt werden
- Eine vollständige Baumstruktur nur bei Spielen mit wenigen Zuständen möglich
- Heuristiken (Abschätzen von erwartetem Gewinn) nicht immer gut möglich (z.B. bei Go)

MCTS erzeugt eine solche (ggf. unvollständige) Baumstruktur und ermöglicht das Finden eines optimalen / guten Spielzuges



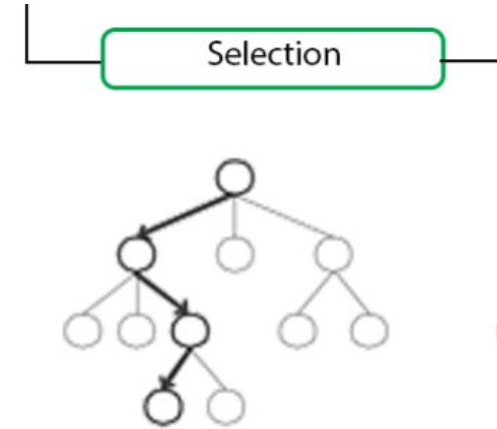
Monte Carlo Tree Search (MCTS)



Monte Carlo Tree Search (MCTS)

Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

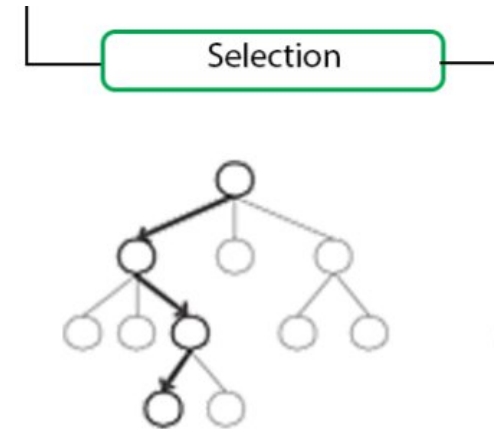


Monte Carlo Tree Search (MCTS)



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT



v_k = Anteil der gewonnenen Simulationen am Knoten k
(Aus Sicht des Spieler der den Zug macht, der zu Knoten k führt)
 n_p = Anzahl der von k 's Elternknoten gespielten Simulationen
 n_k = Anzahl der von Knoten k gespielten Simulationen

$$UCB_k = v_k + C \times \sqrt{\frac{\log n_p}{n_k}}$$

C = Parameter (je höher C umso wichtiger wird Exploration)

UCT: wähle den Knoten mit höchstem UCB

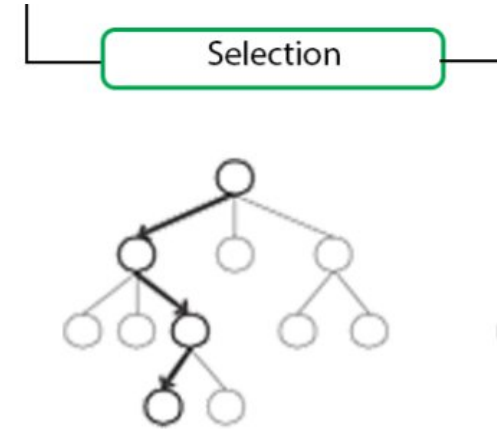


Monte Carlo Tree Search (MCTS)



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT



v_k = Anteil der gewonnenen Simulationen am Knoten k
(Aus Sicht des Spieler der den Zug macht, der zu Knoten k führt)
 n_p = Anzahl der von k 's Elternknoten gespielten Simulationen
 n_k = Anzahl der von Knoten k gespielten Simulationen

Exploitation $\rightarrow$ $UCB_k = v_k + C \times \sqrt{\frac{\log n_p}{n_k}}$ $\leftarrow$ Exploration

C = Parameter (je höher C umso wichtiger wird Exploration)

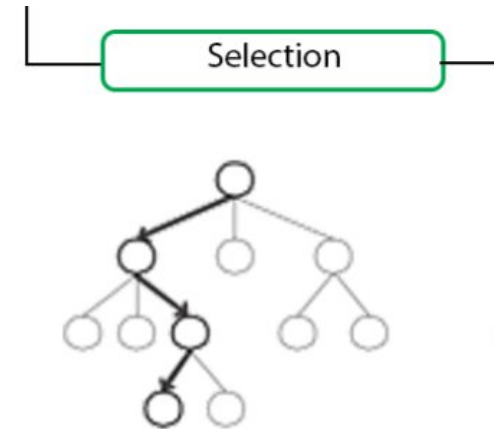


Monte Carlo Tree Search (MCTS)



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT



v_k = Anteil der gewonnenen Simulationen am Knoten k
(Aus Sicht des Spieler der den Zug macht, der zu Knoten k führt)
 n_p = Anzahl der von k 's Elternknoten gespielten Simulationen
 n_k = Anzahl der von Knoten k gespielten Simulationen

$$UCB_k = v_k + C \times \sqrt{\frac{\log n_p}{n_k}}$$

C = Parameter (je höher C umso wichtiger wird Exploration)

C Werte experimentell ermitteln / starte in der Nähe von $\sqrt{2}$ Kocsis & Szepesvari, ECML, 2006



Monte Carlo Tree Search (MCTS)

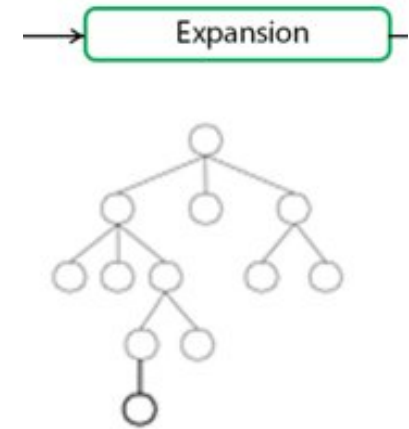


Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

Expansion

- Baum erweitern durch Kinder Knoten



Monte Carlo Tree Search (MCTS)



Selection

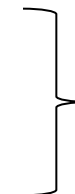
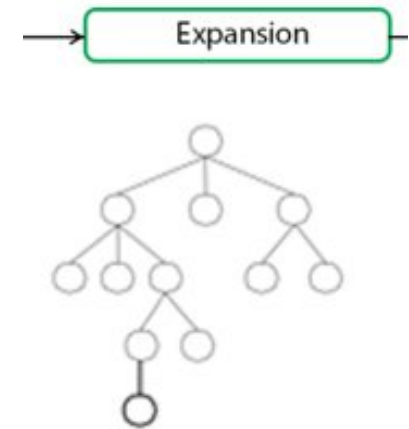
- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

Expansion

- Baum erweitern durch Kinder Knoten

Expansionsregeln

- Innere Knoten, bei denen alle Kinder bereits expandiert sind, können nicht weiter expandiert werden -> Auswahl (Selection) eines Kindes per UCT
- Blattknoten ohne Kinder
 - Variante 1:
Erzeuge alle Kinder, initialisiere diese und führe für jeden Kindknoten eine Simulation durch
 - Variante 2:
Erzeuge einen zufällig ausgewählten Kindknoten, initialisiere diesen und führe eine Simulation durch



bei hohem
Branching Faktor
sinnvoll



Monte Carlo Tree Search (MCTS)



Selection

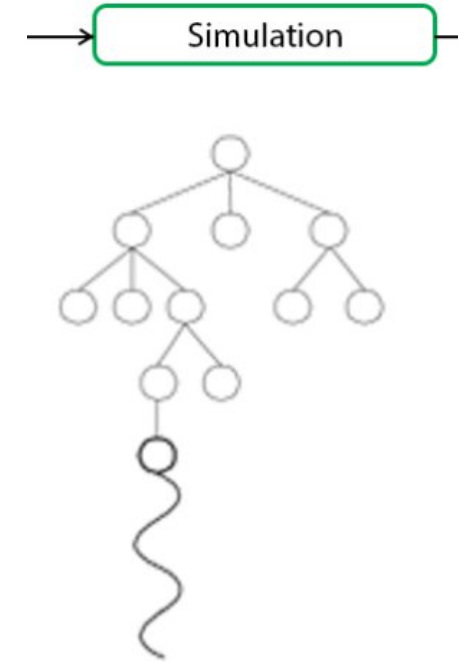
- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

Expansion

- Baum erweitern durch Kinder Knoten

Simulation

- Ein Spiel durchspielen (playout).
- Die Züge werden nicht im Suchbaum gespeichert



Monte Carlo Tree Search (MCTS)



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

Expansion

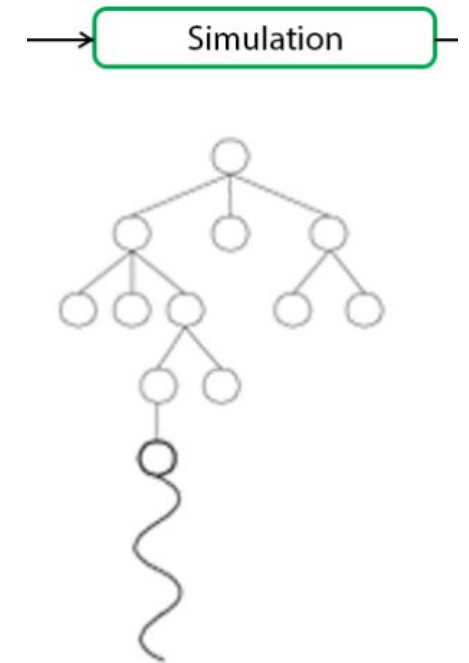
- Baum erweitern durch Kinder Knoten

Simulation

- Ein Spiel durchspielen (playout).
- Die Züge werden nicht im Suchbaum gespeichert



Hier reicht einfaches zufälliges Ausspielen -> Statistik ist entscheidend



Monte Carlo Tree Search (MCTS)



Selection

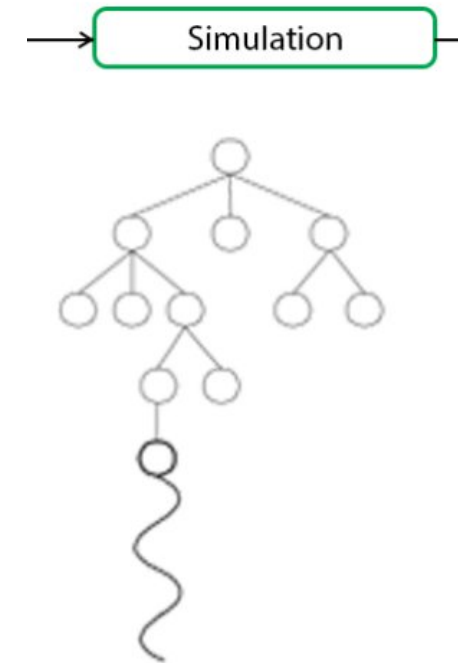
- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

Expansion

- Baum erweitern durch Kinder Knoten

Simulation

- Ein Spiel durchspielen (playout).
- Die Züge werden nicht im Suchbaum gespeichert



Hier reicht einfaches zufälliges Ausspielen -> Statistik ist entscheidend
Eine verfeinerte Playout-Policy kann das Training verkürzen & verbessern.

Monte Carlo Tree Search (MCTS)



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

Expansion

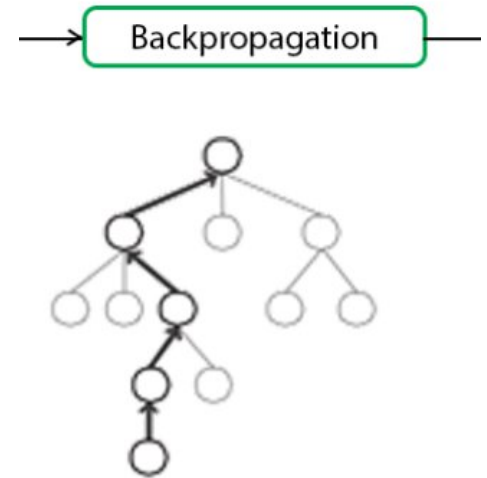
- Baum erweitern durch Kinder Knoten

Simulation

- Ein Spiel durchspielen (playout).
- Die Züge werden nicht im Suchbaum gespeichert

Back-propagation

- Spielergebnis wird verwendet, um alle Knoten des Baumes auf dem Weg zu updaten.



Monte Carlo Tree Search (MCTS)



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

Expansion

- Baum erweitern durch Kinder Knoten

Simulation

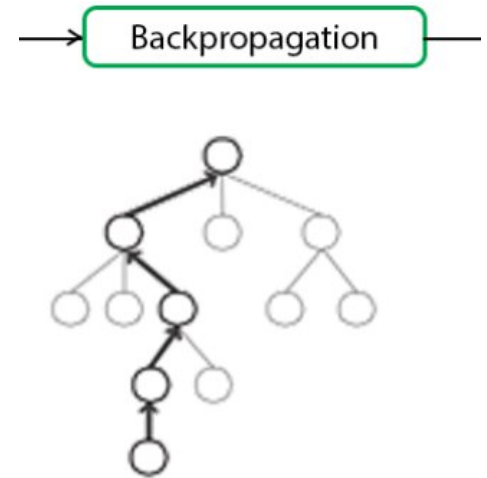
- Ein Spiel durchspielen (playout).
- Die Züge werden nicht im Suchbaum gespeichert

Back-propagation

- Spielergebnis wird verwendet, um alle Knoten des Baumes auf dem Weg zu updaten.



im Besonderen: v_k , n_k



K
A

Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

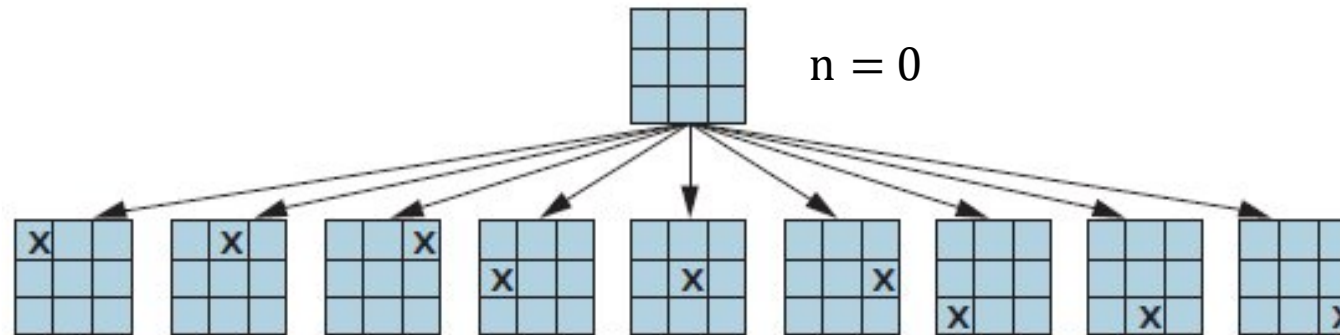


$n = 0$

Zu Beginn kann nur der Root Knoten selektiert werden.

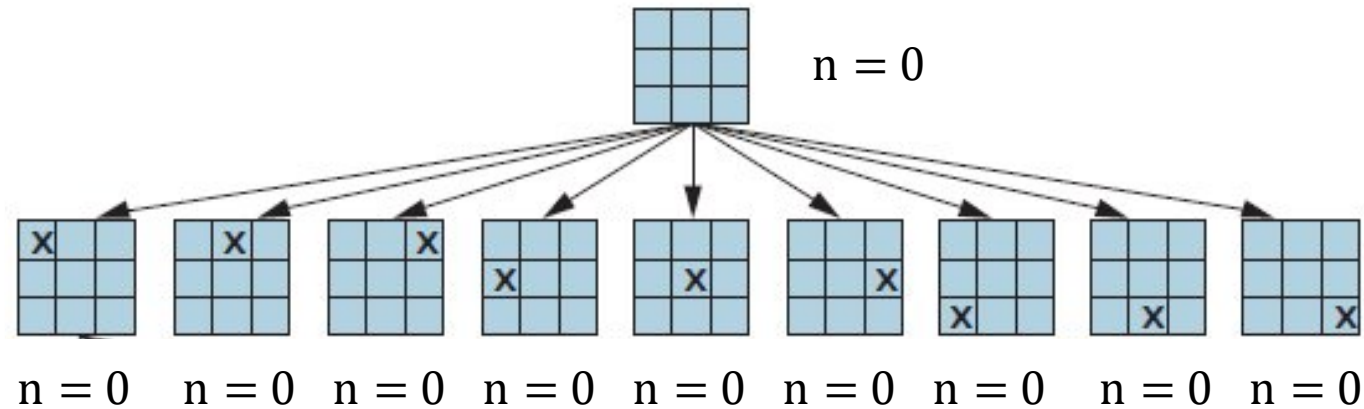
Expansionsregeln

- Blattknoten ohne Kinder
 - Variante 1:
Erzeuge alle Kinder, initialisiere diese und führe für jeden Kindknoten eine Simulation durch



Expansionsregeln

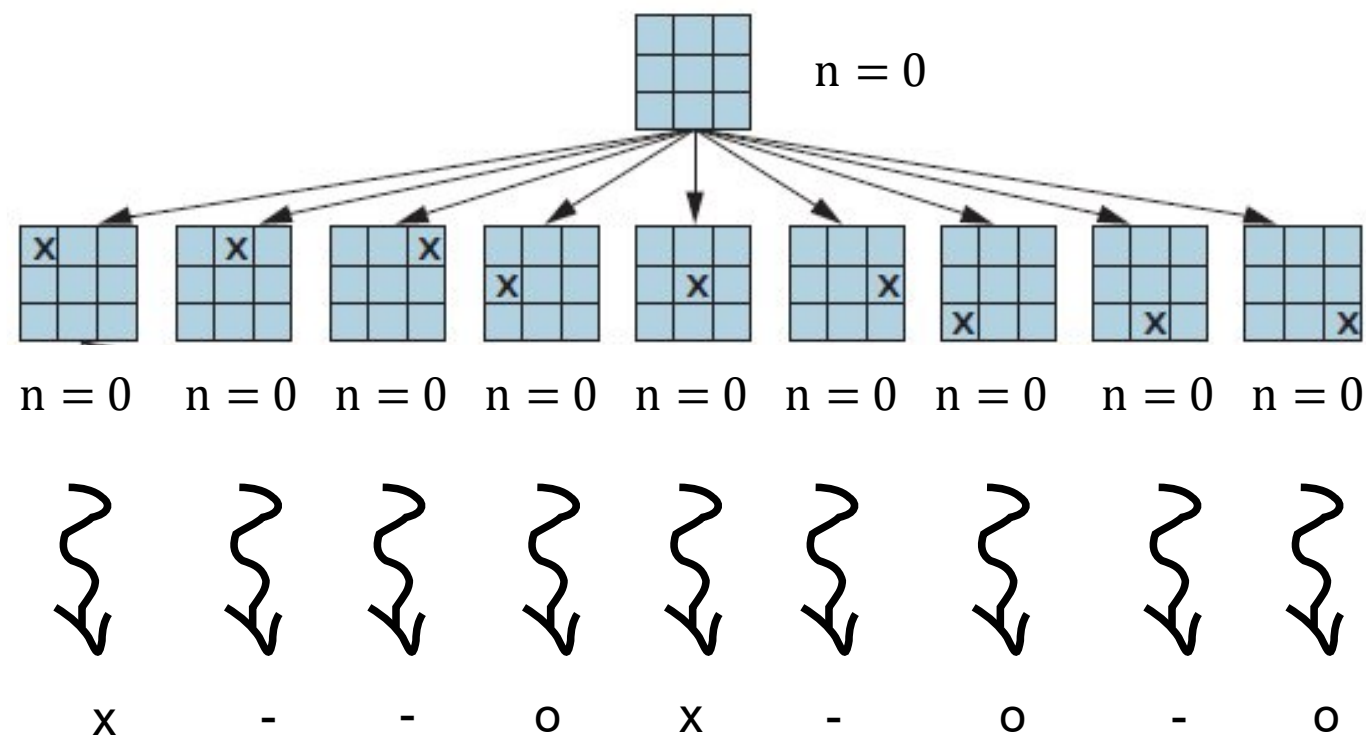
- Blattknoten ohne Kinder
 - Variante 1:
Erzeuge alle Kinder, **initialisiere** diese und führe für jeden Kindknoten eine Simulation durch





Simulation

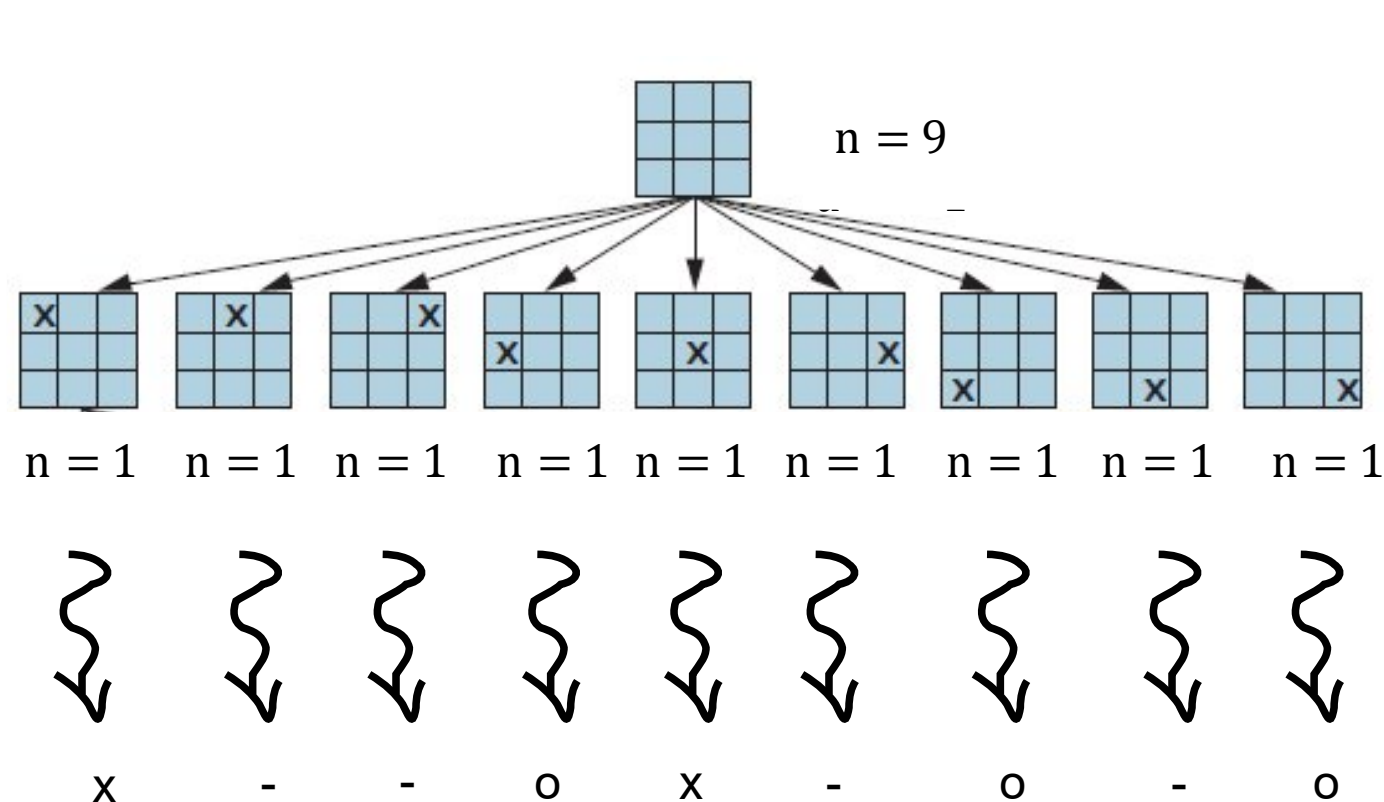
Erzeuge alle Kinder, initialisiere diese und führe für jeden Kindknoten eine Simulation durch





Back-propagation

- Spielergesult wird verwendet, um alle Knoten des Baumes auf dem Weg upzudaten.

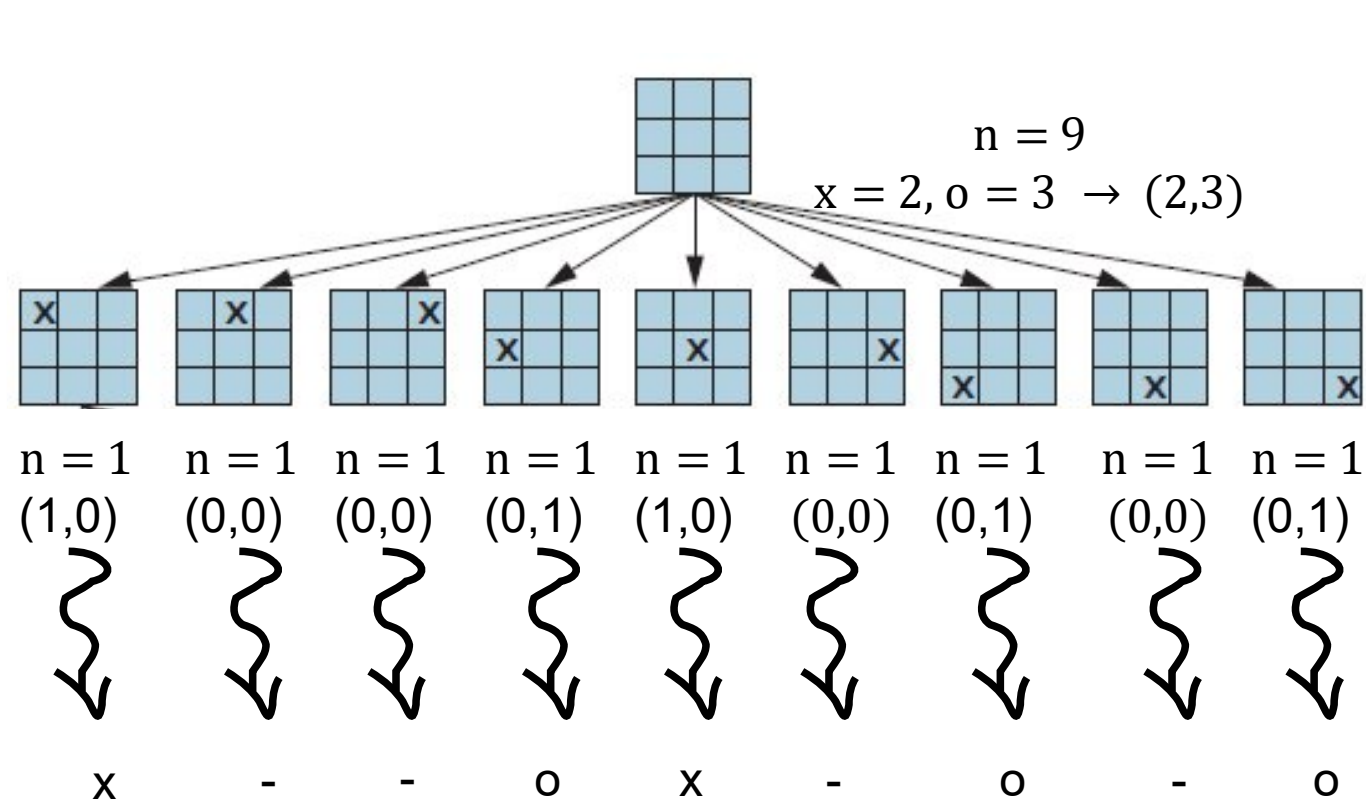


Update
1. Anzahl Spiele durch den Knoten



Back-propagation

- Spielergesultis wird verwendet, um alle Knoten des Baumes auf dem Weg upzudaten.



- Update
1. Anzahl Spiele durch den Knoten
 2. Info Gewinn/Verlust

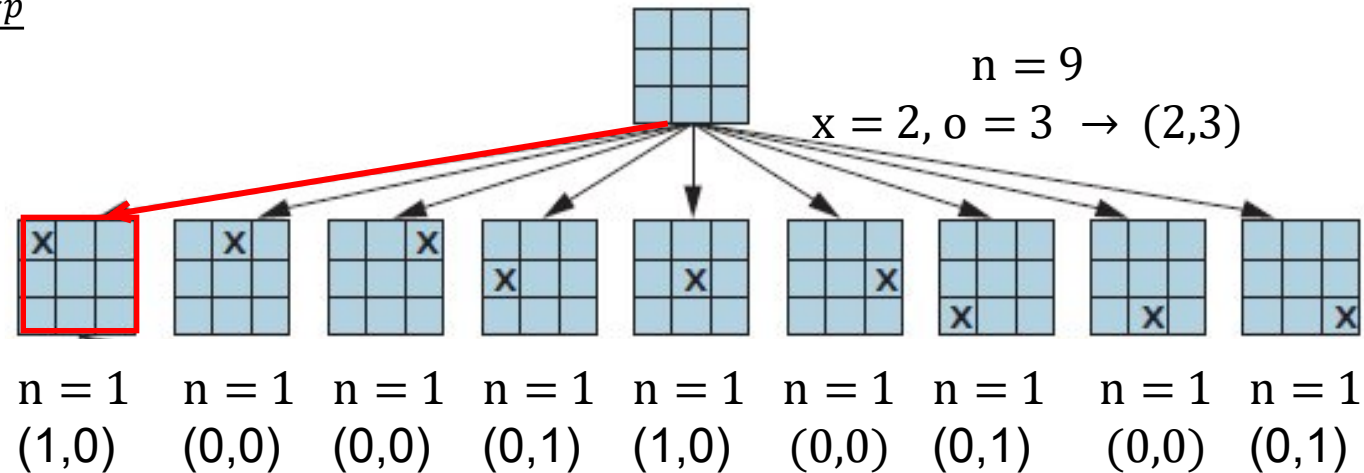
Gewonnen: x oder o
Unentschieden: -



Selection

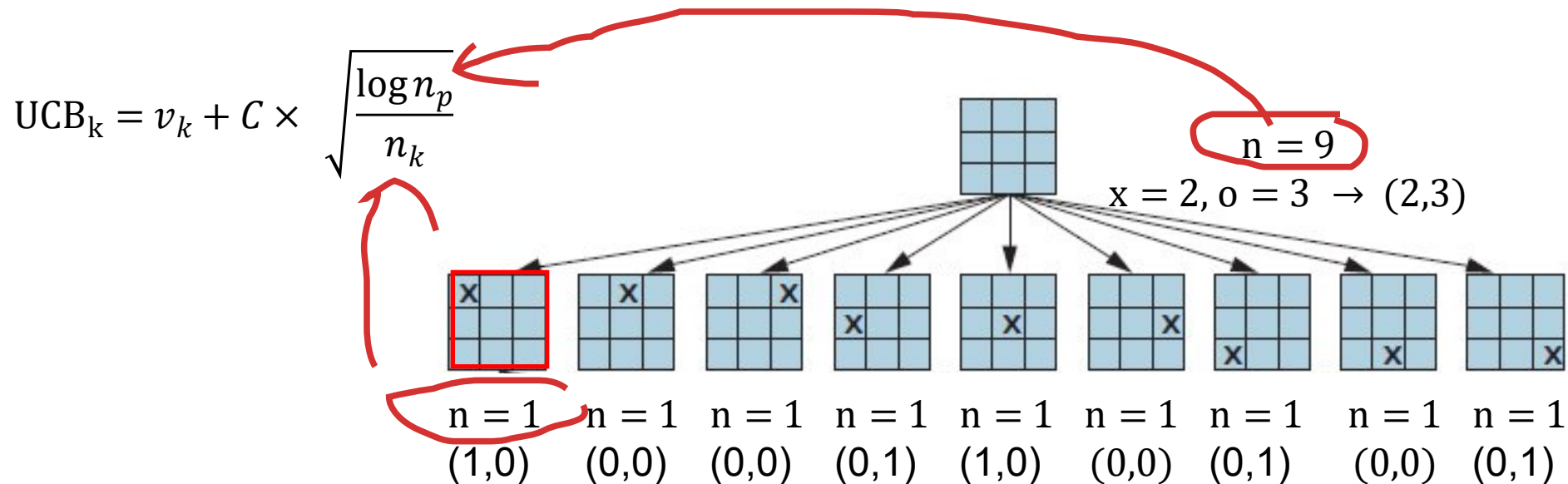
- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

$$UCB_k = v_k + C \times \sqrt{\frac{\log n_p}{n_k}}$$



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT



MCTS Beispiel



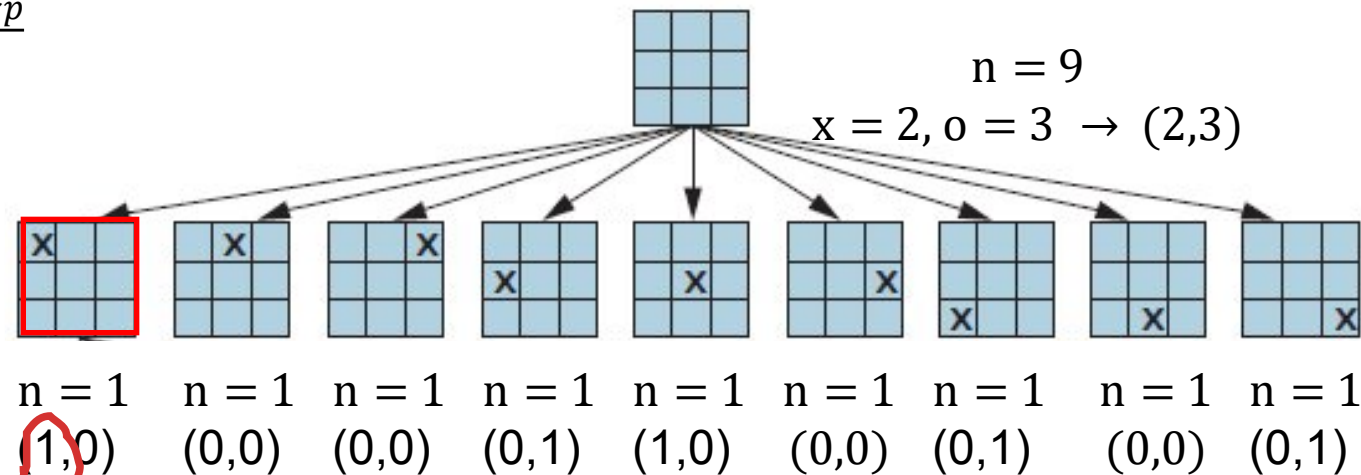
Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

$$UCB_k = v_k + C \times \sqrt{\frac{\log n_p}{n_k}}$$

mit

$$v_k = \frac{u_k}{n_k}$$



UCT: wähle den Knoten mit
höchstem UCB

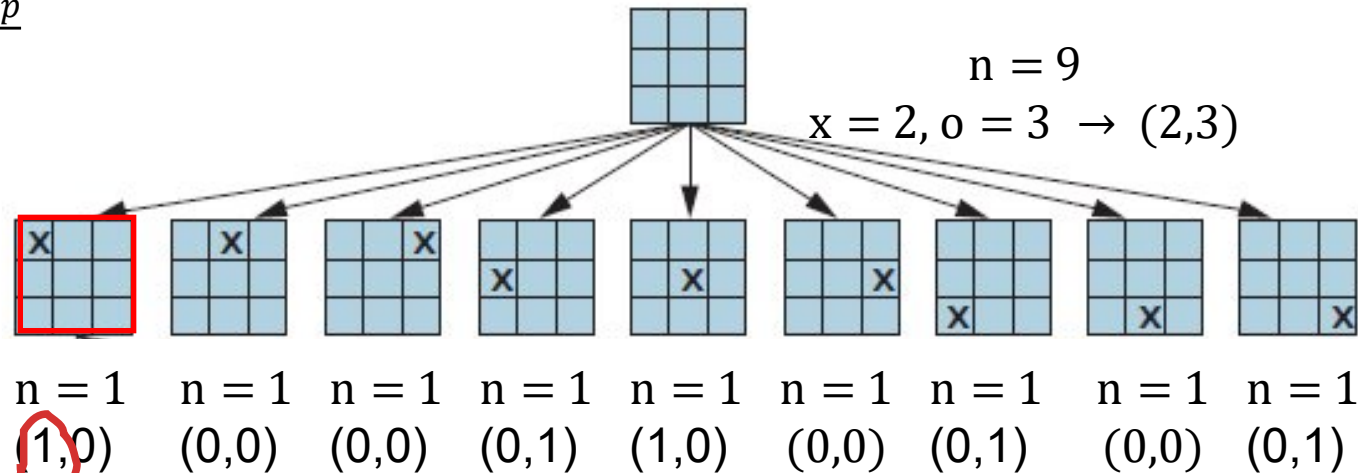
Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

$$UCB_k = v_k + C \times \sqrt{\frac{\log n_p}{n_k}}$$

mit

$$v_k = \frac{u_k}{n_k}$$



UCT: wähle den Knoten mit höchstem UCB



Haben mehrere Knoten den gleichen maximalen UCB Wert, so wird einer zufällig ausgewählt.

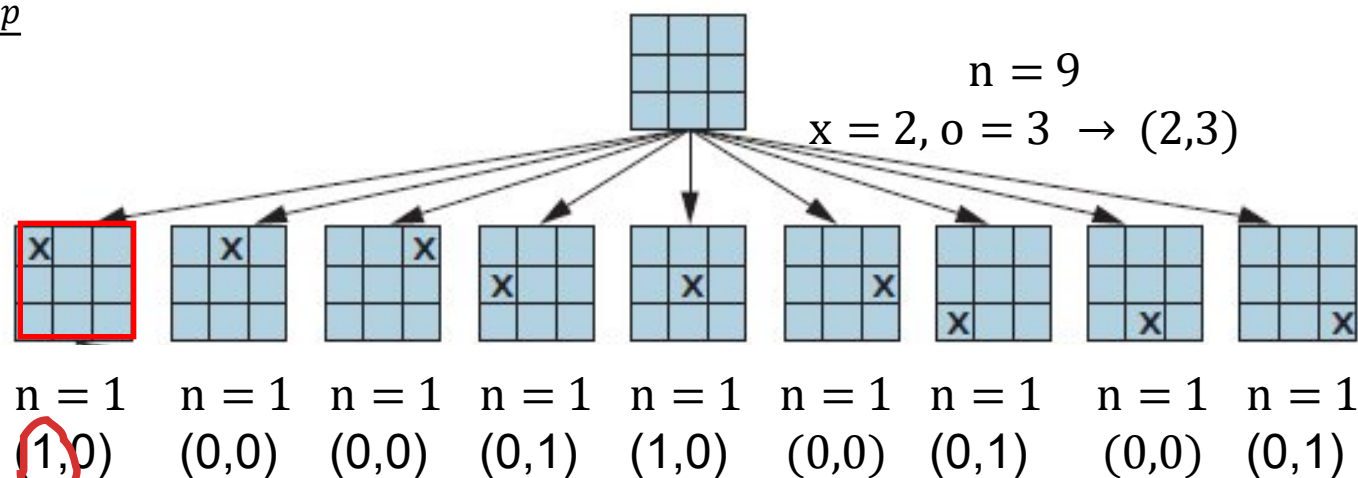
Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

$$UCB_k = v_k + C \times \sqrt{\frac{\log n_p}{n_k}}$$

mit

$$v_k = \frac{u_k}{n_k}$$



UCT: wähle den Knoten mit höchstem UCB

Achtung: v_k ist der Anteil der gewonnenen Simulationen am Knoten k
Aus Sicht des Spieler der den Zug macht, der zu Knoten k führt!



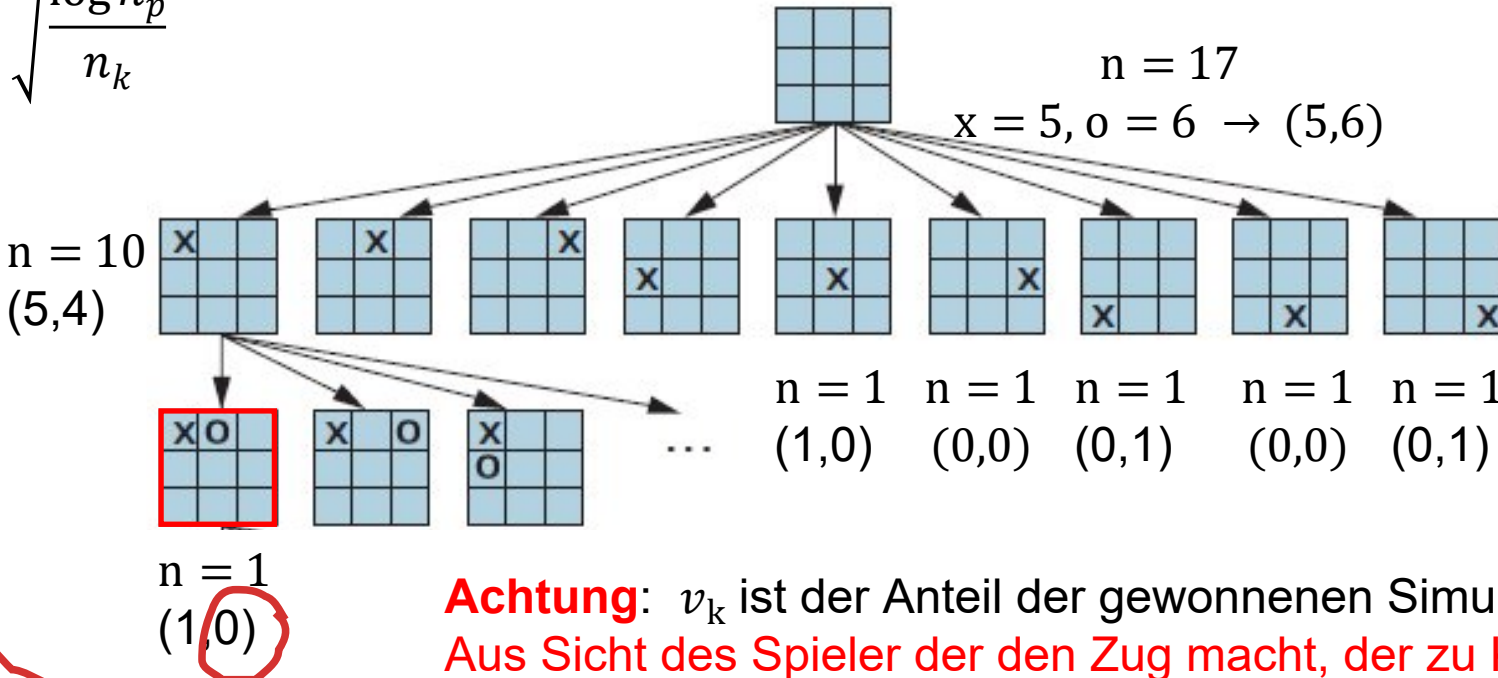
Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

$$UCB_k = v_k + C \times \sqrt{\frac{\log n_p}{n_k}}$$

mit

$$v_k = \frac{u_k}{n_k}$$



Spieler X

Spieler O

Spieler wechselt!

UCT: wähle den Knoten mit höchstem UCB

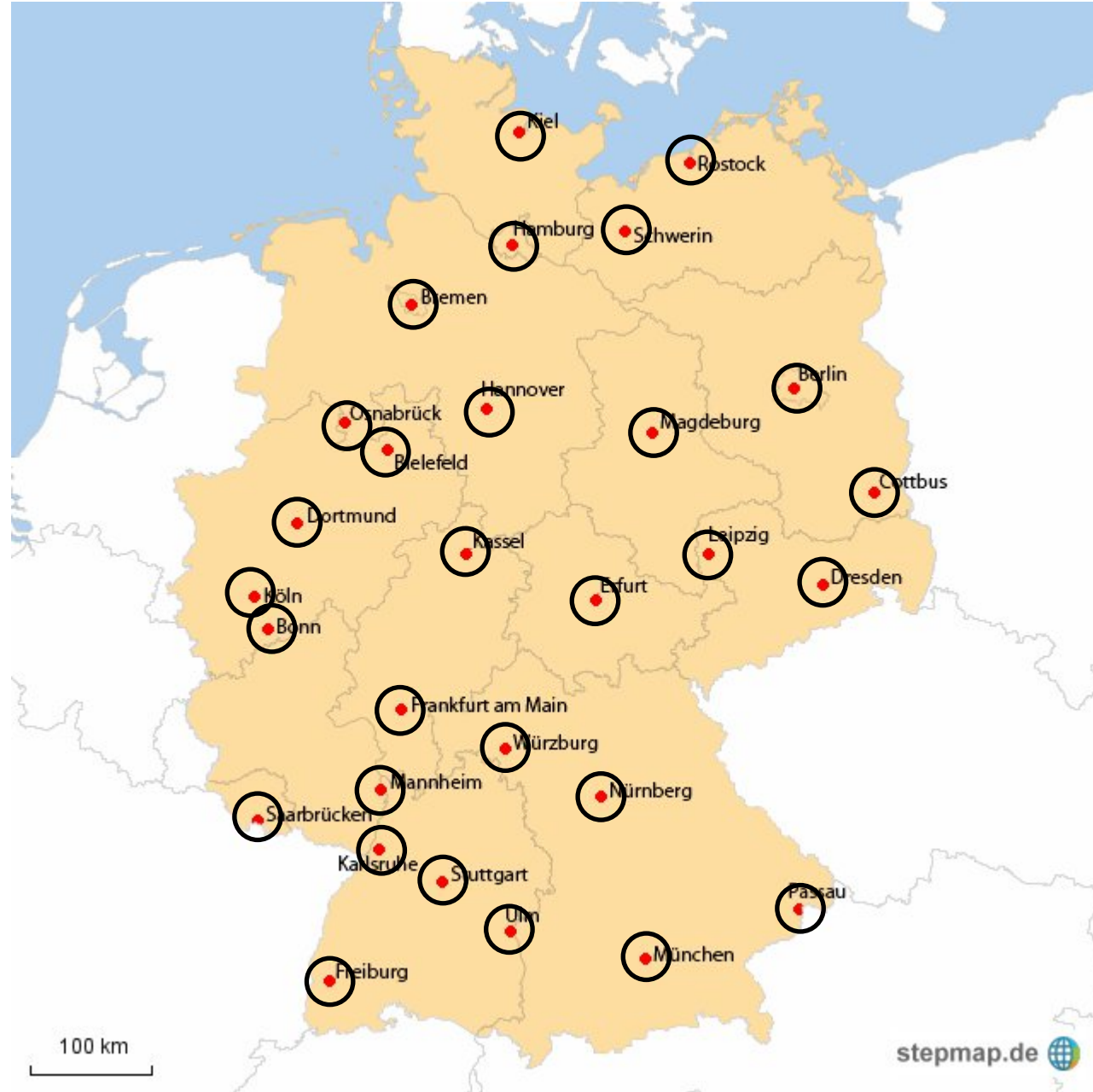
Achtung: v_k ist der Anteil der gewonnenen Simulationen am Knoten k
Aus Sicht des Spieler der den Zug macht, der zu Knoten k führt!



Dijkstra's algorithm



Dijkstra's algorithm



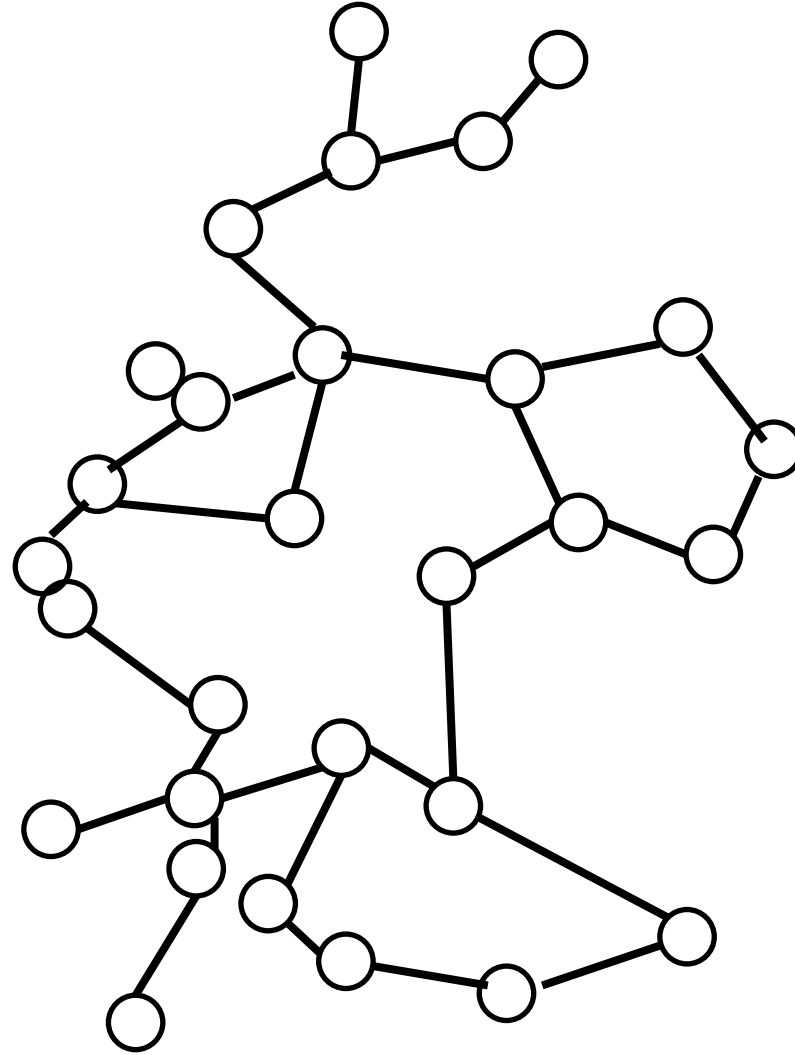
Ungerichteter Graph

- Knoten sind die Städte
- Kanten sind ungerichtete (bidirektionale) Verbindungen



Ungerichteter Graph

- Knoten sind die Städte
- Kanten sind ungerichtete (bidirektionale) Verbindungen



Ungerichteter Graph



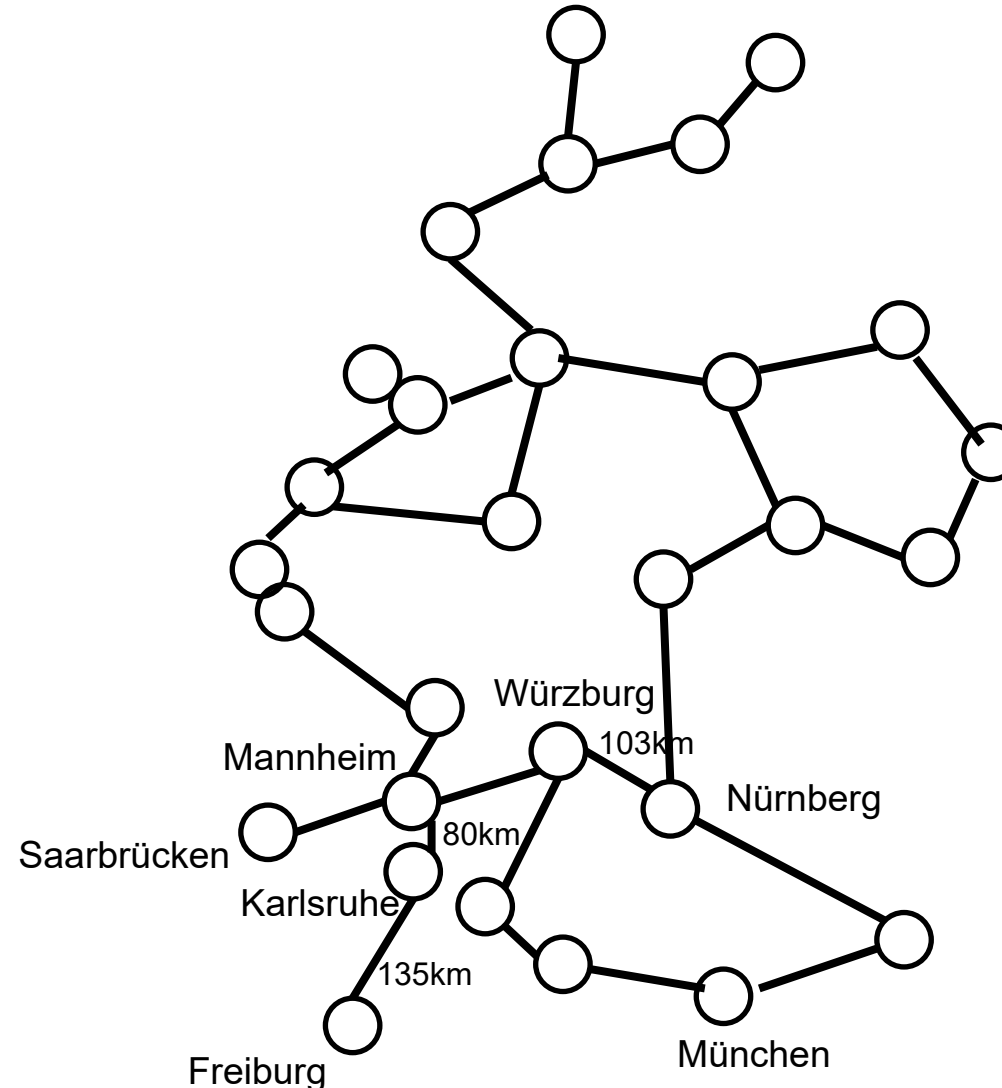
- Knoten sind die Städte
- Kanten sind ungerichtete (bidirektionale) Verbindungen

Knoten

- Informationen Stadt
- Position (z.B. GPS Koord.)

Kanten

- Distanz
- Info. z.B. Art der Straße



Gerichteter Graph



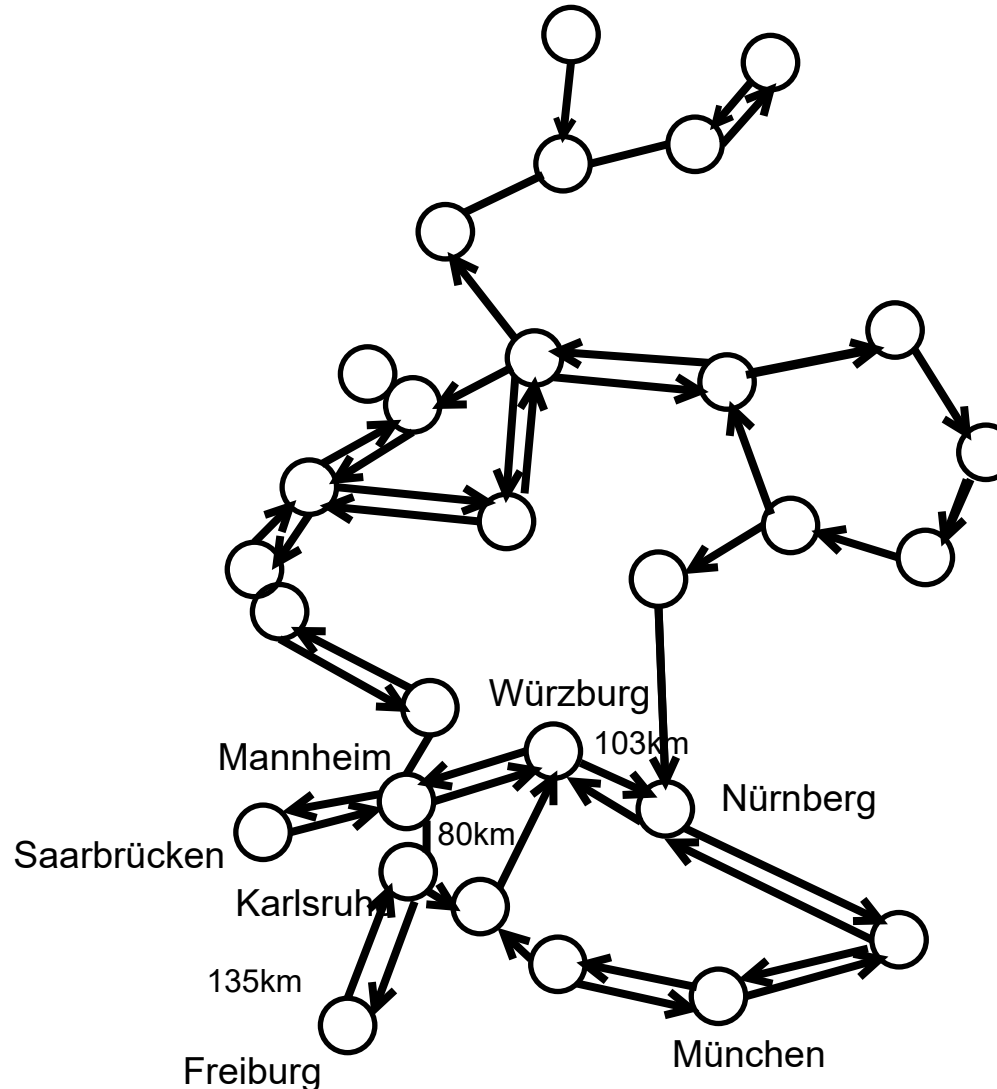
- Knoten sind die Städte
- Kanten sind gerichtete Verbindungen

Knoten

- Informationen Stadt
- Position (z.B. GPS Koord.)

Kanten

- Distanz
- Info. z.B. Art der Straße

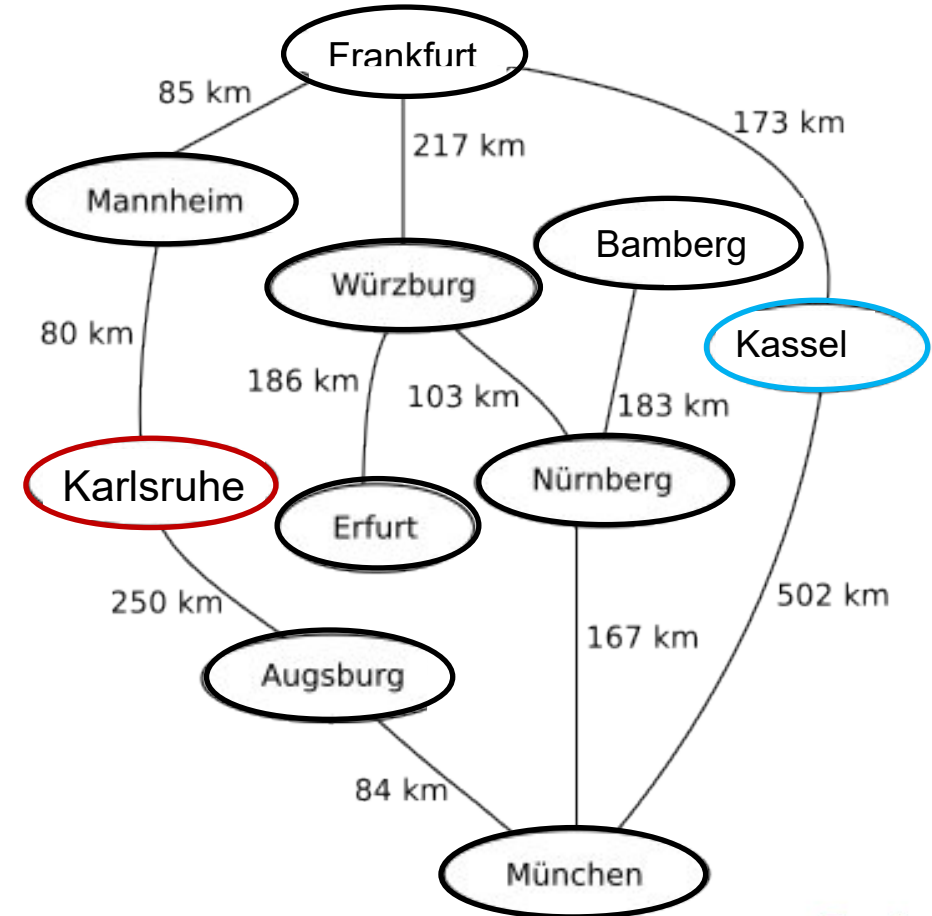


Dijkstra's algorithm



- löst das Problem der kürzesten Pfade
- zwischen Startknoten und einem Zielknoten
- kantengewichtete Graph

z.B. von **Karlsruhe** nach **Kassel**

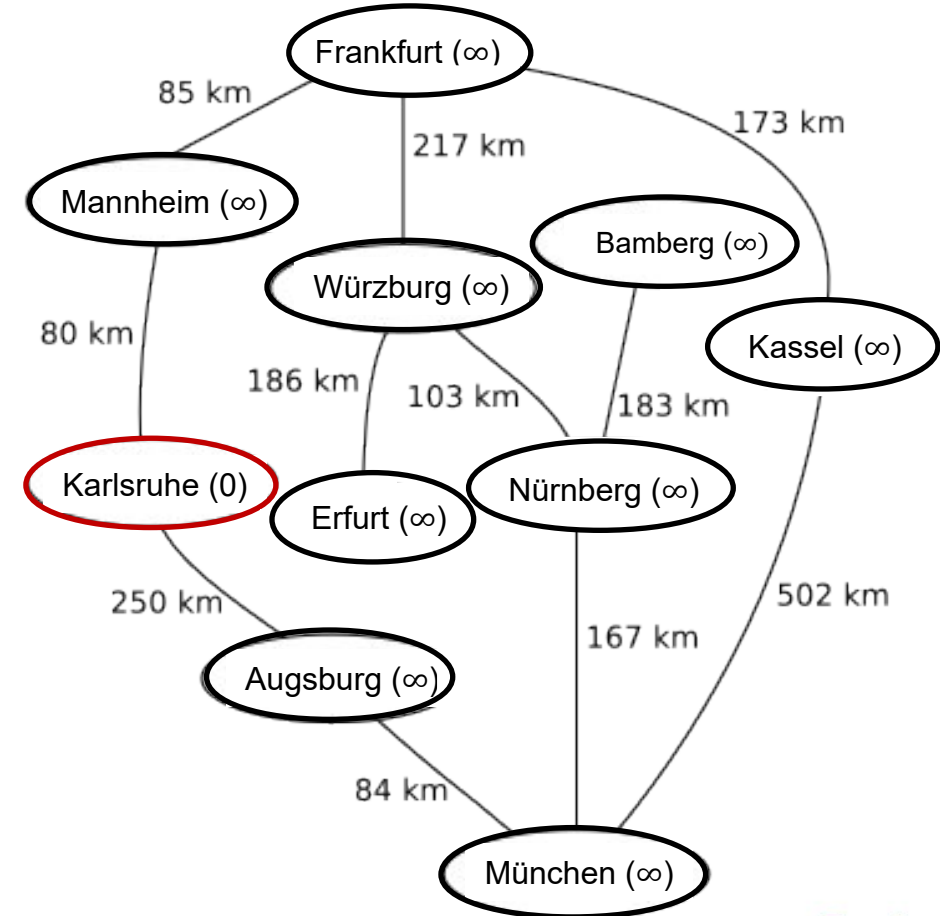


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die **Eigenschaften *Distanz* & *Vorgänger***.
Initialisiere: Distanz (**Startknoten**=0, ∞ alle anderen).

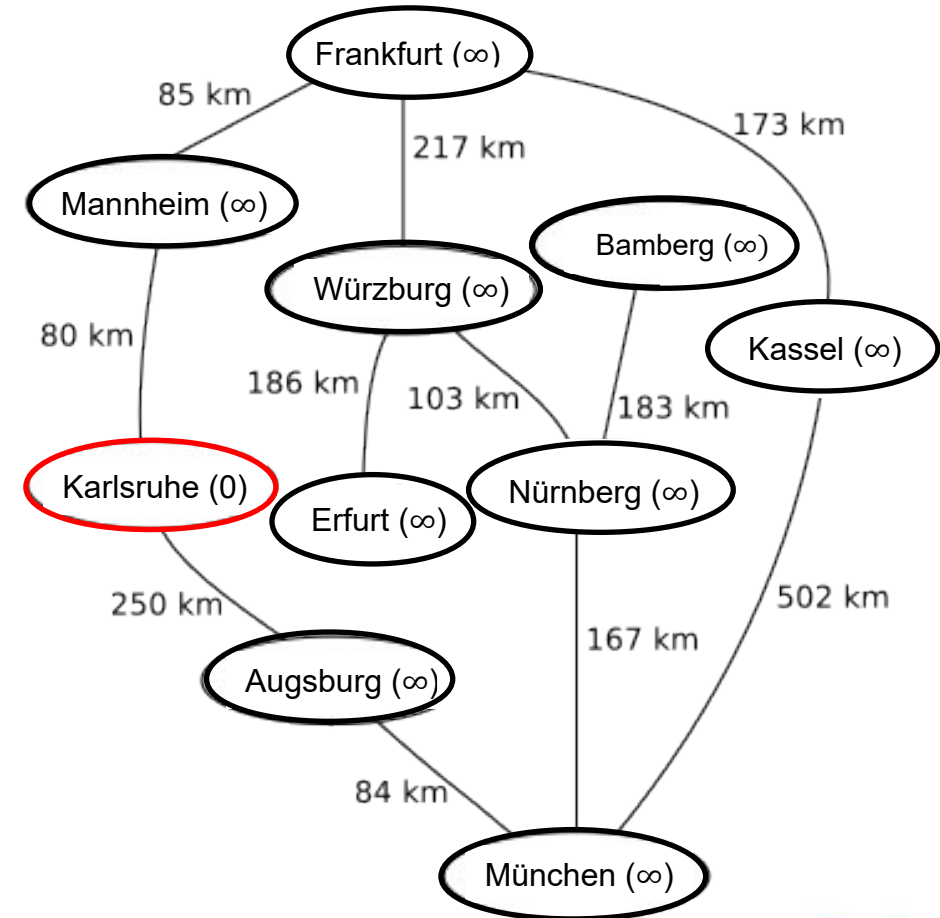


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus

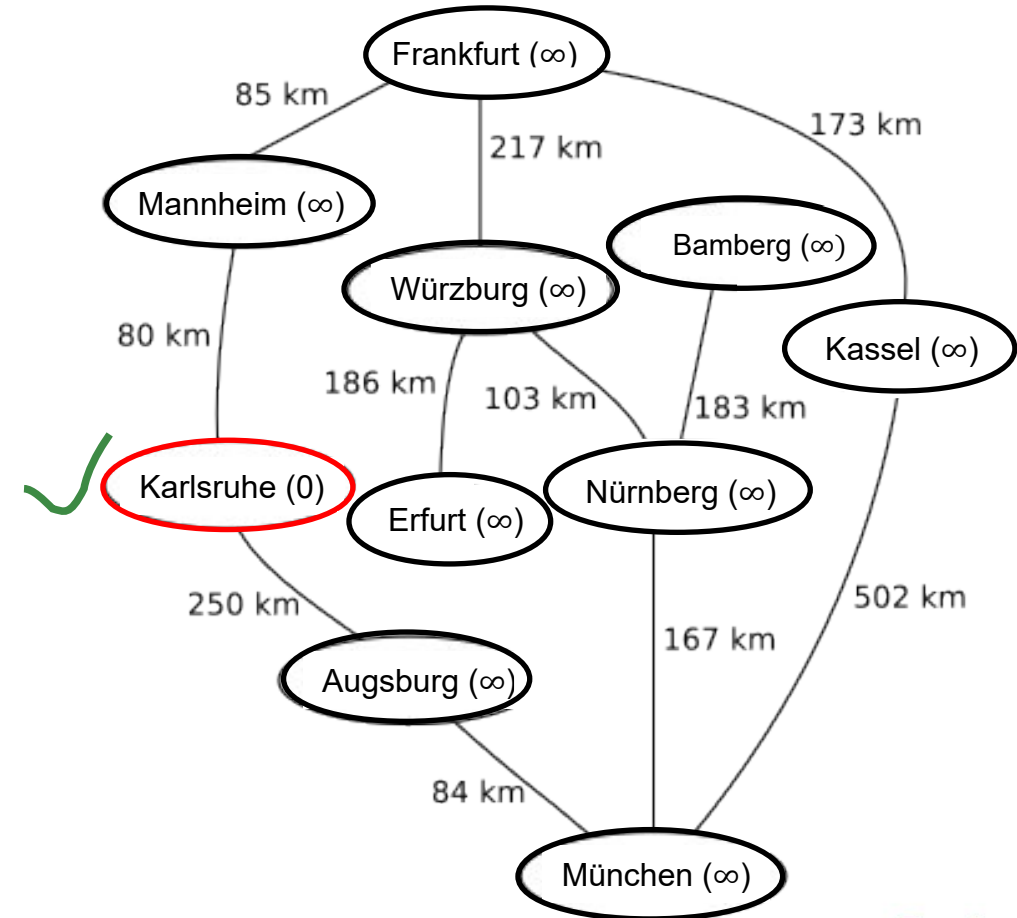


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
1. Markiere diesen Knoten **als besucht**.

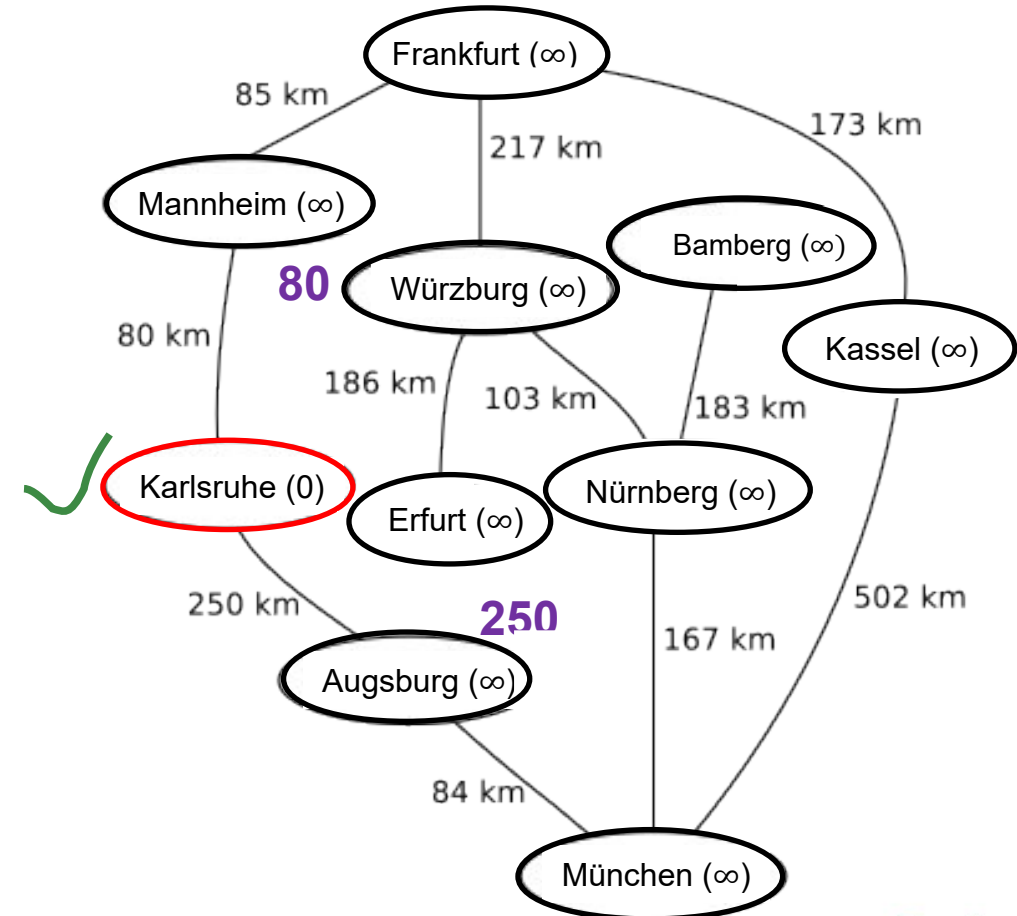


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).

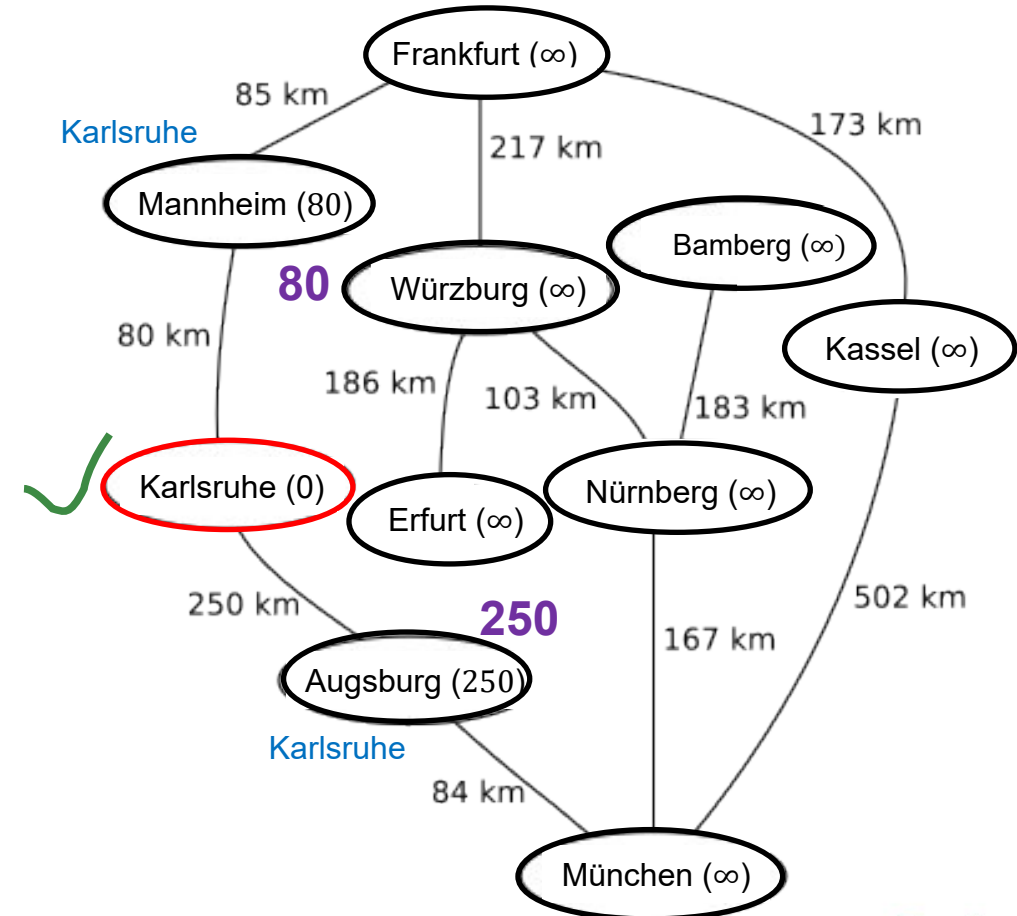


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. Berechne für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. **Relaxation**: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

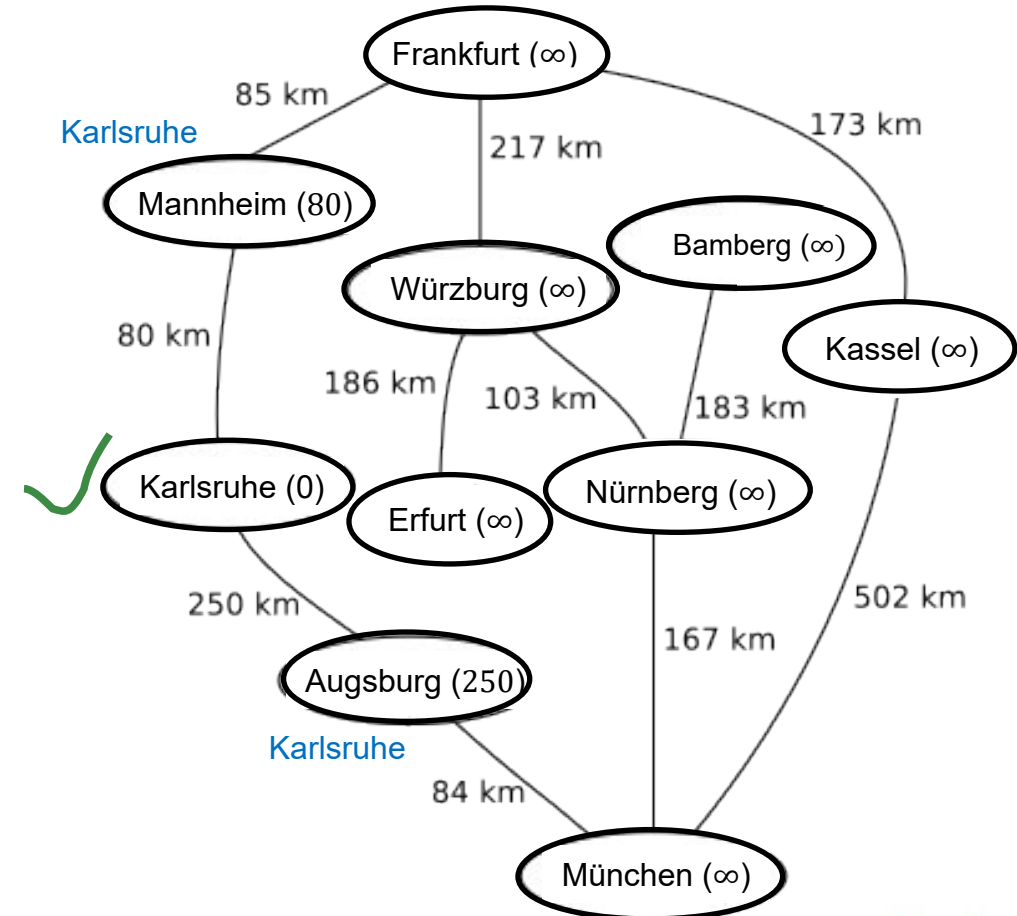


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. Berechne für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. **Relaxation**: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

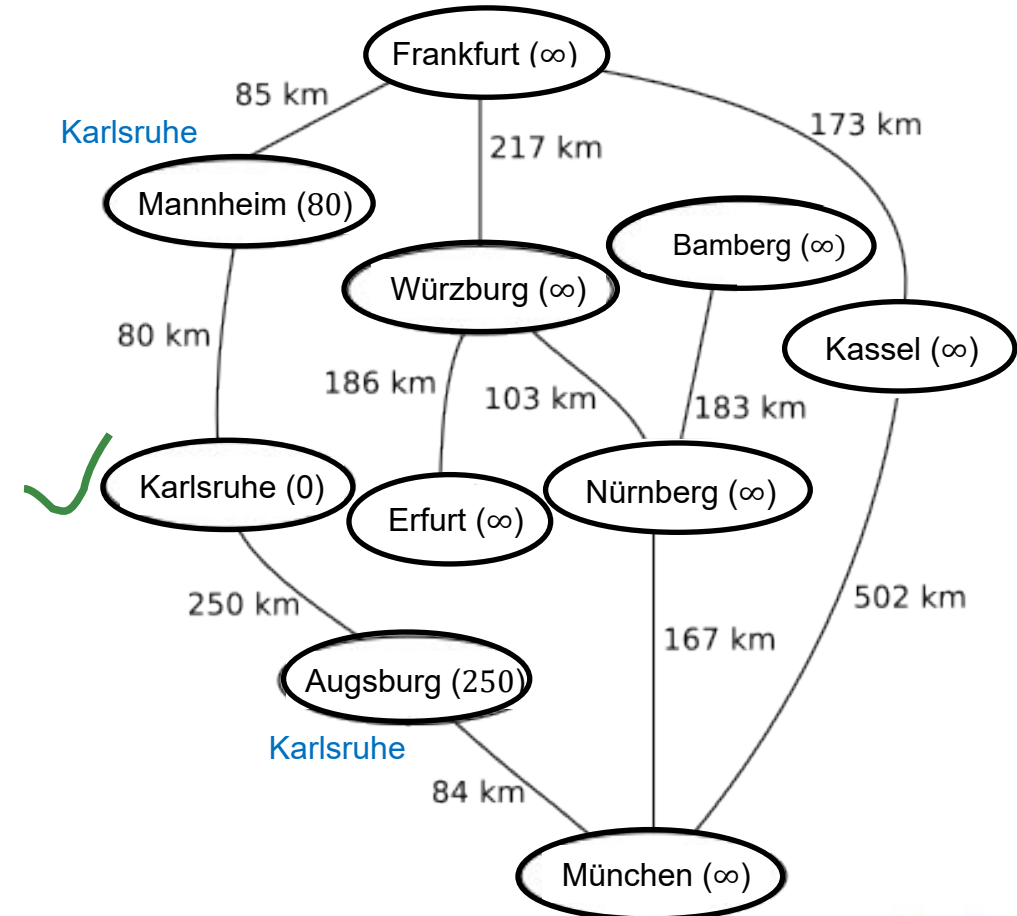


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
 3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

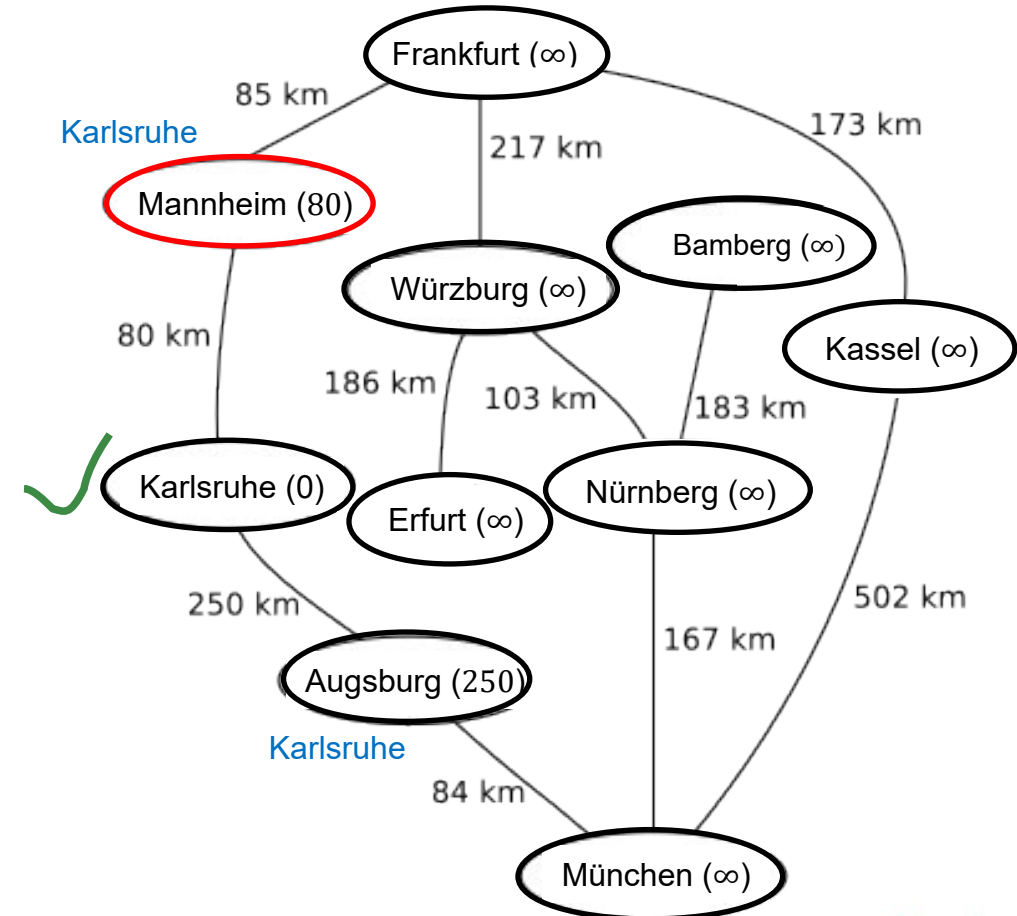


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus

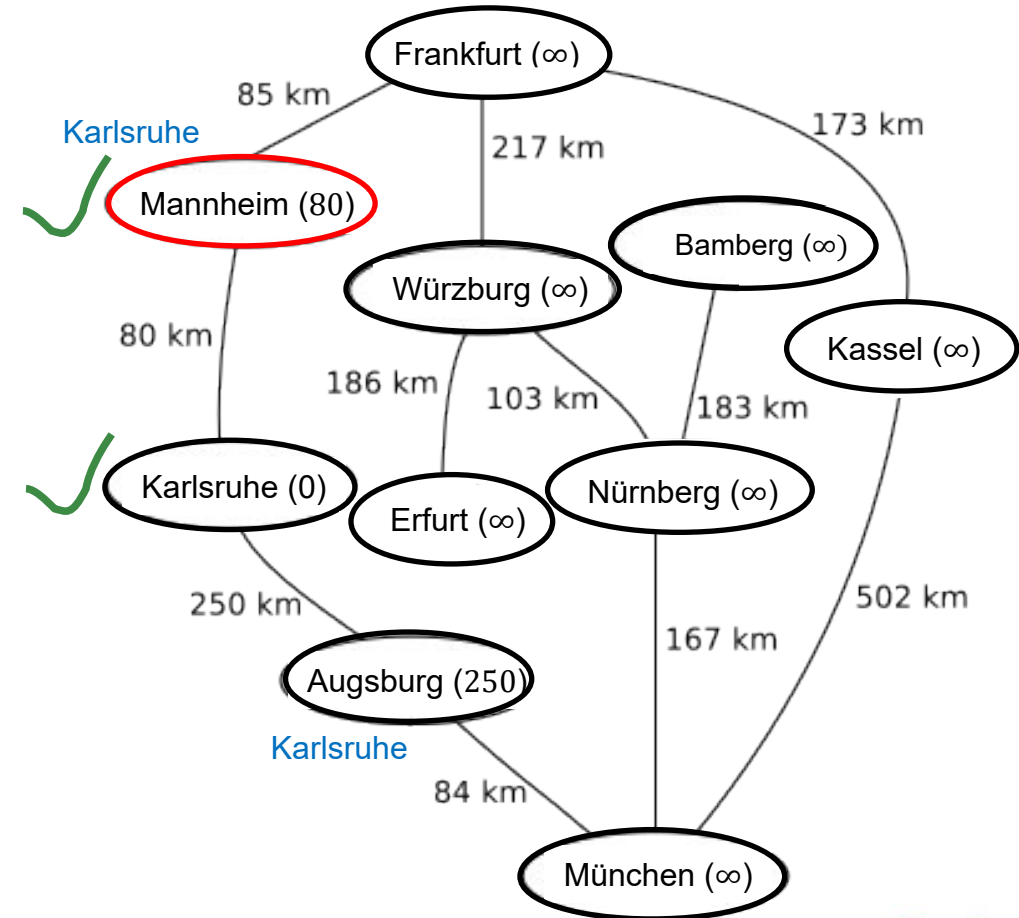


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
1. Markiere diesen Knoten **als besucht**.

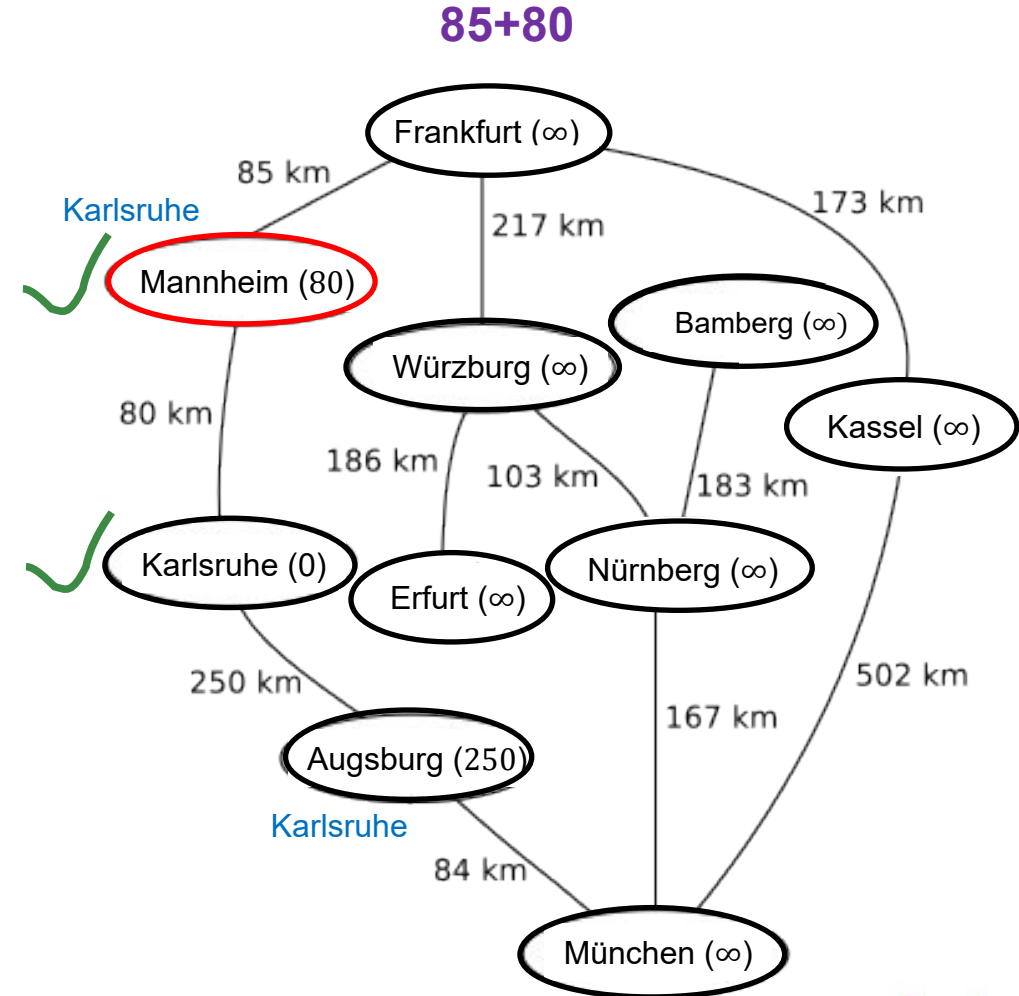


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).

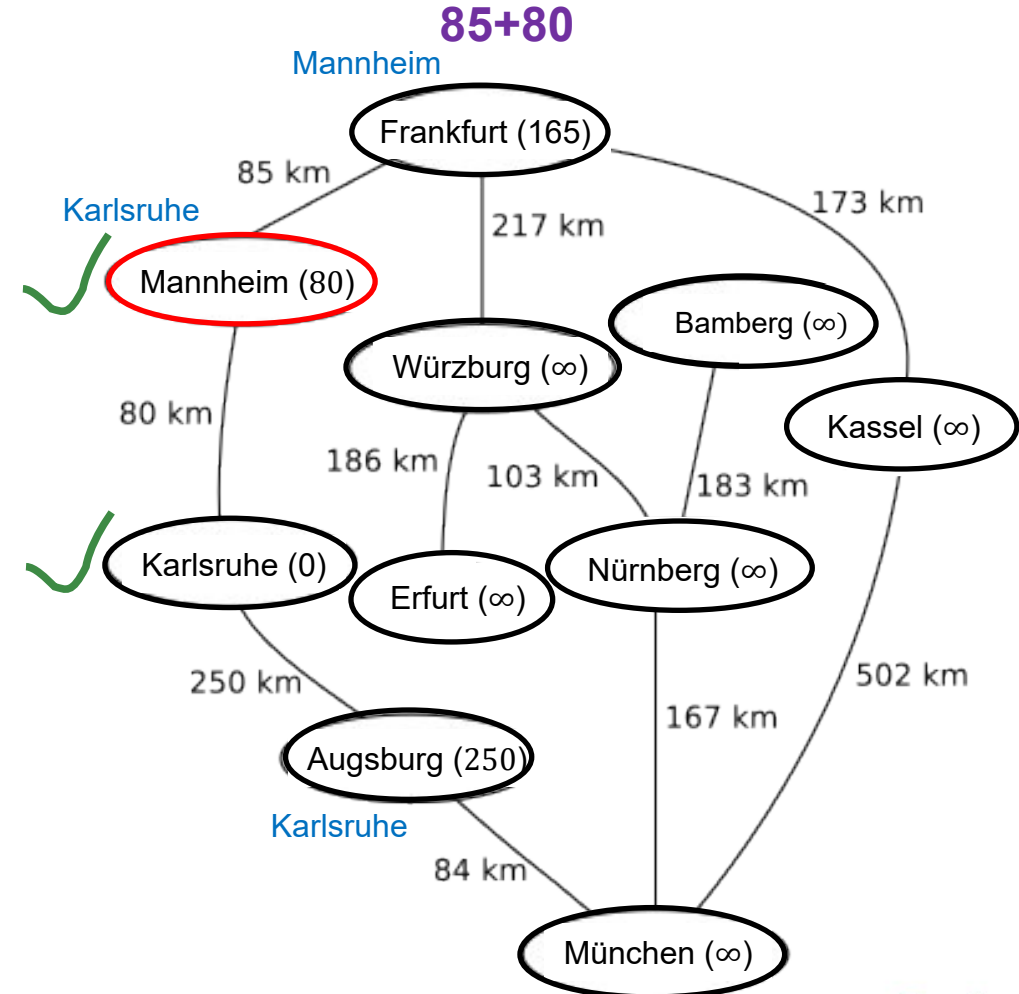


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. **Relaxation**: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

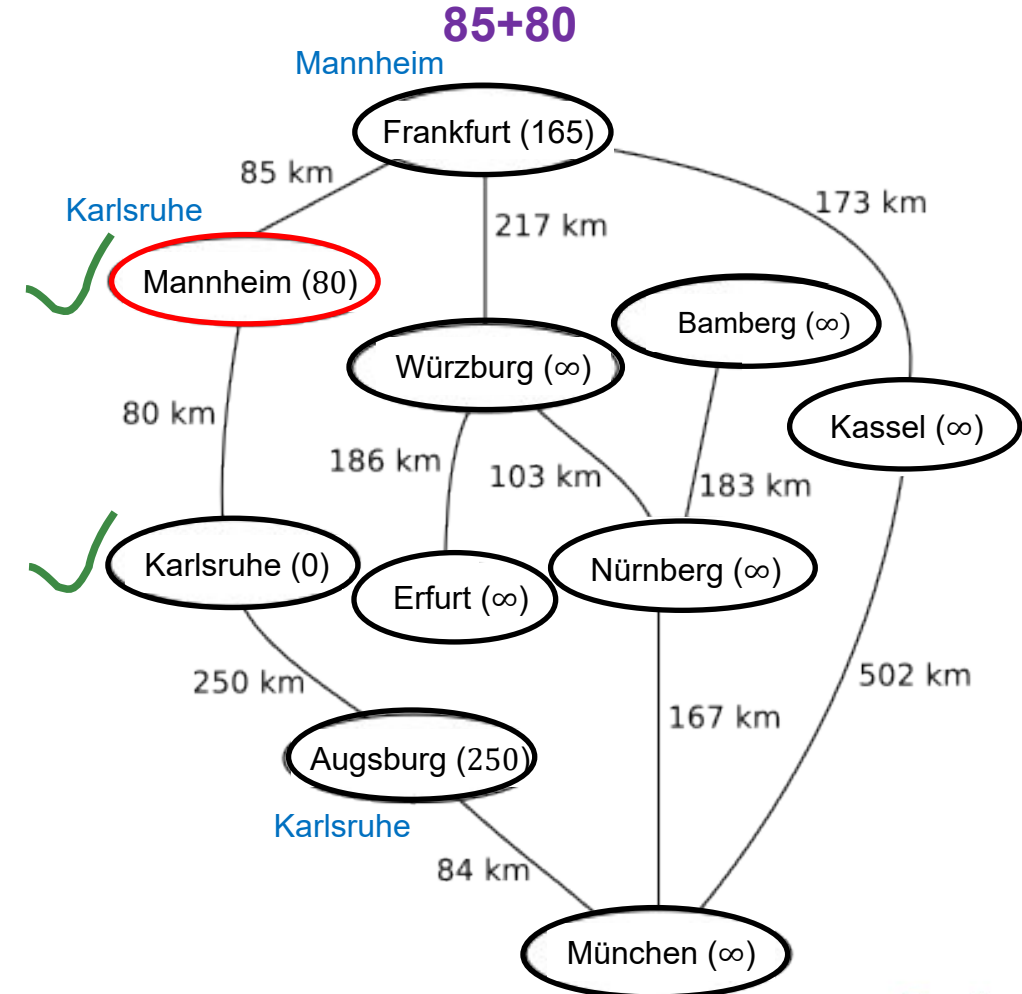


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
 3. **Relaxation**: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

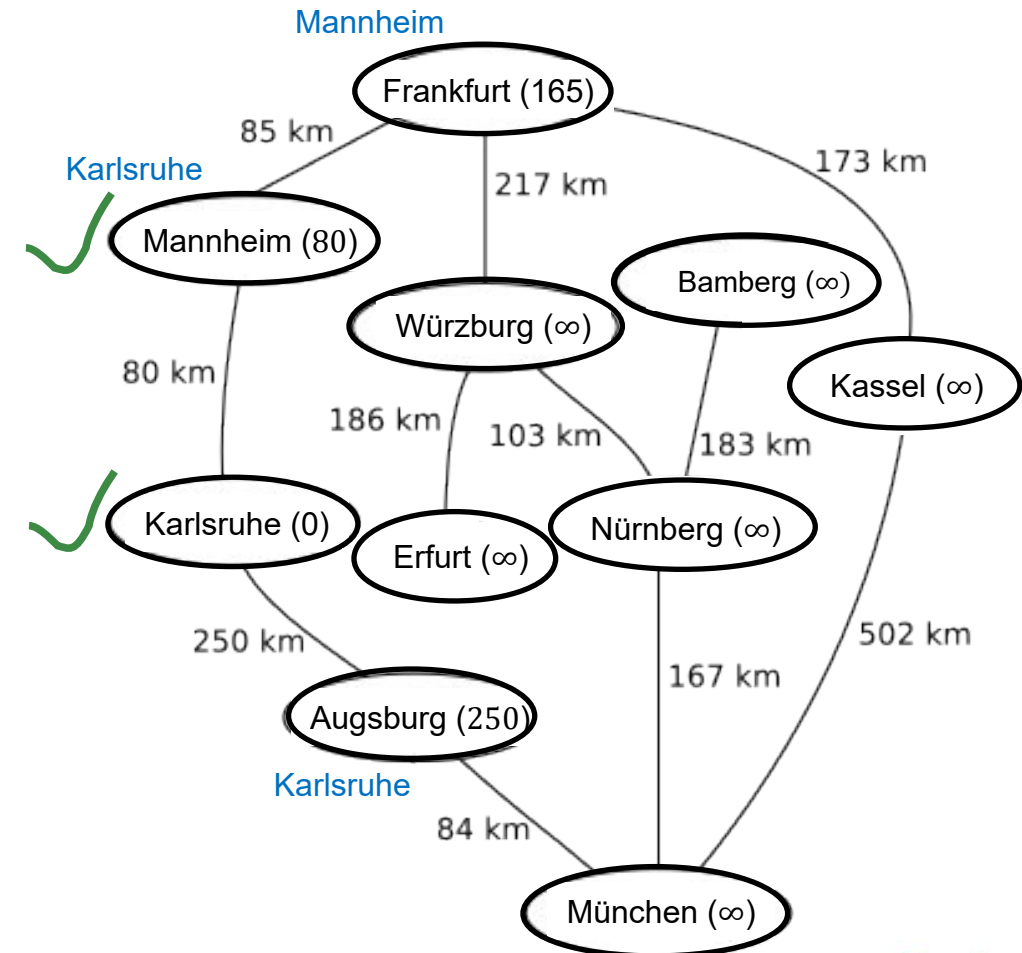


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

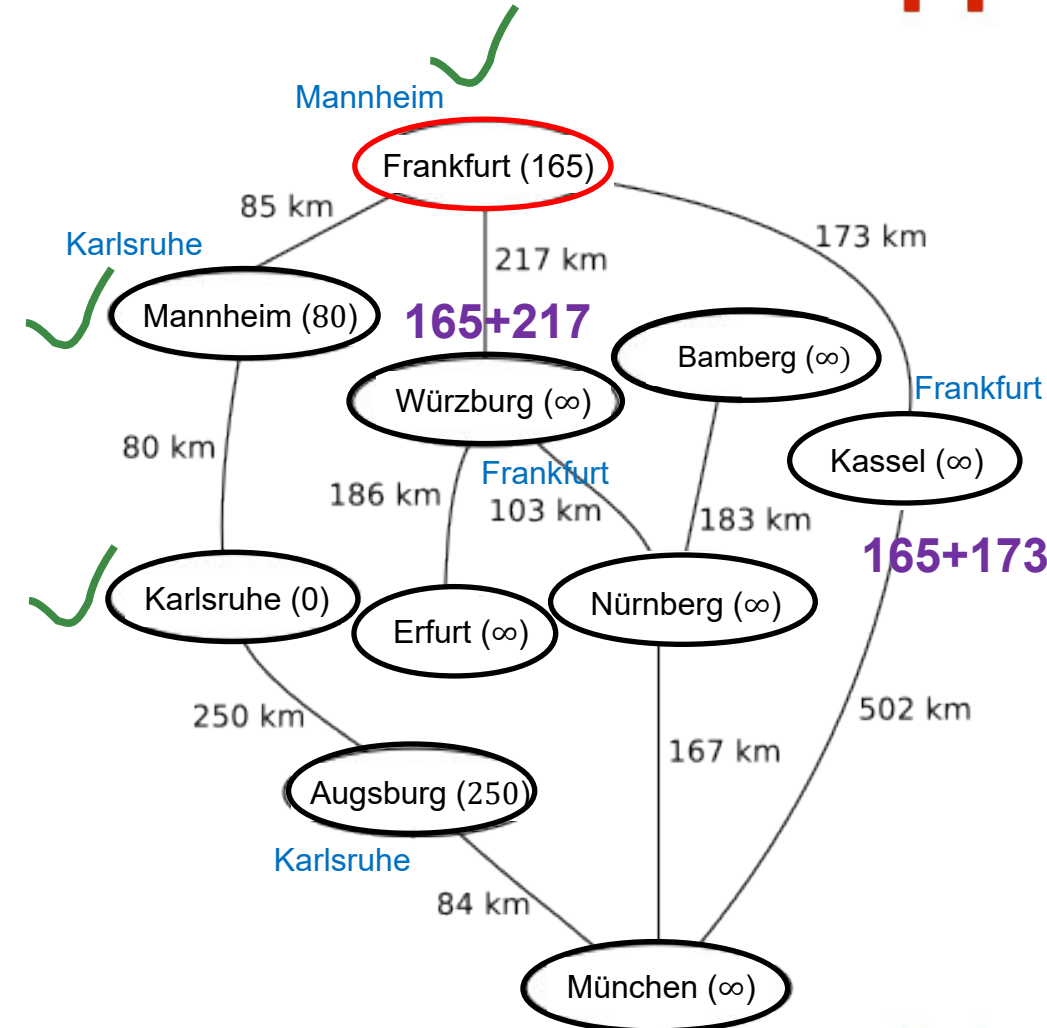


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.



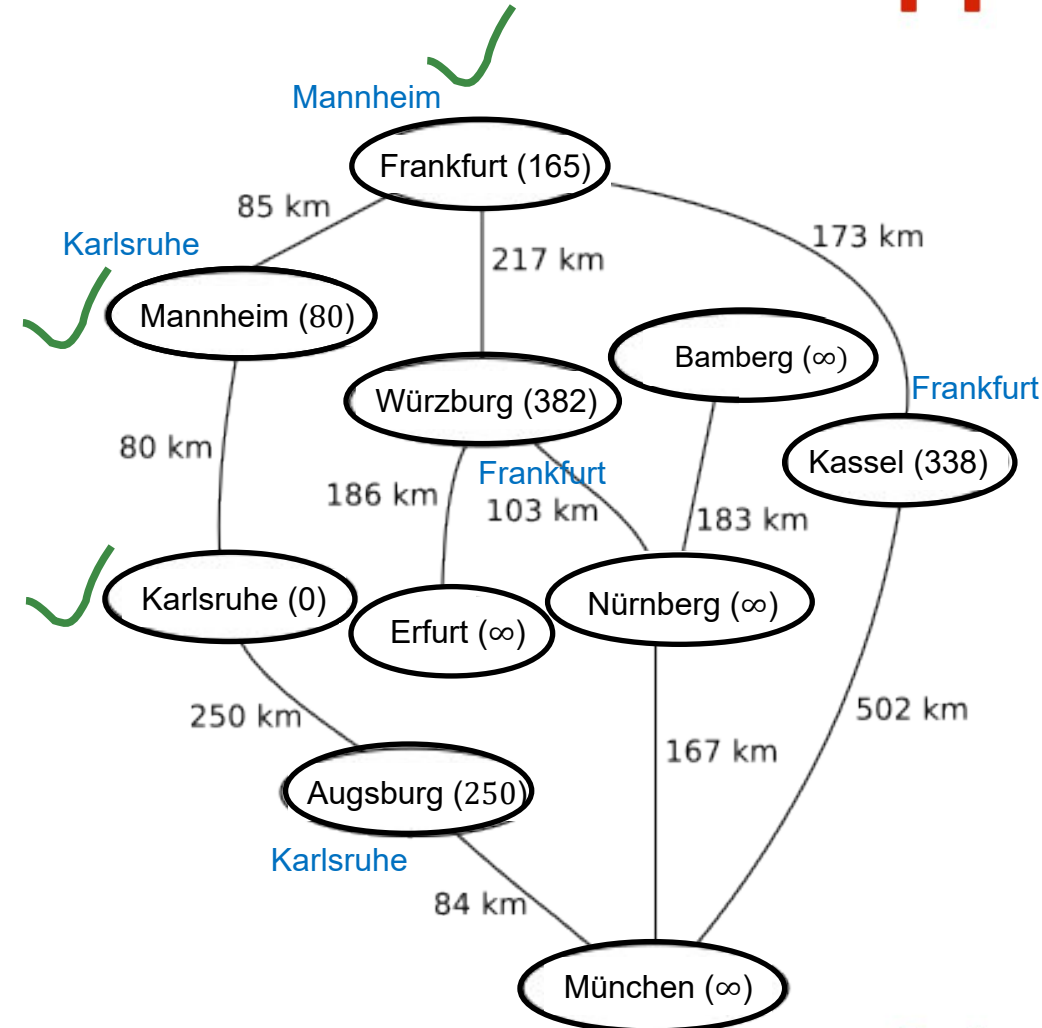
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



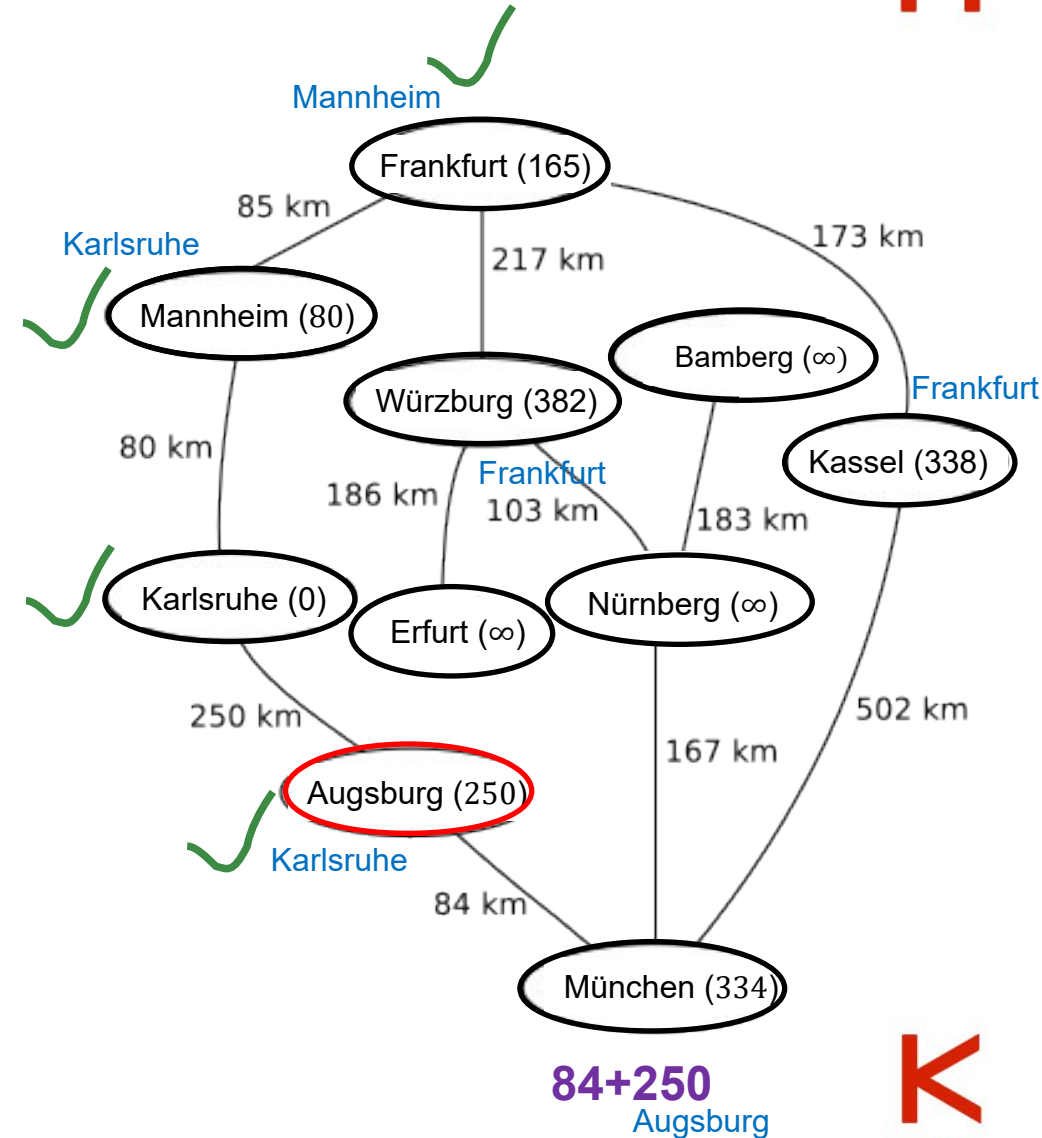
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



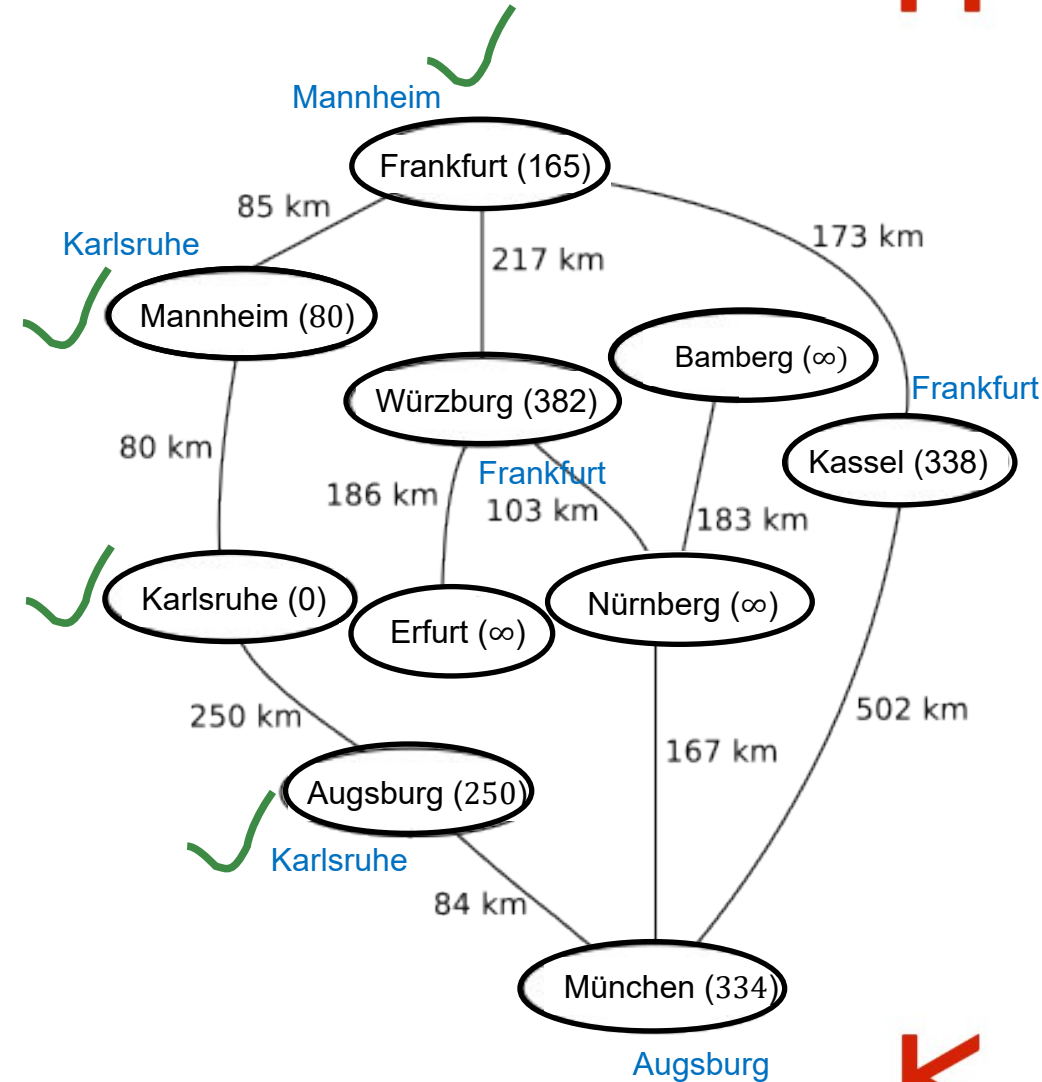
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



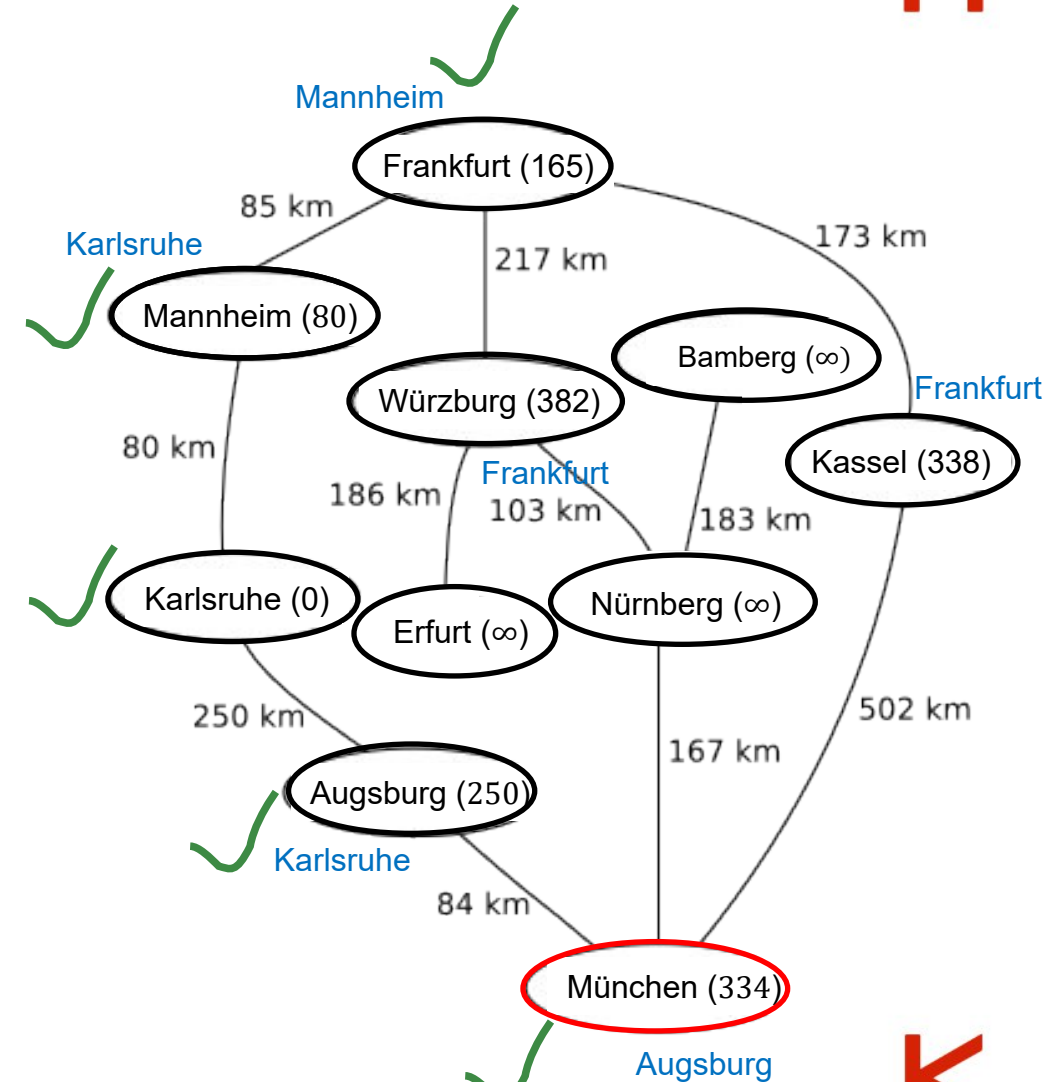
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



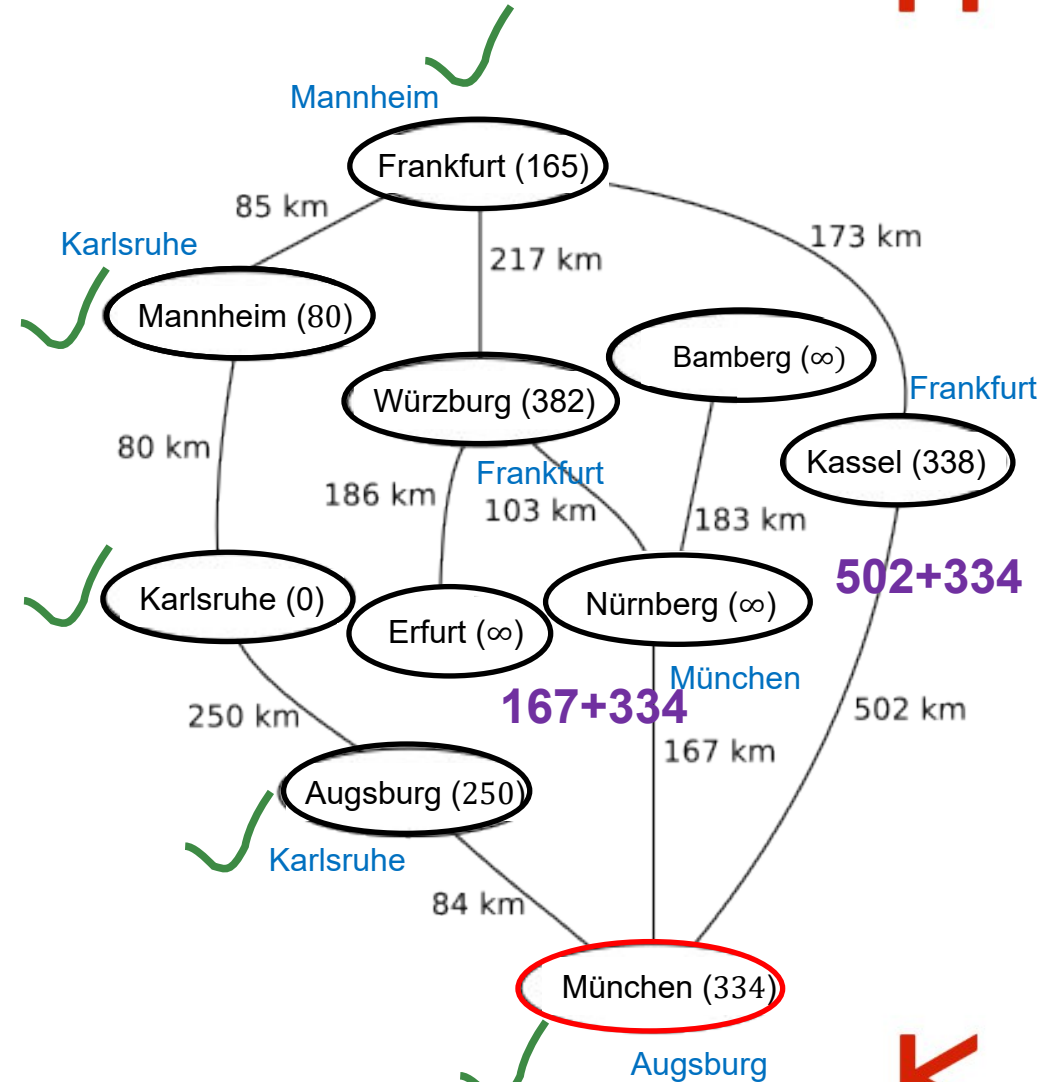
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.

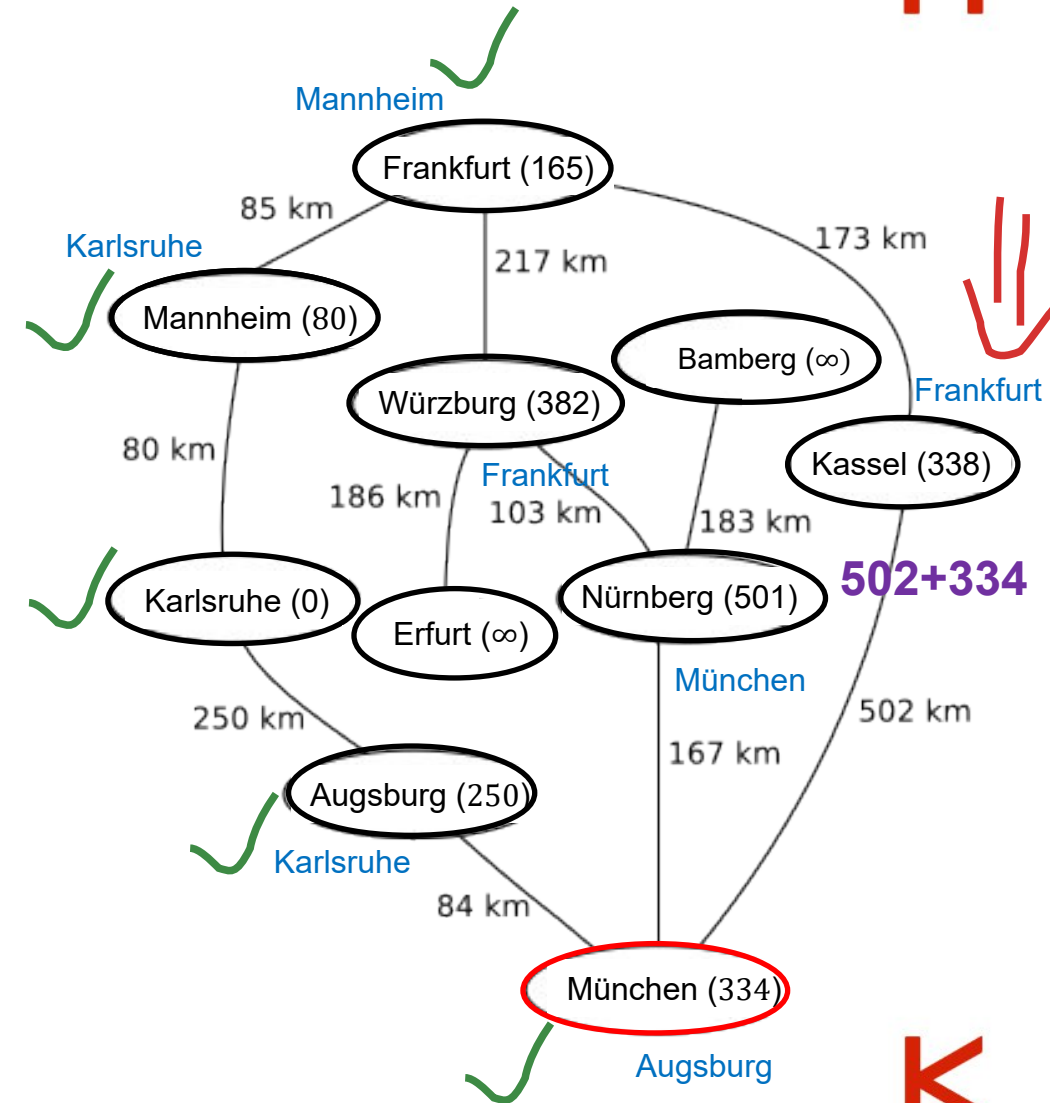


Dijkstra's algorithm (single-pair shortest path)

z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten)
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



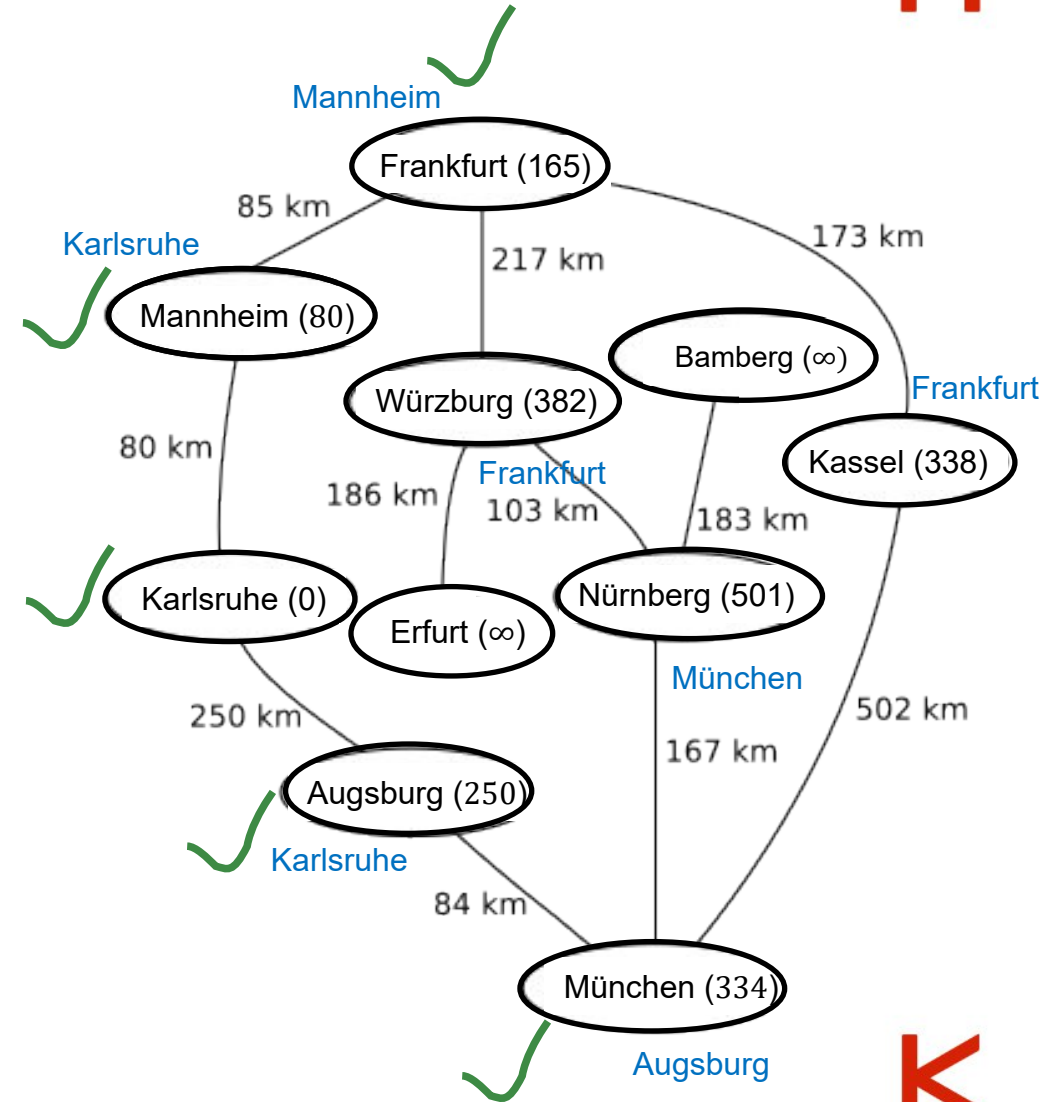
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



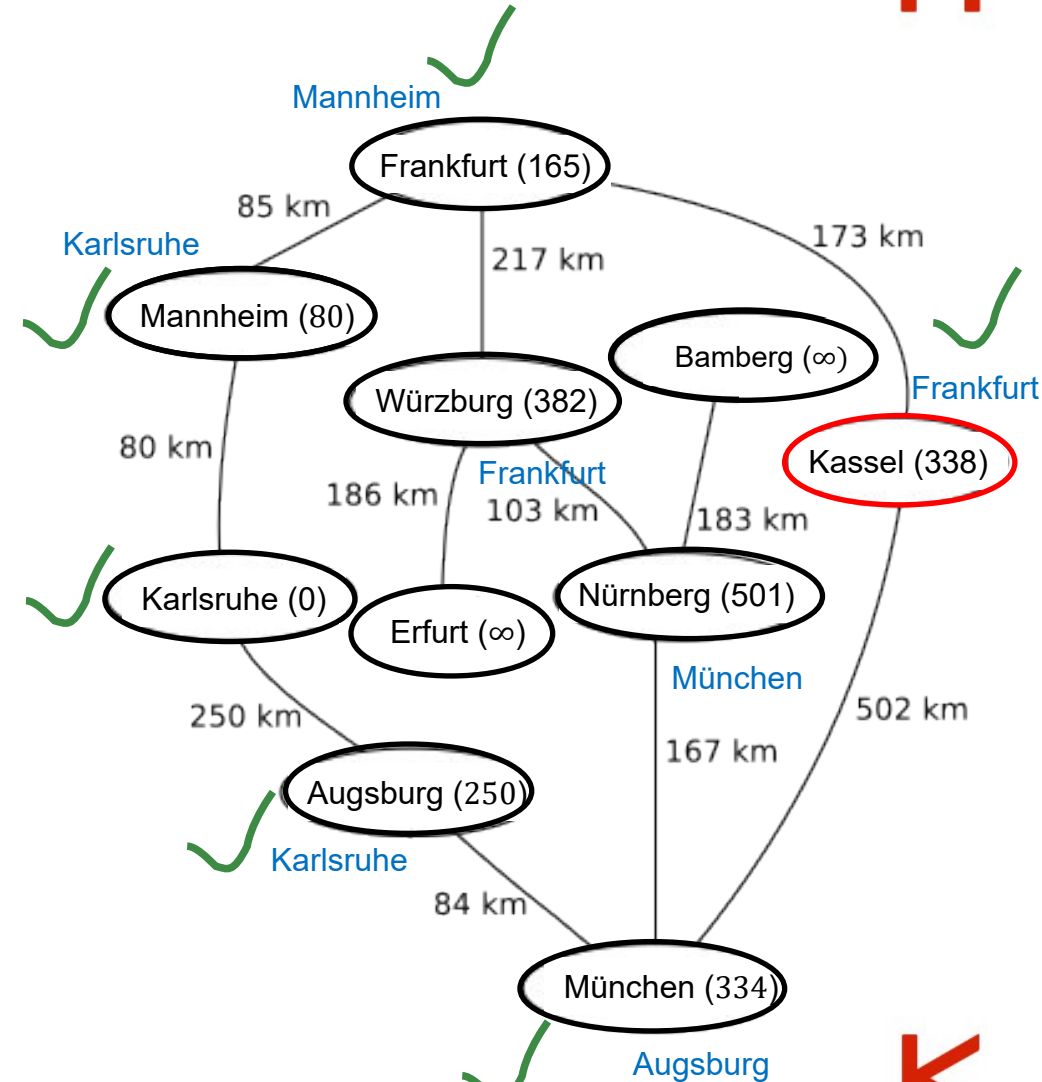
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach **Kassel**

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man **in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist**.



Dijkstra's algorithm (single-pair shortest path)

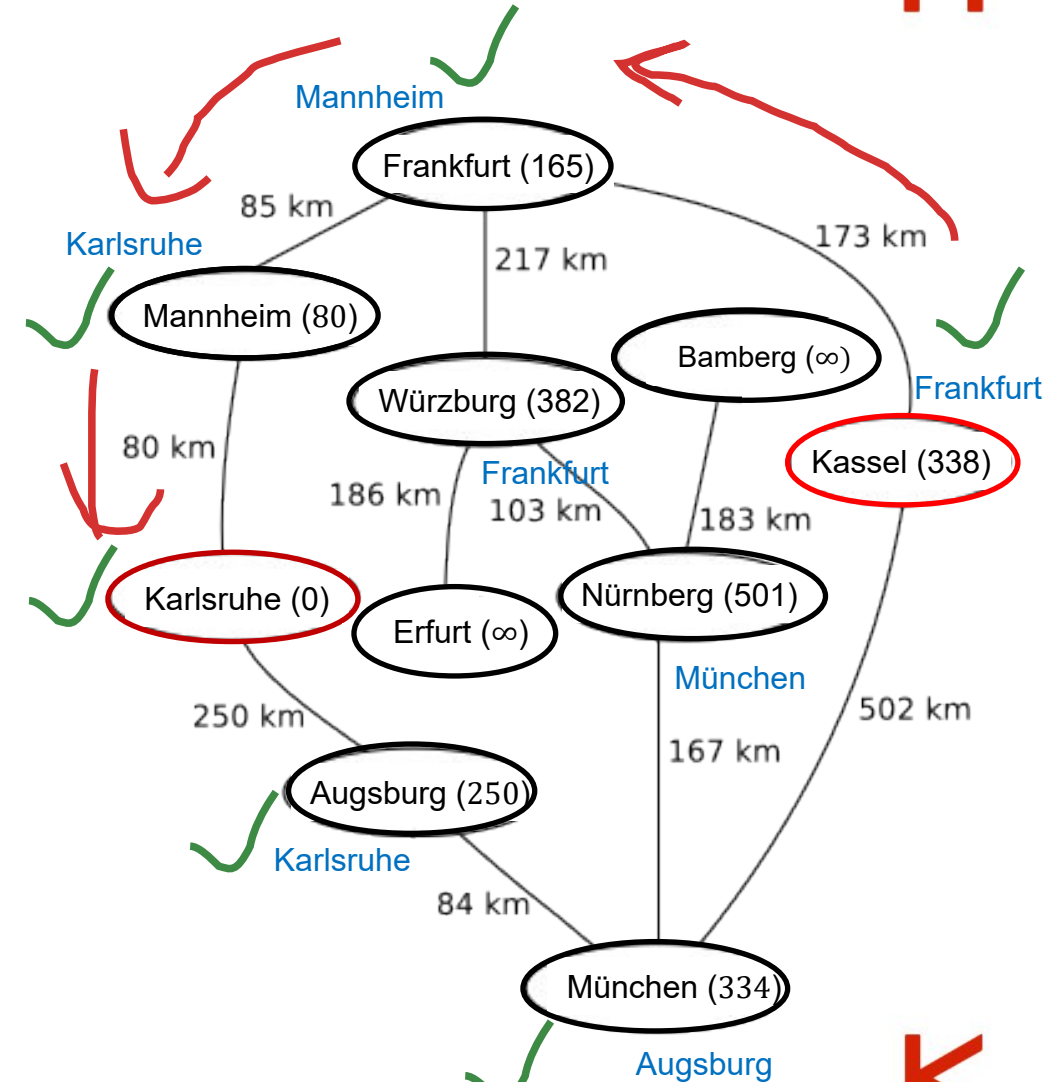
Kürzeste Strecke 338km



z.B. von **Karlsruhe** nach **Kassel**

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man **in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist**.



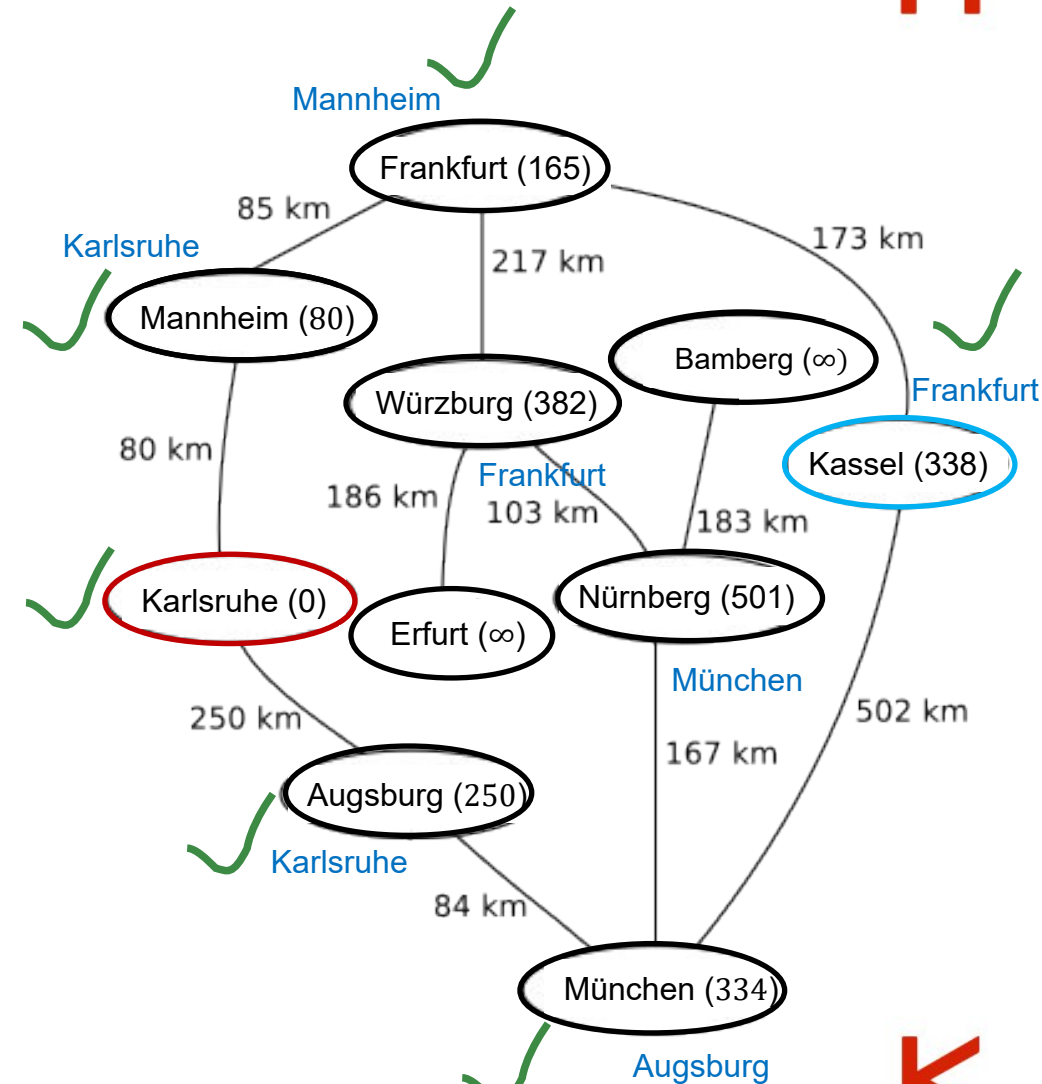
Dijkstra's algorithm (single-pair shortest path)



z.B. von **Karlsruhe** nach **Kassel**

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, wähle denjenigen mit minimaler (aufsummierter) Distanz aus
 1. Markiere diesen Knoten als besucht.
 2. Berechne für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als Vorgänger.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



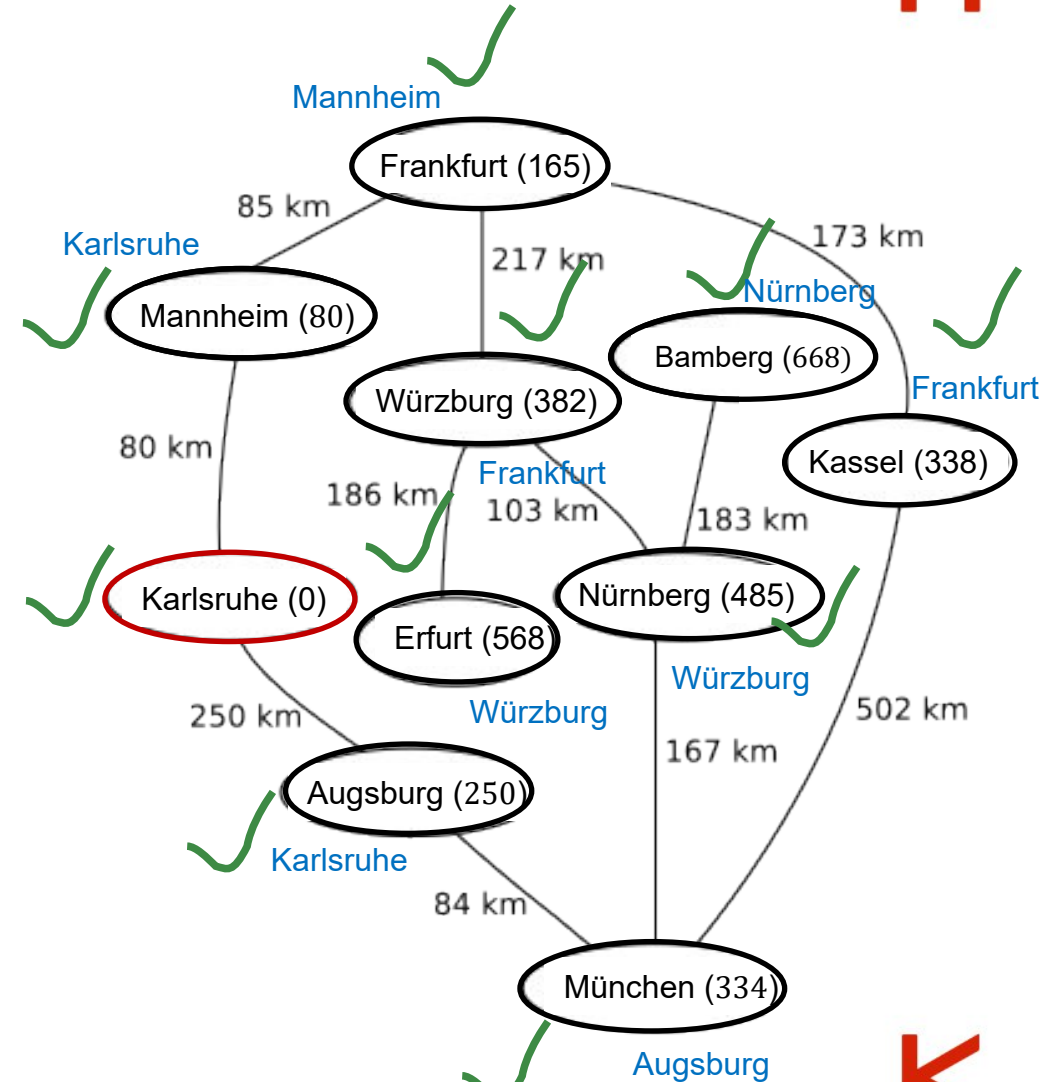
Dijkstra's algorithm (single-source shortest path)



z.B. von **Karlsruhe** zu allen anderen Orten

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, wähle denjenigen mit minimaler (aufsummierter) Distanz aus
 1. Markiere diesen Knoten als besucht.
 2. Berechne für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als Vorgänger.

~~Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.~~





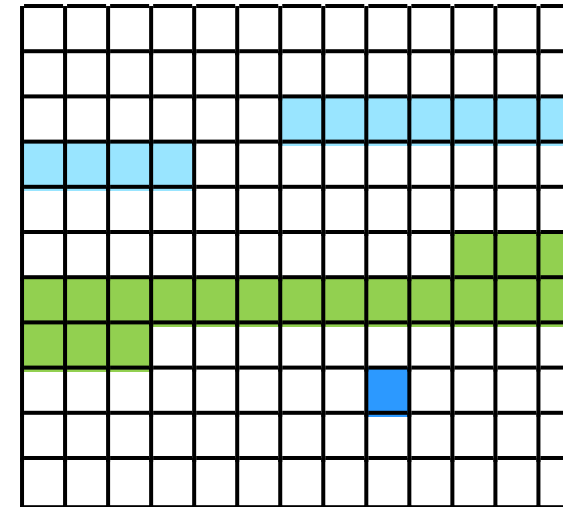
C++

Dynamische Speicherverwaltung - Grundlagen



- **Statische Deklaration:** Bis zu diesem Punkt der Vorlesung wurden Variablen statisch deklariert, das heißt, die Anzahl der Variablen und ihre Lage im Speicher war fest und bekannt.
- Häufig müssen aber **dynamische Speichermengen** verwaltet werden; so ändert sich die Zahl der Studierenden einer Hochschule beispielsweise mit jedem Semester
- Daher ist **Nutzung einer dynamische Speicherverwaltung** in Software-Applikationen allgegenwärtig.
- Eine solche dynamische Speicherverwaltung wird meist **vom Betriebssystem angeboten** und von gängigen **Programmiersprachen** unterstützt.
- Mittels **spezieller Funktionen** können vom Freispeicher (Halde oder Heap) Speicherbereiche unterschiedlicher Größe
 - angefordert werden ("**new**", Allokation),
 - wie eine normale Variable genutzt werden (über Zeiger) und
 - wieder freigegeben werden ("**delete**")
- Bemerkung: dieser "Heap" ist nicht mit der gleichnamigen Datenstruktur zu verwechseln

Haldenspeicher ("Heap")



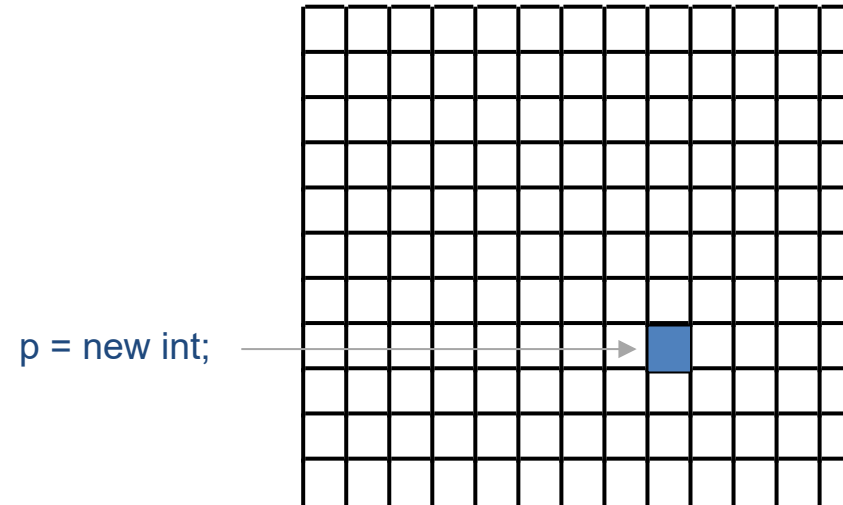
Dynamische Speicherverwaltung - C++



- Zugriff erfolgt ausschließlich über Zeiger!

```
// Einfacher Zeiger  
int* p = NULL;  
p = new int;
```

Haldenspeicher ("Heap")



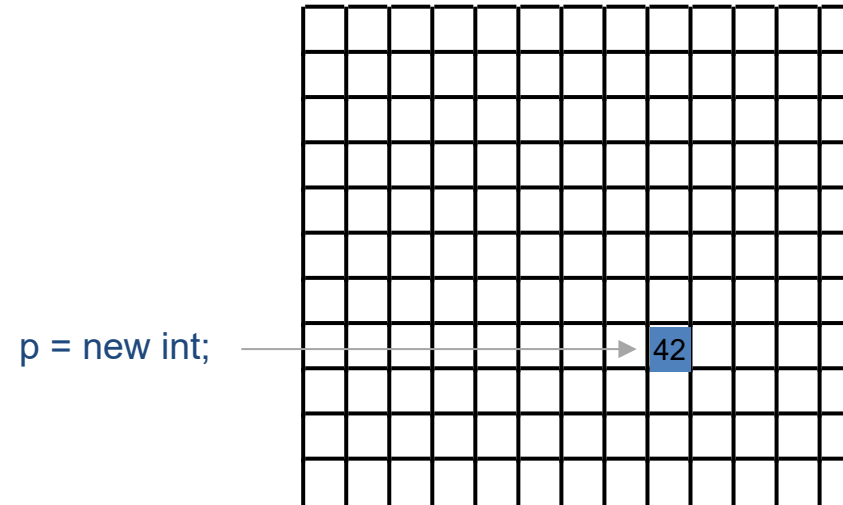
Dynamische Speicherverwaltung - C++



- Zugriff erfolgt ausschließlich über Zeiger!

```
// Einfacher Zeiger  
int* p = NULL;  
p = new int;  
*p = 42;  
cout << "p = " << *p << endl;
```

Haldenspeicher ("Heap")



Dynamische Speicherverwaltung - C++

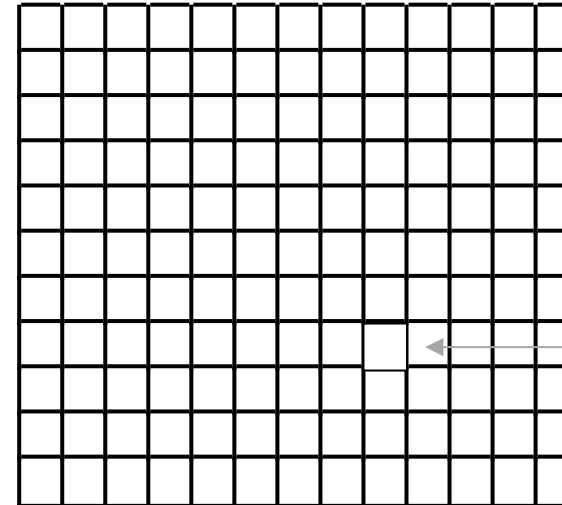


- Zugriff erfolgt ausschließlich über Zeiger!

```
// Einfacher Zeiger
int* p = NULL;
p = new int;
*p = 42;
cout << "p = " << *p << endl;

// Freigabe
delete p;
```

Haldenspeicher ("Heap")



delete p;

Gibt den Heap
Bereich frei,
kann wieder
überschrieben
werden!



Dynamische Speicherverwaltung - C++



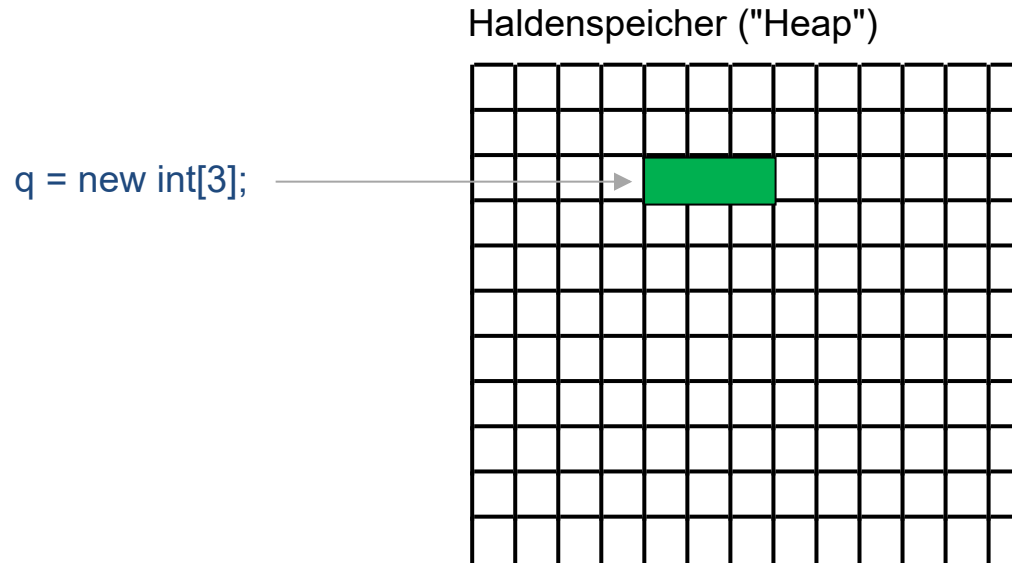
- Zugriff erfolgt ausschließlich über Zeiger!

```
// Einfacher Zeiger
int* p = NULL;
p = new int;
*p = 42;
cout << "p = " << *p << endl;
```

```
// Zeiger / Feld Dualitaet
int* q = new int[3];
```

```
// Freigabe
delete p;
```

```
// Fehler!
cout << "(free'd) p = " << *p << endl;
```



Dynamische Speicherverwaltung - C++



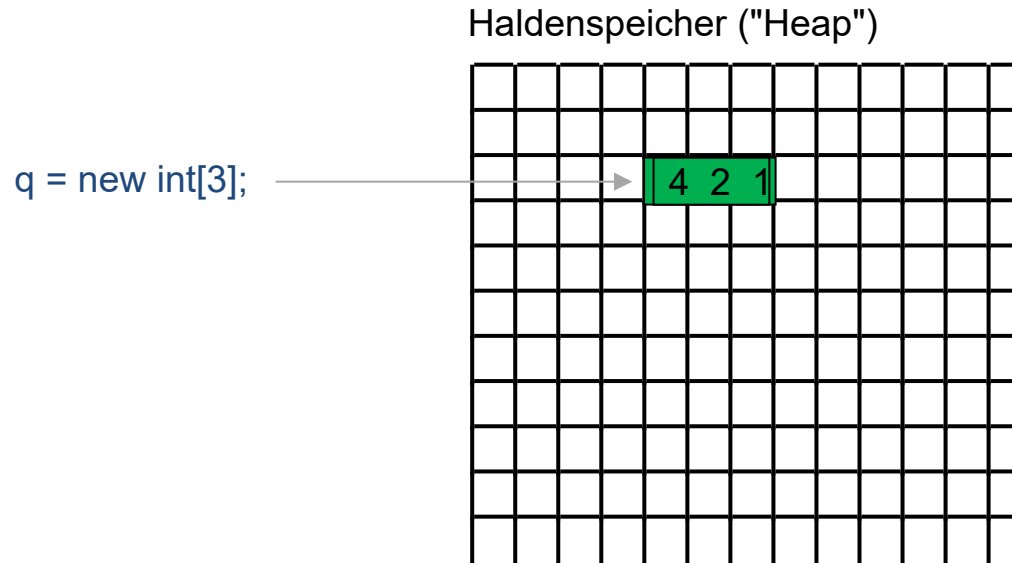
- Zugriff erfolgt ausschließlich über Zeiger!

```
// Einfacher Zeiger
int* p = NULL;
p = new int;
*p = 42;
cout << "p = " << *p << endl;
```

```
// Zeiger / Feld Dualitaet
int* q = new int[3];
q[0] = 4; q[1] = 2; q[2] = 1;
cout << "sum of q = "
    << q[0] + q[1] + q[2]
    << endl;
```

```
// Freigabe
delete p;
```

```
// Fehler!
cout << "(free'd) p = " << *p << endl;
```



Dynamische Speicherverwaltung - C++



- Zugriff erfolgt ausschließlich über Zeiger!

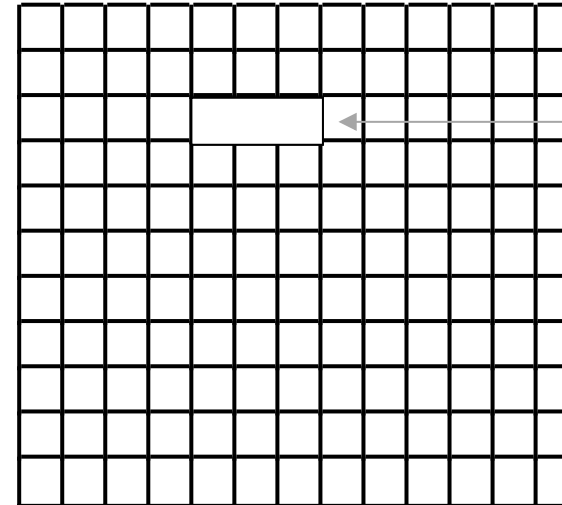
```
// Einfacher Zeiger
int* p = NULL;
p = new int;
*p = 42;
cout << "p = " << *p << endl;

// Zeiger / Feld Dualitaet
int* q = new int[3];
q[0] = 4; q[1] = 2; q[2] = 1;
cout << "sum of q = "
    << q[0] + q[1] + q[2]
    << endl;

// Freigabe
delete p;
delete[] q;

// Fehler!
cout << "(free'd) p = " << *p << endl;
```

Haldenspeicher ("Heap")



delete[] q;

Gibt den Heap
Bereich frei,
kann wieder
überschrieben
werden!

Dynamische Speicherverwaltung - C++



- Zugriff erfolgt ausschließlich über Zeiger!

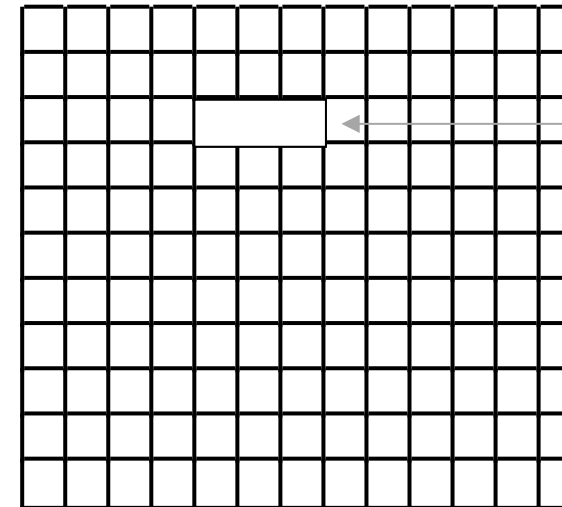
```
// Einfacher Zeiger
int* p = NULL;
p = new int;
*p = 42;
cout << "p = " << *p << endl;
```

```
// Zeiger / Feld Dualitaet
int* q = new int[3];
q[0] = 4; q[1] = 2; q[2] = 1;
cout << "sum of q = "
    << q[0] + q[1] + q[2]
    << endl;
```

```
// Freigabe
delete p;
delete[] q;
```

In Entsprechung zu den Operatoren `new` und `new [ ]` wird zwischen den Operatoren `delete` und `delete [ ]` unterschieden.

Haldenspeicher ("Heap")



`delete[] q;`

Gibt den Heap Bereich frei, kann wieder überschrieben werden!



Hinweis

Eine fehlerhafte `delete`-Anweisung für ein Array (das heißt ohne die eckigen Klammern) resultiert nach dem C++-Standard in »undefiniertem Verhalten« (englisch *undefined behaviour*).

Dynamische Speicherverwaltung - C++



- Zugriff erfolgt ausschließlich über Zeiger!

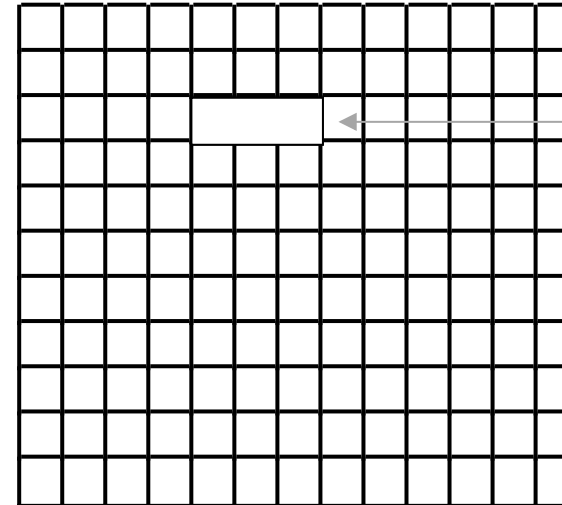
```
// Einfacher Zeiger
int* p = NULL;
p = new int;
*p = 42;
cout << "p = " << *p << endl;

// Zeiger / Feld Dualitaet
int* q = new int[3];
q[0] = 4; q[1] = 2; q[2] = 1;
cout << "sum of q = "
    << q[0] + q[1] + q[2]
    << endl;

// Freigabe
delete p;
delete[] q;

// Fehler!
cout << "(free'd) p = " << *p << endl;
```

Haldenspeicher ("Heap")



delete[] q;

Gibt den Heap
Bereich frei,
kann wieder
überschrieben
werden!

Was wird cout ausgegeben?

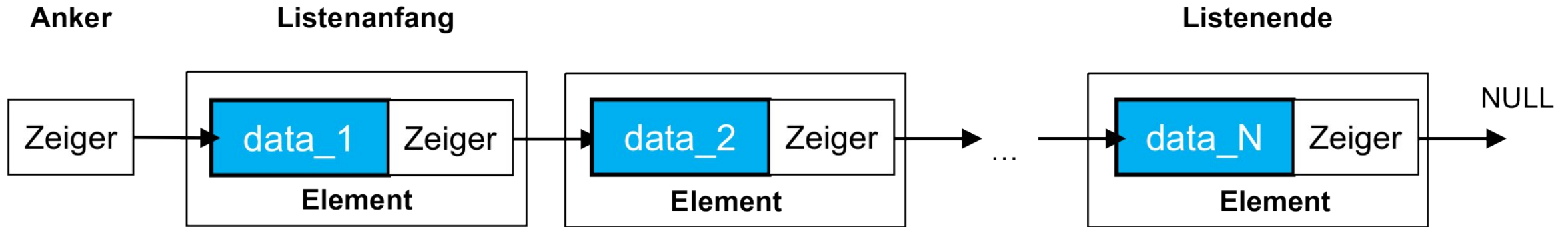


- Strukturierte Datentypen nutzen die dynamische Speicherverwaltung häufig, da:
 - Daten nur temporär genutzt werden,
 - mehrere Datenelemente behandelt werden müssen ("alle Studierenden"),
 - Datenelemente auf andere Elemente verweisen müssen ("Studierender X hat Vorlesungen Y und Z").
- Daher weist eine Software-Applikation häufig ein ganzes **Geflecht von dynamisch allokierten strukturierten Datentypen** auf
- Bemerkung: Das Beherrschen und Debuggen dieser dynamischen Datenstrukturen ist eine **wichtige Fähigkeit für Software-Entwickler & -Entwicklerinnen**.

Listenstruktur - Grundbegriffe



Eine **verkettete Liste** ist eine Datenstruktur, in der Datenelemente vorzugsweise **geordnet** gespeichert sind.



Listenstruktur – Dynamisch!



Eine **verkettete Liste** ist eine **dynamische** Datenstruktur, in der Datenelemente vorzugsweise **geordnet** gespeichert sind.



Dynamische Speicherverwaltung & Strukturierte Datentypen



■ Beispiel in C++:

```
struct Student {  
    string first_name;    // Vorname  
    string last_name;    // Nachname  
    int matric_num = 0;   // Matrikelnummer  
};  
  
Student* QueryStudent(string message)  
{  
    Student* temp = new Student;  
    cout << endl << message << endl;  
    cout << "Vorname>"; cin >> temp->first_name;  
    cout << "Nachname>"; cin >> temp->last_name;  
    cout << "Matrikelnr.>"; cin >> temp->matric_num;  
    return temp;  
}
```

Rückgabe von
allokierten
Datenstrukturen

aus "." wird "->"
für **Zeiger auf
Datenstrukturen**

```
void HeapAndStructure()  
{  
    Student* s1 = QueryStudent("Studierender #1");  
    Student* s2 = QueryStudent("Studierender #2");  
  
    bool s2_younger = s2->matric_num > s1->matric_num;  
    cout << "Studierender #2 ist nach #1 aufgenommen worden: "  
         << s2_younger << endl;  
  
    delete s1, s2;  
}  
  
int main()  
{  
    HeapAndStructure();  
}
```

Speicherlayout



■ "Code-Segment"

```
int global1 = 42;
int* globptr = &global1;

void FuncA(int arg1)
{
    int local_funca = 1001;
}

int main()
{
    int local_main = 13;
    FuncA(42);

    int* feld = new int[1000];
    feld[0] = 17;
    delete feld;

    return (0);
}
```

■ "Stack-Segment"

⋮

■ "Daten-Segment" (globale) Variablen

■ "Heap-Segment"



■ "Code-Segment"

```
int global1 = 42;
int* globptr = &global1;

void FuncA(int arg1)
{
    int local_funca = 1001;
}

int main()
{
    int local_main = 13;
    FuncA(42);

    int* feld = new int[1000];
    feld[0] = 17;
    delete feld;

    return (0);
}
```

■ "Stack-Segment"

⋮

■ "Daten-Segment" (globale) Variablen

Adr	Name	Wert
0x0200 0	global1	42
0x0200 4	globptr1	0

■ "Heap-Segment"



■ "Code-Segment"

```
int global1 = 42;
int* globptr = &global1;

void FuncA(int arg1)
{
    int local_funca = 1001;
}

int main()
{
    int local_main = 13;
    FuncA(42);

    int* feld = new int[1000];
    feld[0] = 17;
    delete feld;

    return (0);
}
```

■ "Stack-Segment"

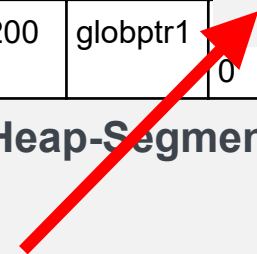
⋮

■ "Daten-Segment" (globale) Variablen

Adr	Name	Wert
0x0200 0	global1	42
0x0200 4	globptr1	0

■ "Heap-Segment"

?





■ "Code-Segment"

```
int global1 = 42;
int* globptr = &global1;

void FuncA(int arg1)
{
    int local_funca = 1001;
}

int main()
{
    int local_main = 13;
    FuncA(42);

    int* feld = new int[1000];
    feld[0] = 17;
    delete feld;

    return (0);
}
```

■ "Stack-Segment"

⋮

■ "Daten-Segment" (globale) Variablen

Adr	Name	Wert
0x02000	global1	42
0x02004	globptr1	0x02000

■ "Heap-Segment"



■ "Code-Segment"

```
int global1 = 42;
int* globptr = &global1;

void FuncA(int arg1)
{
    int local_funca = 1001;
}

int main()
{
    int local_main = 13;
    FuncA(42);

    int* feld = new int[1000];
    feld[0] = 17;
    delete feld;

    return (0);
}
```

■ "Stack-Segment"

⋮

	FuncA()	
0x23018	Lokal: local_funca	1001
0x23014	Argument: arg1	42
0x2300c	Rücksprung-Adresse	0x004f

	main()	
0x23010	Lokal: local_main	13
0x23008	Lokal: feld	0x86345
0x23000	Rücksprung-Adresse	0x23535434

■ "Daten-Segment" (globale) Variablen

Adr	Name	Wert
0x02000	global1	42
0x02004	globptr1	0x02000

■ "Heap-Segment"



■ "Code-Segment"

```
int global1 = 42;
int* globptr = &global1;

void FuncA(int arg1)
{
    int local_funca = 1001;
}

int main()
{
    int local_main = 13;
    FuncA(42);

    int* feld = new int[1000];
    feld[0] = 17;
    delete feld;

    return (0);
}
```

■ "Stack-Segment"

⋮

	FuncA()	
0x23018	Lokal: local_funca	1001
0x23014	Argument: arg1	42
0x2300c	Rücksprung-Adresse	0x004f

	main()	
0x23010	Lokal: local_main	13
0x23008	Lokal: feld	0x86345
0x23000	Rücksprung-Adresse	0x23535434

■ "Daten-Segment" (globale) Variablen

Adr	Name	Wert
0x02000	global1	42
0x02004	globptr1	0x02000

■ "Heap-Segment"

Adr	Name	Wert
0x86345	feld	[17, ..]
..	..	..

- Schreiben Sie ein C++-Programm, welches:
 - die Nutzerin fragt, wie viele Gleitkomma-Zahlen erfasst werden sollen,
 - ein entsprechendes Feld für diese Zahlen dynamisch allokiert,
 - nacheinander diese Zahlen von der Nutzerin erfragt,
 - die Summe der Quadrate dieser Zahlen ausrechnet.

```
// HSKA - Technische Informatik 1
// Strukturierte Datentypen
// Basics
```

```
#include <iostream>
using namespace std;
```

```
void main()
{
```

```
    // Problemgröße handhaben
```

```
    int num;
```

```
    do {
```

```
        cout << "Wieviele Zahlen>";
```

```
        cin >> num;
```

```
    } while (num < 1);
```

```
    // dynamisch allokieren
```

```
    float* zahlen = new float[num];
```

```
    // Erfassung
```

```
    for (int i = 0; i < num; i++)
```

```
    {
```

```
        // Erfassen
```

```
        cout << "Erfassung von Zahl #" << i + 1 << " >";
```

```
        cin >> zahlen[i];
```

```
    }
```

```
    // Berechnung
```

```
    float sq = 0.0;
```

```
    for (int i = 0; i < num; i++)
```

```
        sq += zahlen[i] * zahlen[i];
```

```
    // Ausgabe
```

```
    cout << "Summe der Quadrate = "
```

```
        << sq << endl;
```

```
    // freigeben
```

```
    delete[] zahlen;
```

```
}
```

- Die Funktion „malloc“ (memory allocate) fordert eine Speichermenge aus dem „Heap“ (Speichervorrat des Rechners) an, die Funktion „free“ gibt diesen wieder frei. Die Basisadresse des zugeteilten Speichers wird über einen Zeiger übergeben.

```
i_ptr = (int*)malloc(n*sizeof(int));
```

- Wenn die Speicherzuteilung nicht funktioniert zeigt der übergebene Zeiger nirgendwohin, also auf NULL: `void* malloc(size_t size)`
- Wenn wir den Speicher nicht mehr brauchen, müssen wir ihn wieder freigeben...
`free (i_ptr);`
- ...und den Zeiger auf NULL setzen:
`i_ptr = NULL;`
- Sonst kann er später nicht verwendet werden und wir haben ein „schönes“ Speicherleck (= Sicherheitsproblem) erzeugt.



Dynamische Datenstrukturen

Stack (Stapel)

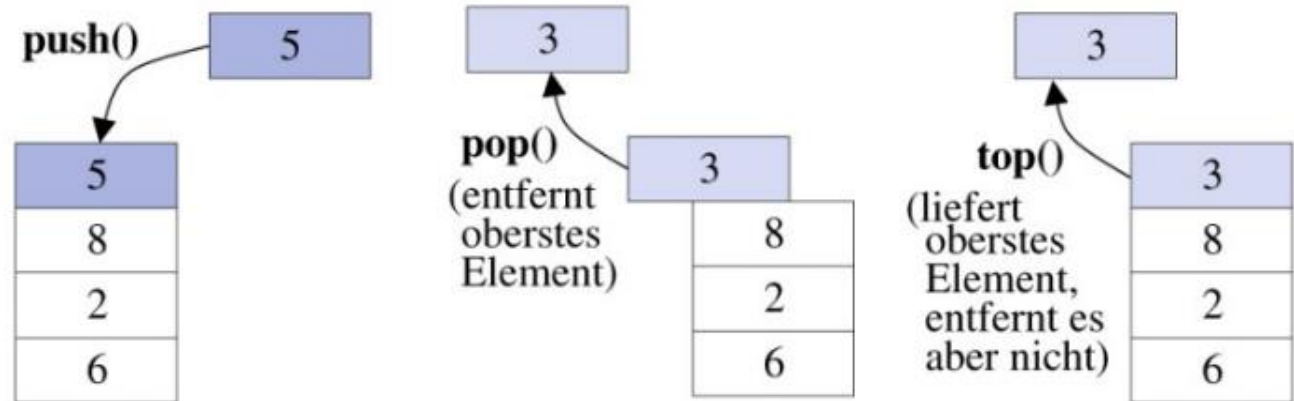


■ **Definition:** Ein Stack (Stapel) ist eine Datenstruktur mit den Operationen Push, Pop und Top, welche nach dem Last-in-First-Out-Prinzip (LIFO) arbeitet. Dies bedeutet, dass das zuletzt eingefügte Element auch wieder als erstes entnommen wird.

- Ist die Anzahl der Elemente im Stack vorab bekannt, kann ein Array als Ausgangsdatenstruktur verwendet werden. Ansonsten eignen sich verkettete Datenstrukturen.

■ Operationen:

- **Push:** Neues Element am Listenanfang einfügen, liegt in $\mathcal{O}(1)$
- **Pop:** Erstes Element wird entnommen und vom Stack gelöscht, liegt in $\mathcal{O}(1)$
- **Top:** Liefert erstes Element ohne es vom Stack zu entfernen, liegt in $\mathcal{O}(1)$



Quelle [1]

Stack (Stapel)



Kartenspiele

- Viele Kartenspiele beruhen auf einem Stapel, der nur die oberste Karte zeigt ("**top()**"). Die Spieler ziehen nacheinander Karten ("**pop()**"). Initial wurde der Stapel aufgebaut ("**push()**").



Tellerstapel beim Abspülen

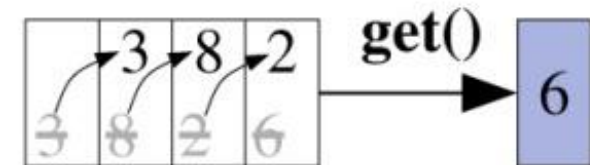
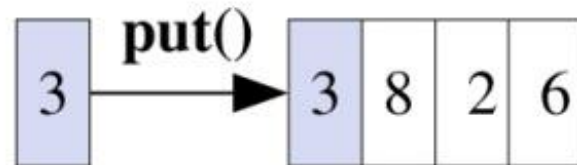
- Beim Abspülen wird durch das Vorspülen ein Tellerstapel aufgebaut ("**push()**"). Nur der oberste Teller kann jeweils gefahrlos entnommen werden ("**pop()**").



Queue (Warteschlange)



- **Definition:** Eine Queue (Warteschlange) ist eine Datenstruktur mit den Operationen Put (Enqueue) oder Get (Dequeue), welche nach dem First-In-First-Out-Prinzip (FIFO) arbeitet. Dies bedeutet, dass neue Elemente nur am Anfang in die Datenstruktur eingefügt werden können, während nur das letzte Element entnommen werden kann.
 - Ist die maximale Anzahl der Elemente im Stack vorab bekannt, kann ein Array als Ausgangsdatenstruktur verwendet werden (Ringpuffer). Ansonsten eignen sich verkettete Datenstrukturen.
- **Operationen:**
 - **Put ("enqueue"):** Neues Element am Schlangenende (Listenanfang) einfügen, liegt in $\mathcal{O}(1)$
 - **Get ("dequeue"):** Element am Listenende entnehmen und liefert es zurück, liegt in $\mathcal{O}(1)$



Quelle [1]

Queue (Warteschlange)



Warteschlange beim Drucker

- Auszudruckende Dokumente verschiedener Programme werden nacheinander in eine Warteschlange gereiht ("enqueue()"). Der Drucker arbeitet jeweils das nächste ab ("dequeue()").



Warteschlangen z.B. in der Apotheke

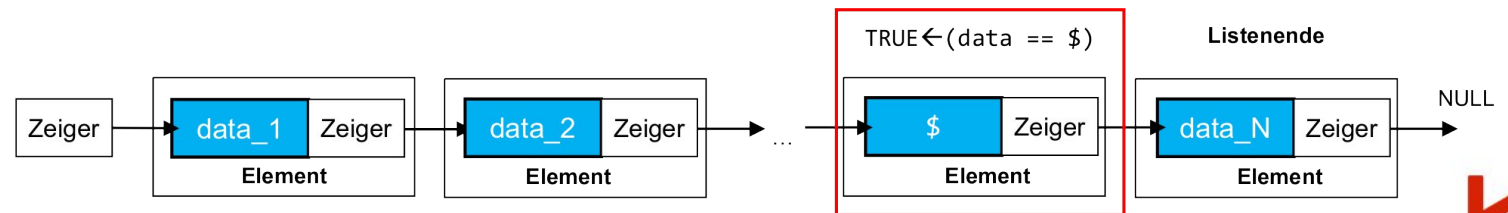
- Ankommende Kund:innen ziehen ein Ticket und reihen sich hinten in die Schlange ein ("enqueue()"). Die jeweils nächste Nummer wird aufgerufen ("dequeue()") und wird bedient.



Verkettete Listen



- **Motivation:** Stack und Queue sind spezialisierte Datenstrukturen. Diese können zu verketteten Listen verallgemeinert werden.
- **Definition:** Eine **verkettete Liste** ist eine dynamische Datenstruktur, in der Datenelemente **geordnet** gespeichert sind. Bei ihrer Erstellung braucht die maximale Anzahl der Elemente nicht festgelegt zu werden, und die Anzahl darf während der Laufzeit beliebig variieren. [1]
- **Operationen:**
 - **Push(), Pop(), Top(), Get() und Tail():** können von Stack, Queue übernommen werden und liegen in $\mathcal{O}(1)$.
 - **Insert():** vor oder nach einem bekannten Element der Liste wird ein Weiteres eingefügt. Liegt in $\mathcal{O}(1)$.
 - **Delete():** ein bekanntes Element der Liste wird aus der Verkettung entfernt. Liegt in $\mathcal{O}(1)$.
 - **Find():** Ist das Element nicht bekannt, muss es durch Durchlaufen der Liste gesucht werden. Liegt damit in $\mathcal{O}(n)$.



Dynamische Bäume & Graphen...





Code – dynamischer Speicher



// dynamisch Speicher nutzen

```
int* ptr_x = new int;  
*ptr_x = 9;  
cout << *ptr_x << endl;  
delete ptr_x;  
cout << *ptr_x << " ... nach Freigabe " << endl;
```

```
int* ptr = new int[5];  
ptr[0] = 7;  
*(ptr + 1) = 8;  
  
delete[] ptr;
```



Code – Verkettete Liste erzeugen/ausgeben



```
struct Element {
    string Name = "Mustermann";
    string VorName = "Max";
    int MatrNummer;
};

int main() {
    Element* Anker = nullptr;
    Element* Hilfzeiger = nullptr;
    int AnzahlStudis;
    cout << "Wieviele Studenten dazufuegen?:\n";
    cin >> AnzahlStudis;
    for (int jj = 0; jj < AnzahlStudis; jj++) {
        Element* Ptr_NeuesElement = new Element;
        Ptr_NeuesElement->MatrNummer = jj;
        if (jj == 0) {
            Anker = Ptr_NeuesElement;
            Hilfzeiger = Ptr_NeuesElement;
        } else {
            Hilfzeiger->next = Ptr_NeuesElement;
            Hilfzeiger = Ptr_NeuesElement;
        }
    }
}
```

```
Hilfzeiger = Anker;
while (Hilfzeiger != nullptr) {
    cout << Hilfzeiger->MatrNummer << " -> ";
    Hilfzeiger = Hilfzeiger->next;
}

return 0;
}
```

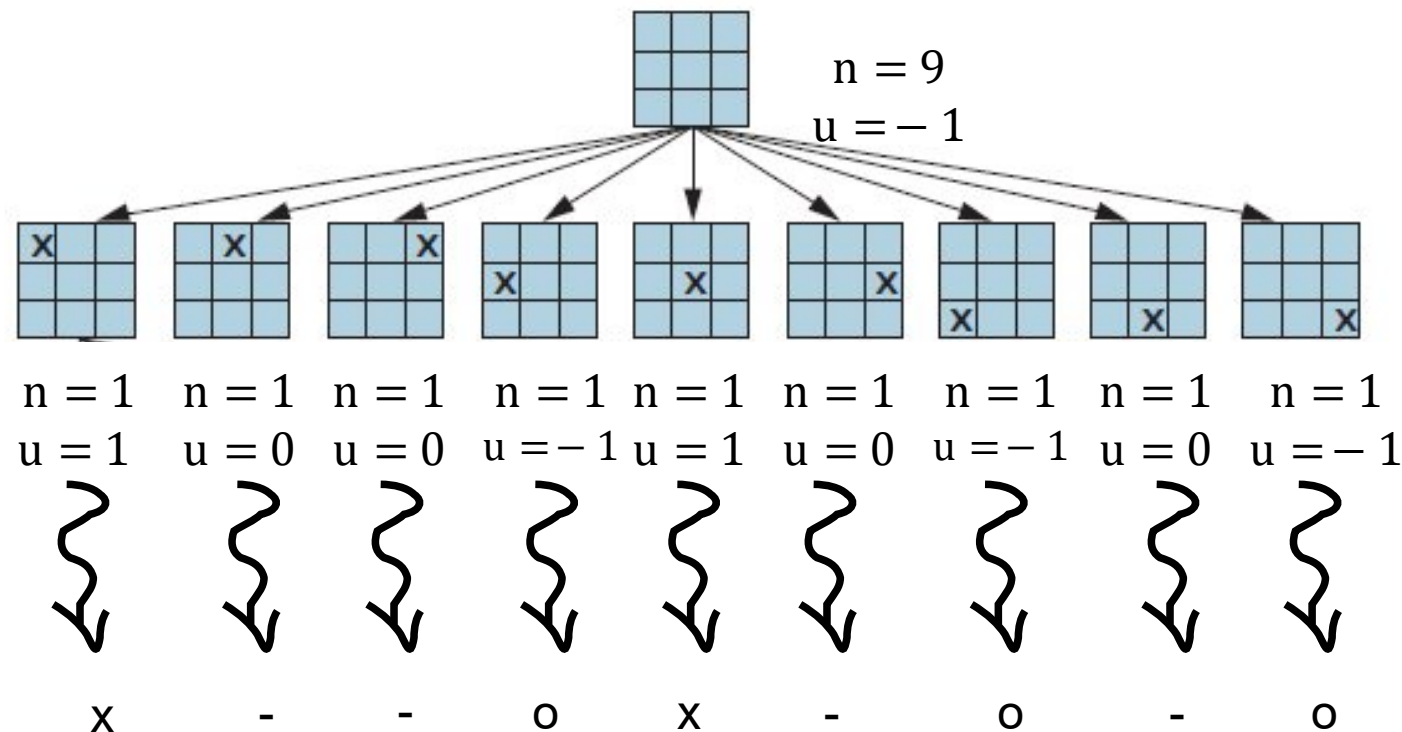


MCTS Beispiel (average utility)



Back-propagation

- Spielergesultis wird verwendet, um alle Knoten des Baumes auf dem Weg upzudaten.



Update

1. Anzahl Spiele durch den Knoten
2. Info Gewinn/Verlust



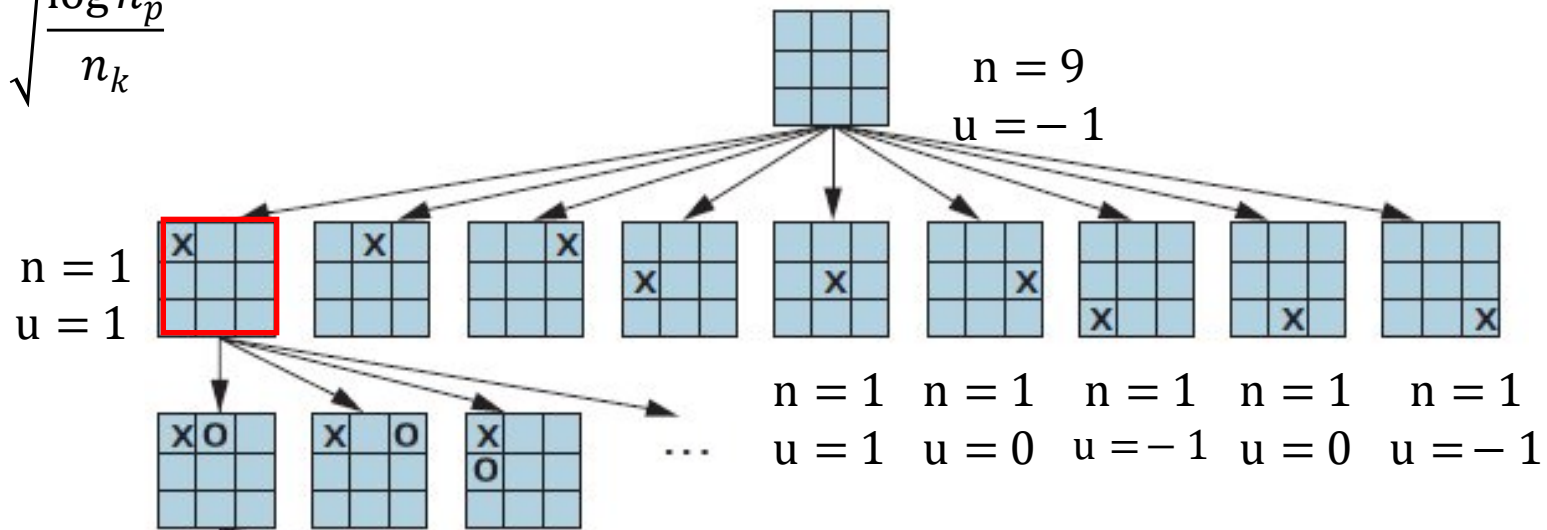
MCTS Beispiel (average utility)



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT

$$UCB_k = v_k + C \times \sqrt{\frac{\log n_p}{n_k}}$$



Update

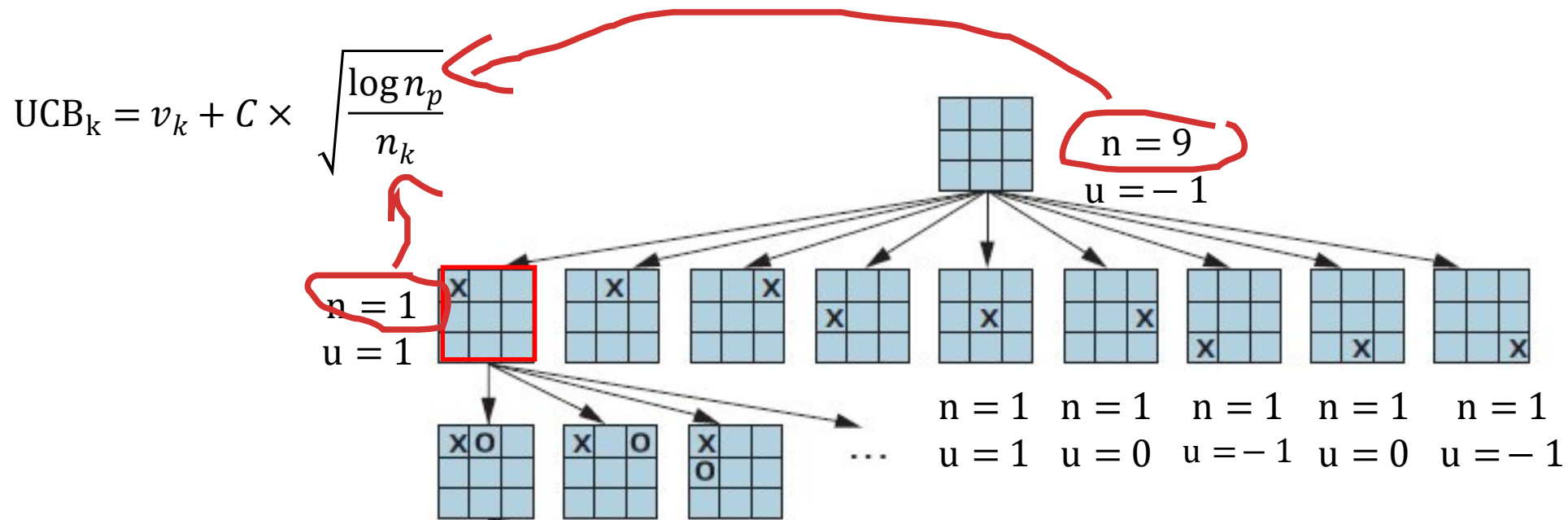
1. Anzahl Spiele durch den Knoten
2. Info Gewinn/Verlust

MCTS Beispiel (average utility)



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT



Update

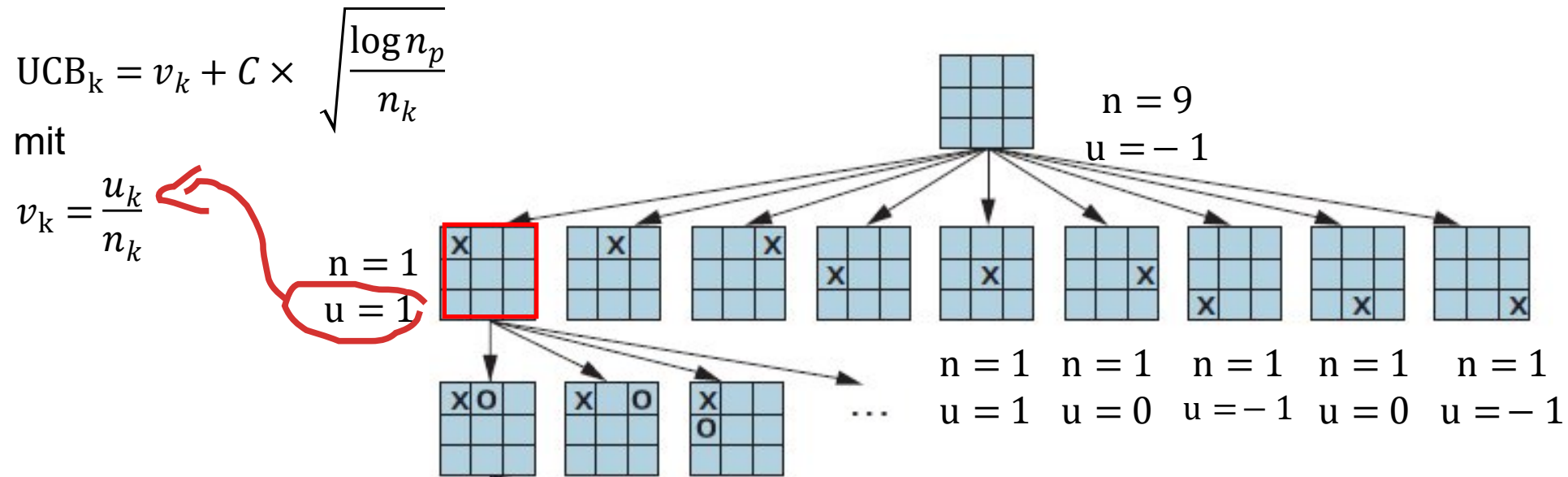
1. Anzahl Spiele durch den Knoten
2. Info Gewinn/Verlust

MCTS Beispiel (average utility)



Selection

- Start mit Root Knoten
- Selektion basierend auf Selektions-Policy (Regel)
- Effektive Policy: „upper confidence bounds applied to trees“ UCT



UCT: wähle den Knoten mit höchstem UCB